

ТЕХНОЛОГИЧНО УЧИЛИЩЕ ЕЛЕКТРОННИ СИСТЕМИ

към ТЕХНИЧЕСКИ УНИВЕРСИТЕТ - СОФИЯ

ДИПЛОМНА РАБОТА

по професия код 481020 “Системен програмист”

специалност код 4810201 “Системно програмиране”

Тема: Проектиране и реализация на LLM система GIANT v2 с разширение Anchor-TiDAR за ускорена генерация

Дипломант: Антон Христов

Дипломен ръководител:

СОФИЯ

2 0 2 6

УВОД

През последните години големите езикови модели (Large Language Models, LLM) се превърнаха в основен компонент в съвременните софтуерни системи за търсене, анализ, програмиране и диалогови интерфейси. Паралелно с това нарасна и нуждата от practically usable open implementation, която да позволява не само обучение и инференс, но и инженерна устойчивост: повторяеми експерименти, работа на ограничен брой GPU ресурси, преносимост между локална и отдалечена среда и ясно структурирани артефакти.

Настоящата дипломна работа е фокусирана върху разработката на системата SUPER-GIANT, състояща се от две основни части:

- базова LLM платформа GIANT/v2 (данни, модел, обучение, инференс);
- разширение TiDAR, което използва базовата инфраструктура и надгражда обучението и декодирането със забързан diffusion-ориентиран подход, базиран на научния труд “Think in Diffusion, Talk in Autoregression” от NVIDIA Research и надграждането му със собствени модификации.

Реализацията е изцяло на Python, върху JAX стак, с фокус върху:

- скалируем data pipeline с шардове и лесна интеграция с големи хъбове за данни като Huggingface или Kaggle;
- стабилно обучение с checkpoint/resume и възстановяване на състоянието на data loader;
- ускорен KV-cache инференс;
- инженерни практики за контейнеризация, синхронизация със S3-съвместимо хранилище и работа с GPU в облака.

Целта на дипломната работа е да се проектира и реализира цялостен програмен продукт за обучение и използване на езиков модел, който едновременно:

- осигурява стабилна базова архитектура за autoregressive обучение (GIANT/v2);
- позволява бързо експериментиране с нови training objectives и decoding схеми (TiDAR);
- остава практически приложим в реални инженерни условия (CI/CD, remote execution, cache/sync, reproducibility).

Основни задачи на дипломната работа:

- проучване на съществуващи LLM системи и инструменти;
- описание на архитектурата, алгоритмите и вътрешния модел на данните;
- реализация на ключовите модули в GIANT/v2 и TiDAR;
- валидиране чрез експерименти, логове, benchmark артефакти и потребителски сценарии.

ПЪРВА ГЛАВА

ПРЕГЛЕД НА СЪЩЕСТВУВАЩИ СИСТЕМИ, ПОДОБНИ НА GIANT, И ИЗВЕСТНИ РАЗВОЙНИ СРЕДСТВА

В тази глава са разгледани решения и инструменти, близки до тематиката на дипломната работа: обучение и ускорен инференс на decoder-only езикови модели.

LLaMA семейство (Meta)

LLaMA моделите се утвърдиха като де факто стандарт за отворени foundation модели. Основните им предимства са добра scaling стратегия, стабилни архитектурни избори (RMSNorm, RoPE, SwiGLU) и голяма екосистема от производни реализации. За проекта GIANT тези модели са референтна точка за архитектурни и обучителни практики.

SmolLM / компактни open модели

SmolLM профилите са важни като practical baseline за експерименти с ограничен GPU бюджет. В TiDAR частта на проекта се използват точно такива размери (135M и 360M), което позволява многократни сравнения между loss конфигурации и decode варианти в разумно време.

nanoGPT / minGPT

Тези проекти показват минимална, ясно четима реализация на autoregressive Transformer. Те са полезни за концептуално разбиране, но не покриват production-like нуждите от sharding, resume, инфраструктура за отдалечен GPU и систематични benchmark отчети.

Megatron-LM и distributed тренировка

Megatron-LM е ориентир за мащабно обучение с моделна/данна паралелизация. В настоящата работа не се гони multi-node distributed setup; фокусът е единичен GPU workflow с максимална инженерна практичност. Въпреки това принципите за разделяне на data/model/optimizer отговорности са сходни.

vLLM и TensorRT-LLM

Тези системи са водещи при serving/latency оптимизации и силно ефективно управление на KV-cache. В GIANT/TiDAR част от идеите за cache политика и throughput benchmarking са приложени в custom JAX стек, като изследователският фокус е върху Anchor-TiDAR декодиране и loss-driven подобрения.

Python

Изборът на Python е определен от бързия цикъл за изследователска разработка, широка библиотечна поддръжка и добра интеграция с ML инструменти, cloud execution и автоматизация.

JAX + Flax

JAX е избран заради XLA компилацията, контрол върху устройството и възможност за високопроизводителни JIT графи. Flax осигурява структурирано model definition API и variable collections, критични за KV-cache и за чисто разделяне на train/inference състояние.

Optax + Orbach

Optax е използван за optimizer pipeline (clip + AdamW + scheduler), а Orbach - за асинхронни mini checkpoint-и. Тази комбинация подобрява устойчивостта при long-running експерименти и намалява риска от загуба на прогрес при прекъсване.

HuggingFace Hub / Datasets / Transformers

Data pipeline частта използва datasets за ingest на публични корпуси (streaming и non-streaming режими), а transformers - за tokenizer интеграция. Това позволява стабилен и възпроизводим preprocess workflow към Arrow шардове.

Docker, Tailscale и S3 инструменти

Инфраструктурният слой в CI/CD/ дава:

- Docker образи с готов CUDA/JAX стек;
- Tailscale-based secure remote достъп;
- S3 операции чрез `s5cmd wrapper`;
- скриптове за бърз sync на локални промени към remote GPU среда.



Figure 1: Контейнерна среда за reproducible изпълнение

ВТОРА ГЛАВА

ФУНКЦИОНАЛНИ ИЗИСКВАНИЯ, АРГУМЕНТАЦИЯ НА ИЗБОРА НА СРЕДСТВА И ФОРМАЛЕН МОДЕЛ

2.1 Функционални изисквания

2.1.1 Входни източници и ingest pipeline

Системата трябва да поддържа ingest на текстови корпуси от:

- HuggingFace datasets (streaming и non-streaming);
- локални JSON/JSONL директории;
- chat структурирани източници с role/content полета.

Изискването включва нормализация, токенизация и поддръжка на различни stage профили в единен pipeline.

2.1.2 Подготовка на корпус, токенизация и stage-и

Системата работи с training stages и sequence потоци. Необходимо е:

- последователно изпълнение на множество stages;
- различни seq_len и fraction по stages;
- прозрачен преход между stages при запазване на optimizer/loader състояние.

2.1.3 Конфигурационно управление на експерименти

Конфигурационно редактиране на експерименти.

- глобални настройки в Global_Config.yml;
- локални model/training настройки в Config.yml;
- stage-level overrides за loss коефициенти и dataset дялове;
- CLI overrides за бързи промени без пренаписване на конфиг.

2.1.4 Мониторинг и метрики по време на изпълнение

Системата трябва да предоставя оперативна телеметрия:

- loss и допълнителни метрики по стъпки;
- acceptance статистики при TiDAR;
- checkpoint metadata за training/eval;
- benchmark отчети в Typst/PDF формат.

2.1.5 Управление на хиперпараметри и decode политика

Управление на ключови параметри на модела и декодирането:

- embedding_size, num_heads, num_kv_heads, num_layers;03-
- compute_dtype, param_dtype;
- context length и KV-cache policy;
- TiDAR draft_length, bias стойности и decode режим.

2.1.6 Артефакти, checkpoint-и и отчетност

Системата трябва да експортира и съхранява:

- dataset шардове + manifest + статистики;
- checkpoints + optimizer/dataloader state;
- инференс и benchmark артефакти;
- анализи и документация в .md, .typ, .pdf.

2.2 Аргументация за избор на езика и софтуерните средства

2.2.1 Избор на езика за програмиране

Python е избран поради:

- бърз експериментален цикъл;
- силна ML екосистема;
- лесна интеграция с tooling за инфраструктура и автоматизация.

2.2.2 Избор на платформа за разработка

Избран е GPU-ориентиран модел на работа, защото паралелните изчислителни възможности на съвременните видеокарти са ключови за машинното обучение и на практика се използват постоянно при обучение и инференс на LLM модели. Тъй като не разполагам със собствена GPU, разработката е организирана като локална среда + remote GPU изпълнение чрез Docker. Това позволява според задачата да се наемат по-мощни видеокарти за тежки обучения или по-малки и по-евтини карти за бърза експериментация.

2.2.3 Избор на фреймуърк за интерфейс

Няма графичен интерфейс; системата е CLI-first. Причините са:

- директен контрол върху експериментите;
- scriptable workflow, по-лесен за работа през shell/vim/ssh;
- проектът е предимно научен, така че няма нужда от GUI слой.

2.2.4 Избор на изчислителен фреймуърк

Изборът на JAX е направен с performance-first mindset. Основното предимство е, че чрез jit и XLA компилация изчислителният граф се оптимизира за конкретни статични размерности и се получава kernel fusion на последователни операции, което намалява overhead-a и подобрява реалната производителност при training и inference. Функционалният стил и immutable моделът на данните правят изпълнението по-предвидимо и улесняват агресивните оптимизации от компилатора.

JAX е и достатъчно лек като рамка спрямо по-тежки all-in-one подходи: дава силен autograd (grad, value_and_grad, vmap, lax.scan), без да скрива прекалено много от вътрешната логика. Това е особено подходящо за бързо прототипиране с фокус върху производителност и за DIY проект като настоящия, където целта е не само да се ползват готови компоненти, а и да се разбира и имплементира значителна част от механиката.

Flax е избран като надстройка над JAX за структуриране на модела: модулна дефиниция на слоеве, ясна работа с параметри и non-parameter state, и удобна интеграция с JIT/XLA pipeline-a. Комбинацията JAX + Flax осигурява добър баланс между ниско ниво на контрол, висока производителност и практична скорост на разработка.

2.2.5 Избор на optimizer/checkpoint инструменти

Optax покрива optimizer/schedule нуждите, а Orbx е използван за mini checkpoint устойчивост. Тази комбинация е критична при дълги run-ове с риск от прекъсване.

2.2.6 Избор на средства за deployment и синхронизация

Използвани са Docker, Tailscale, s5cmd и custom scripts (sync-gpu.py). Така се осигурява практичен MLOps workflow за прехвърляне на данни, remote достъп и reproducible execution.

2.3 Структура на данните и вътрешен модел на системата

Системата използва файлово-ориентиран модел вместо релационна база. Основният работен формат за обучителните данни са Arrow shard файлове с manifest и статистики, които са оптимизирани за последователно четене от data loader-a. За пренос и синхронизация между среди се използва S3-съвместим storage workflow; JSON/TXT се ползват само като входен или междинен формат в етапа на ingest, а не като основен storage backend.

2.3.2. Клас на времевата линия

“Времевата линия” в този проект е curriculum от stages:

```
@dataclass
class StageConfig:
    name: str
    dataset: str
    seq_len: int
    epochs: int
    end_ratio: float
```

2.3.3. Конфигурационни домейни на системата

Конфигурационните домейни са model, optimizer, training, inference, tidar, комбинирани чрез merge на глобална и локална конфигурация.

2.3.4 Класове за клипове

Единичният dataset запис в Arrow шард е фиксиран по дължина и съдържа:

- input_ids;
- length;
- loss_mask.

StageDataLoader преобразува тези полета до input/target/mask батч структура.

2.3.5 Класове за източници

StageSourceCfg описва source типа, полетата за текст, chat mapping и streaming поведение. Това позволява единна обработка на различни входни формати.

2.4 Цялостна архитектура и структурна организация

Системата е разделена на три слоя:

- интерфеен слой (CLI + YAML);
- управляващи модули (pipeline, training, inference, checkpoints);
- математически модел (Transformer + TiDAR разширения).

Потребителски интерфейс

Потребителят работи чрез entry point скриптове и конфигурации. Това позволява пълна проследимост на параметрите в експериментите.

Управляващи модули (мениджъри)

TiDAR преизползва тези мениджъри от GIANT (data pipeline, checkpointing, част от inference/runtime слоя) и ги надгражда със специализирана логика за dual-regime обучение и Anchor-TiDAR декодиране.

Ключовите мениджъри са:

- dataset builder (build_corpus.py);
- trainer (Run_training.py);
- checkpoint manager (checkpoint_manager.py);
- inference runtime (jit_inference.py, Generate_faster.py, TiDAR/model/inference.py).

2.5 Основен математически модел на трансформера

2.5.1 Теоретична отправна точка: Attention Is All You Need

Теоретичната основа на системата е архитектурата Transformer, въведена от Ashish Vaswani и съавтори в статията “Attention Is All You Need” (2017). Ключовата идея е да се замени рекурентната обработка с attention механизъм, който позволява паралелна обработка на целия токенен контекст. Това е фундаментално за LLM, защото training и inference трябва да използват максимално добре паралелизма на GPU.

В оригиналния формализъм attention операторът е:

$$A(Q, K, V) = S \left(\frac{QK^T}{\sqrt{d_k}} + M \right) V$$

където Q, K, V са query/key/value матрици, M е маска, а S е softmax функцията.

$$S(z_i) = \frac{\exp(z_i)}{\sum_j \exp(z_j)}$$

Формулировката от оригиналния текст (както е изписана в paper-а, без експлицитно показана маска) е:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

Multi-head формата (както е на фигурата) е:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O$$

където:

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

Еквивалентно, в компактна форма:

$$H(Q, K, V) = [h_1 \parallel \dots \parallel h_n] W^O, h_i = A(QW_i^Q, KW_i^K, VW_i^V)$$

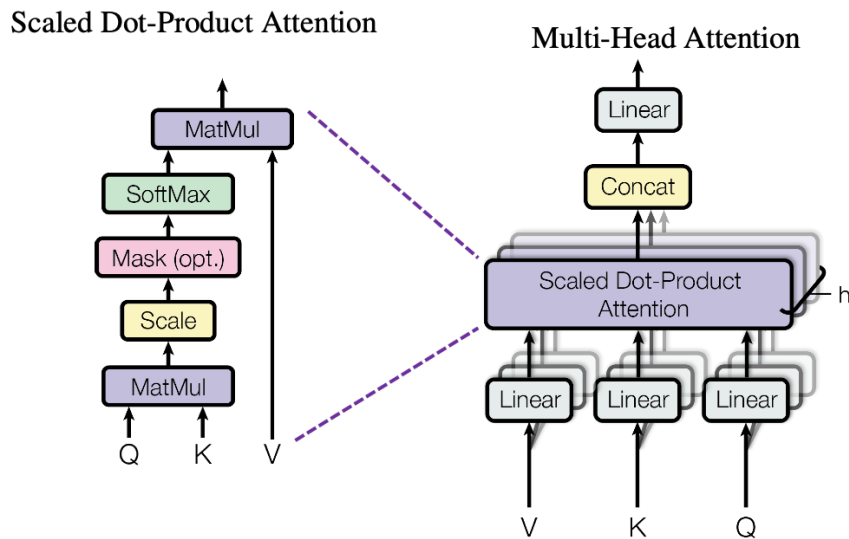


Figure 2: Multi-head attention: блокова схема и базови формули

2.5.2 От класически Transformer към decoder-only LLM

Оригиналната работа описва encoder-decoder архитектура. В GIANT се използва decoder-only вариант, подходящ за next-token prediction. Практически това означава:

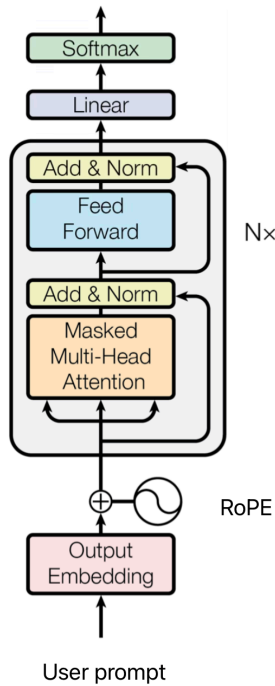
- причинно (causal) маскиране в self-attention;
- autoregressive objective за следващ token;
- изцяло еднопосочен токенов поток при генерация.

Един блок в decoder-only варианта може да се запише като:

$$x' = x + H(N(x), N(x), N(x))$$

$$y = x' + F(N(x'))$$

където N е нормализация, а F е feed-forward модул.



Фигура: Decoder Transformer архитектура

Диаграмата показва базовия поток в decoder-only модел: входни токени -> embedding -> последователност от Transformer блокове -> нормализация -> проекция към логити. В GIANT тази структура се реализира с модерни компоненти като RoPE, RMSNorm и SwiGLU, а при инференс се допълва с KV cache за ускорение.

Изборът на тези “модерни попълнения” е направен като естествена еволюция спрямо оригиналния Transformer от 2017. В **Attention Is All You Need** се използват LayerNorm (а не BatchNorm), ReLU-базиран feed-forward блок и синусоидални позиционни вектори. В настоящата реализация са предпочетени RMSNorm (по-лек и стабилен при decoder-only LLM), SwiGLU (по-добра експресивност и практическа ефективност спрямо базовите активации) и RoPE (по-устойчива позиционна индукция при по-дълги контексти). Така архитектурата запазва теоретичната основа от 2017, но е адаптирана към съвременния LLM performance профил и в практиката е по-близка до модерен LLaMA-style decoder stack.

2.5.3 Нормализация, активации и позиционна информация в GIANT

Вместо класически LayerNorm, в модела се използва RMSNorm, което е по-леко и работи стабилно в големи decoder-only конфигурации; в този смисъл архитектурата на GIANT е по-близка до съвременен LLaMA-подобен дизайн, отколкото до оригиналния Transformer от 2017.

$$R(x) = \frac{x}{\sqrt{\frac{1}{d} \sum_{i=1}^d x_i^2 + \epsilon}} * g$$

Feed-forward частта използва SwiGLU тип гейтинг:

$$G(x) = s(xW_u) * (xW_v), \quad s(t) = t\sigma(t)$$

Позиционната информация се подава чрез RoPE, което позволява по-добро поведение при по-дълги контексти и е стандартен избор в съвременните decoder-only LLM реализации.

2.5.4 Обучителна цел (AR) и връзка с TiDAR

Базовата objective функция за GIANT е autoregressive cross-entropy:

$$L_1 = -\frac{1}{N} \sum_{b,t} m_{b,t} \log p_{\theta}(x_{b,t+1} | x_{b,\leq t})$$

където L_1 е AR (next-token) cross-entropy.

Примерен изход от GIANT v2 checkpoint, трениран върху корпус със силно Wikipedia присъствие (заедно с други източници), е:

User: What is the capital of France?

Assistant: The capital of France is sometimes called Paris.<EOS>

- размер на модела: приблизително 101 милиона параметъра;
- обем на обучението: приблизително 2 милиарда токена.

За мащабно сравнение:

- Llama 3 70B - trained on 15T tokens;
- DeepSeek R1 671B - trained on 14.8T tokens.

TiDAR надгражда този математически модел, като преизползва същата Transformer основа и добавя diffusion-режим на внимание, специални маски и допълнителни loss термини за съгласуване между AR и Diff логити.

2.6 Спекулативно декодиране (въведение преди TiDAR)

Преди представянето на TiDAR е важно да се въведе идеята за speculative decoding, защото именно тя е базата, върху която стъпва по-късното Anchor-TiDAR разширение.

Класическото autoregressive декодиране генерира по един токен на стъпка. Това е коректно, но често не използва оптимално паралелните възможности на GPU при inference.

Спекулативното декодиране цели да увеличи throughput, като предлага предварителни токени (draft), които после се верифицират и частично или изцяло се приемат. Така за една итерация може да се валидират няколко бъдещи позиции вместо само една.

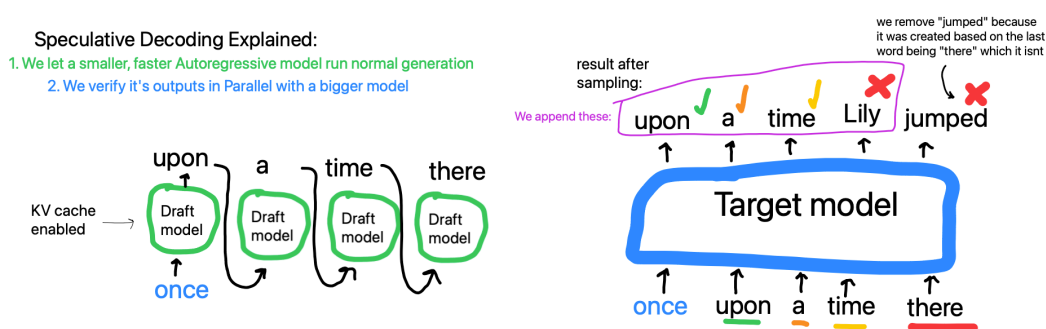


Figure 3: Класически speculative decoding: draft + verify схема

Основната идея е:

- draft е бърза хипотеза за следващите токени;
- verify проверява хипотезата спрямо по-точен/референтен изход;
- приемането на префикс от draft-а води до повече от +1 токен прогрес на итерация.

Тази схема е важна за проекта, защото:

- дава теоретична рамка за ускорение на генерацията;
- естествено се комбинира с KV cache;
- подготвя прехода към TiDAR, където verify и predraft логиката се реализират в един специализиран forward pass.

Важно разграничение: класическите speculative decoding методи обикновено използват втори, по-малък draft модел (например по-малка Llama спрямо по-голям verify модел, напр. 3B срещу 70B) или добавят допълнителни архитектурни компоненти върху базовия модел (например допълнителни глави/слоеве, както при EAGLE).

2.6.1 Защо single-user decode не използва добре GPU

При decode с един потребител и малък effective batch GPU често е ограничен от memory bandwidth, а не от чиста аритметична мощност. Понеже един достъп до HBM/GDDR паметта е сравнително скъп, целта е с едно четене на параметри (например K/V матрици) да се направят повече умножения върху повече токени, т.е. по-голям ефективен batch и по-висока заетост на Tensor Core блоковете.

В проекта се използва и FlashAttention (cuDNN backend), което намалява IO overhead в attention стъпката и допълнително подпомага този подход.

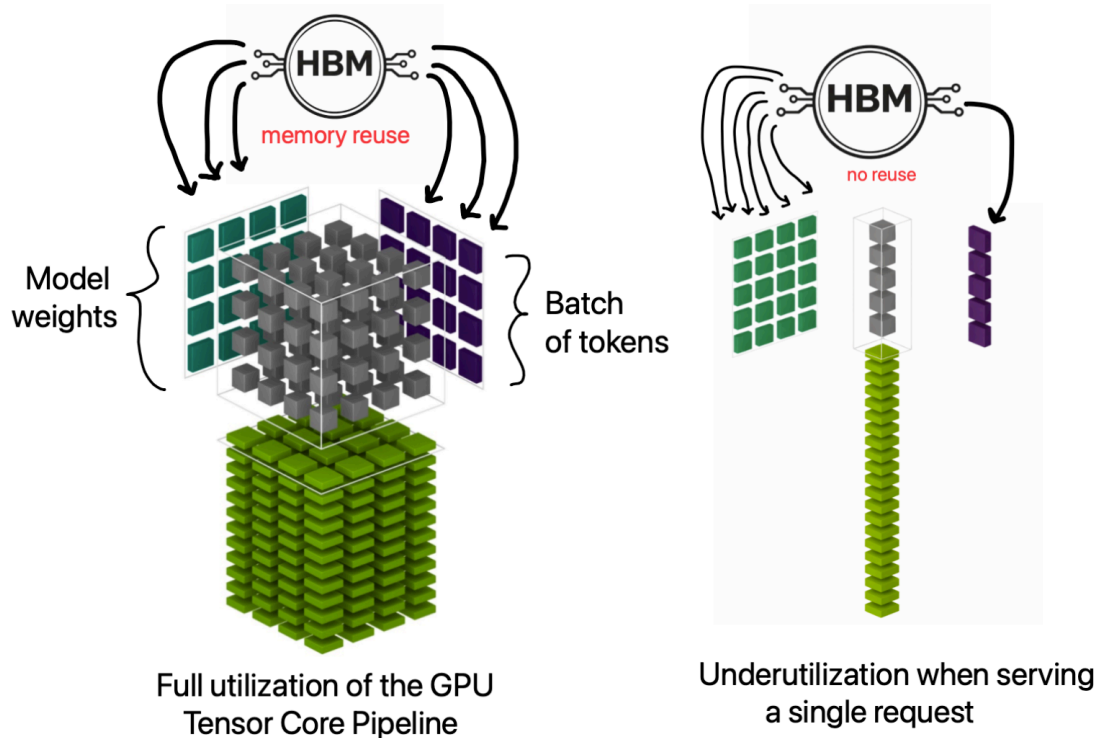


Figure 4: Tensor Core utilization при memory-bound single-user decode

Тази мотивация е важна за разбирането на TiDAR и феномена “Free Token Slots”, където допълнителната паралелна работа в една decode стъпка се използва по-ефективно.

ТРЕТА ГЛАВА

РЕАЛИЗАЦИЯ НА ПРИЛОЖЕНИЕТО

3.1 Реализация на потребителски интерфейс

3.1.1 Основни входни точки (CLI)

Системата е CLI-first и основният интерфейс е набор от entry points:

- GIANT/v2/data_pipeline/build_corpus.py
- GIANT/v2/model/Run_training.py
- GIANT/v2/model/Evaluate.py
- GIANT/v2/model/Generate_faster.py
- GIANT/v2/model/Chat.py
- TiDAR/model/Run_training.py
- TiDAR/model/inference.py

Тези скриптове формират основния интерфейс на системата и покриват пълния цикъл: data prep, обучение, оценка и инференс.

3.1.2 Наблюдение и визуализация на резултати

Визуализацията се реализира чрез логове и експериментални отчети:

- training логове (loss, ppl, accept, greedy_accept);
- checkpoint metadata (train_loss, val_loss, val_ppl);
- Typst отчети с throughput и policy сравнения.

Този подход е избран целенасочено за terminal/vim workflow и за лесно remote наблюдение през SSH.

3.1.3 Управление на stage прогреса

“Времевата линия” в проекта е stage curriculum. Прогресът се движи по stages с различен dataset и контекстна дължина, като се пази възстановимо състояние за всеки етап.

3.1.4 Управление на параметри

Управлението на параметрите е изнесено в YAML конфигурации и CLI flags. Това позволява силно контролирани експерименти с минимални кодови промени.

3.1.5 Добавяне на входни корпуси

Pipeline-ът поддържа HuggingFace и локални JSON/JSONL входи, chat formatting и loss-mask логика.

3.1.6 Експорт на артефакти

Експорт на артефакти към файловата система/S3:

- Arrow shards;
- manifests/statistics;
- checkpoints;
- benchmark logs;
- Typst/PDF отчети.

3.1.7 Примерен training run в терминал

Следният пример показва типично стартиране на обучение през CLI интерфейса:

```
python GIANT/v2/model/Run_training.py \
  --config GIANT/v2/model/Config.yml \
  --global_config GIANT/v2/Global_Config.yml \
  --checkpoint_dir giant-data/GIANT/v2/checkpoints \
```

```
--checkpoint_every 500 \  
--scan_chunk 8
```

Пример за възстановяване на последната налична checkpoint точка:

```
python GIANT/v2/model/Run_training.py \  
--config GIANT/v2/model/Config.yml \  
--global_config GIANT/v2/Global_Config.yml \  
--checkpoint_dir giant-data/GIANT/v2/checkpoints \  
--resume latest
```

Примерен терминален изход при старт от нула (съкратен):

```
[loader] loading shards from /proj/giant-data/GIANT/v2/processed/stage_base  
...  
[loader] stage=base rows=12800000/12800000 ctx=1024 steps_per_epoch=6250  
  
→ Stage base: seq_len=1024 epochs=1 steps=6250 (resume at 0)  
step    100/10449 | stage base          loss 4.8921 ppl 133.21 (18.4s)  
step    200/10449 | stage base          loss 4.6017 ppl 99.66 (17.9s)  
...  
step    500/10449 | stage base          loss 4.2143 ppl 67.64 (17.2s)  
[metadata] Wrote train_loss=4.214300 to checkpoint .../params/step_0000500.npz  
📁 checkpoint → .../params/step_0000500.npz
```

Примерен терминален изход при --resume latest (примерно от стъпка 5432):

```
↪ Resumed from mini checkpoint at step 5432  
► Restored dataloader state at stage 0 step 5432  
  
→ Stage base: seq_len=1024 epochs=1 steps=6250 (resume at 5432)  
step    5500/10449 | stage base          loss 3.9981 ppl 54.50 (16.9s)  
...  
→ Stage wiki: seq_len=1024 epochs=1 steps=4199 (resume at 0)  
step    6400/10449 | stage wiki          loss 3.7129 ppl 40.97 (16.8s)  
...  
✓ Training complete. Final checkpoint: .../params/step_0010449.npz
```

Примерен multi-stage TiDAR training run с различни loss конфигурации по етапи:

```
python TiDAR/model/Run_training.py \  
--config TiDAR/model/training_configs/Greedy_exp_135m.yml \  
--global_config TiDAR/Global_Config.yml \  
--checkpoint_dir giant-data/TiDAR/checkpoints \  
--checkpoint_every 400 \  
--resume latest
```

```
[loader] loading shards from /proj/giant-data/TiDAR/processed/stage_base  
...  
[loader] stage=base rows=6200000/6200000 ctx=1024 steps_per_epoch=3027  
[loader] loading shards from /proj/giant-data/TiDAR/processed/stage_chat  
...  
[loader] stage=chat rows=1800000/1800000 ctx=1024 steps_per_epoch=878  
  
→ Stage base: seq_len=1024 epochs=1 steps=3027 (resume at 0)
```

```

step      1/3905    | stage base                loss 5.1023 ar 4.8122 diff 5.2761
accept 0.117 greedy_acc 0.081 (19.2s)
step     200/3905    | stage base                loss 4.4311 ar 4.1926 diff 4.5214
accept 0.223 greedy_acc 0.147 hard 3.8872 (17.4s)
...

→ Stage align_kl: seq_len=1024 epochs=1 steps=1000 (resume at 0)
step     3200/3905    | stage align_kl            loss 3.9027 ar 3.7710 diff 3.9488
accept 0.356 greedy_acc 0.244 kl_fwd 0.1821 kl_rev 0.0964 distill 0.1418 topk 0.0889
(16.9s)
step     3600/3905    | stage align_kl            loss 3.7814 ar 3.6562 diff 3.8241
accept 0.391 greedy_acc 0.269 kl_fwd 0.1499 kl_rev 0.0792 distill 0.1184 topk 0.0711
(16.6s)
...

→ Stage chat: seq_len=1024 epochs=1 steps=878 (resume at 0)
step     3900/3905    | stage chat                loss 3.7442 ar 3.6214 diff 3.7811
accept 0.405 greedy_acc 0.281 hard 3.1027 distill 0.1021 (16.4s)

[metadata] Wrote train_loss=3.744200 to checkpoint ../params/step_0003905.npz
✓ Training complete. Final checkpoint: ../params/step_0003905.npz

```

3.2 Реализация на основните системни модули

3.2.1 ConfigLoaderManager

Конфигурациите се зареждат чрез `merge` на глобален и локален YAML, след което се резолват относителните пътища спрямо `data_root`. Това осигурява еднакъв кодов път за локално и `remote` изпълнение.

```

def load_configs(config_path=None, global_config_path=None):
    local_cfg = OmegaConf.load(config_path)
    global_cfg = OmegaConf.load(global_config_path)
    cfg = OmegaConf.merge(global_cfg, local_cfg)

    # Унифициране на пътищата спрямо data_root
    cfg.paths.processed_data_root = resolve_path(cfg.paths.processed_data_root,
    cfg.paths.data_root)
    cfg.paths.logs_root = resolve_path(cfg.paths.logs_root, cfg.paths.data_root)
    return cfg

```

3.2.2 TrainLoopOrchestrator

`Run_training.py` реализира `scan`-базиран `training loop` с `gradient accumulation`, `finite checks` и `checkpoint` политика.

3.2.2.1 Основен JIT training loop (`_run_chunk + lax.scan`)

Сърцето на изпълнението е `_run_chunk(...)`, където `batch`-овете се обработват със `lax.scan`, пресмятат се `loss/grad` стойности и се правят проверки за `non-finite` стъпки.

`lax.scan` е JAX примитив за “компилиран `for`-цикъл”: вместо Python да `dispatch`-ва всяка стъпка поотделно, целият цикъл се представя като една JIT-оптимизирана граф операция с `carry` състояние (например `params`, `opt_state`, `accum_grads`, `accum_count`) и последователност от изходи (`losses`). Това намалява `host overhead` и дава по-стабилна производителност при дълги `training run`-ове.

3.2.2.2 Обработка на сигнали и контролирано спиране

Скриптът обработва `SIGINT/SIGTERM` и спира контролирано след текущия `chunk`, за да не се загуби междинно състояние. Практически се поддържат два слоя на устойчивост:

- големи checkpoint-и (params + optimizer + dataloader state), които са “стабилните” запазвания и се пазят;
- мини checkpoint-и, които се записват периодично за бързо възстановяване при внезапно прекъсване (preemption, срив, рестарт).

Така при неочаквано прекъсване може да се продължи от почти последната стъпка, а при нормален stop/етапен преход остават и пълните checkpoint-и за дългосрочно съхранение и проследимост. В практиката training run-овете се поддържат през tmux (включително в CI/CD workflow), което позволява стабилна сесия, лесно повторно закачане към машината и безопасно дълго изпълнение.

3.2.2.3 Gradient accumulation и update политика

Обучението поддържа gradient_accumulation; update се прилага само при достигане на зададения accumulation праг, а остатъчните градиенти се доизчистват при край на stage.

```
# Натрупване на градиенти
accum_grads = tree_map(lambda a, g: a + g, accum_grads, grads)
accum_count = accum_count + 1

# Update само когато достигнем gradient_accumulation
should_update = accum_count == grad_accum
if should_update:
    grads_scaled = tree_map(lambda g: g / grad_accum, accum_grads)
    updates, opt_state = optimizer.update(grads_scaled, opt_state, params)
    params = optax.apply_updates(params, updates)
    accum_grads = tree_map(jnp.zeros_like, accum_grads)
    accum_count = 0
```

На практика това е логика за “мини batch-ове” на ниво update. Вместо целият глобален batch да се побере наведнъж в паметта, той се разделя на няколко по-малки стъпки (micro-batches), които GPU може да обработи с наличния VRAM. Градиентите от тези micro-batches се натрупват и optimizer update се прави накрая, така че се получава ефект на по-голям глобален batch без изискване за голяма еднократна памет.

Този подход е особено важен при по-слаби GPU конфигурации и може да се разшири директно към multi-GPU обучение, където освен локалното accumulation се добавя и синхронизация/редукция между устройствата преди финалния update.

3.2.2.4 Периодично логване на метрики

На всеки log_every стъпки се записват global_step, stage, loss и допълнителни показатели (напр. ppl, accept, greedy_acc, KL/hard/distill/top-k при TiDAR).

Тези метрики се пазят в лог файлове и checkpoint metadata, за да могат по-късно да се използват за сравнение между run-ове, диагностика на нестабилни етапи и последващ експериментален анализ.

3.2.2.5 Resume от checkpoint и dataloader state

Възстановяването зарежда параметри, optimizer state и dataloader прорпес (stage_index, stage_step_total, stage_states), така че обучението да продължи от последната консистентна точка.

```
@jax.jit
def _run_chunk(params, opt_state, batch_chunk, start_step, accum_grads, accum_count):
    # Една компилирана единица работа: loss/grad + accumulation + update
    (params, opt_state, _, accum_grads, accum_count), losses = jax.lax.scan(
        body, (params, opt_state, start_step, accum_grads, accum_count), batch_chunk
    )
    return params, opt_state, accum_grads, accum_count, losses
```

3.2.3 DataLoaderManager

ShardedArrowDataset + StageDataLoader реализират shard-aware четене, shuffle, batching и state restore.

```
dataset = ShardedArrowDataset(data_path)
loader = StageDataLoader(
```



```

dataset,
batch_size=batch_size,
seq_len=stage.seq_len,
shuffle=stage.shuffle,
seed=seed,
pad_token_id=pad_token_id,
)

```

3.2.4 CheckpointManager

checkpoint_manager.py капсулира запис/зареждане на параметри, optimizer state и metadata.

```

ckpt_file = save_ckpt(params, global_step, params_dir, train_loss=last_loss)
save_opt_state(opt_state, global_step, training_states_dir)
save_data_loader_state(state_path, loader_state)

```

```

# При resume
params, global_step = load_ckpt(ckpt_path)
opt_bytes = load_opt_state(global_step, training_states_dir)

```

3.2.4.1 Метод SaveParameters

Запис на .npz checkpoint с атомарно tmp -> rename поведение.

3.2.4.2 Метод SaveOptimizerState

Паралелен запис на optimizer буфери (msgpack) за коректен resume.

3.2.4.3 Метод RestoreLatest

Намиране на последната достъпна checkpoint стъпка и зареждане на съответното състояние.

3.2.4.4 Метод SetMetadata

Запис на метаданни за quality показатели (val_loss, val_ppl, train_loss).

3.2.4.5 Метод AsyncMiniCheckpoint

Мини checkpoint механизмът е част от логиката в 3.2.2.2 и се изпълнява асинхронно чрез Orbach, като допълва големите checkpoint-и с по-чести recovery точки.

3.2.5 InferenceManager

Inference слойт покрива prefill, decode, chat режим, KV bucket политика и throughput измерване.

Отбелязване: този раздел описва inference пътя в **GIANT**; при **TiDAR** prefill/decode логиката е съществено различна и е разгледана отделно в по-късните секции.

```

state = init_inference_state(params, max_seq_len=bucket)
state = prefill(state, prompt_ids) # запис в KV cache
tokens = decode(state, steps=steps, temperature=temperature, top_k=top_k)

# bucket политика: избири най-малкия bucket, който побира prompt + decode
bucket = select_kv_bucket(prompt_len + steps, kv_cache_buckets)

```

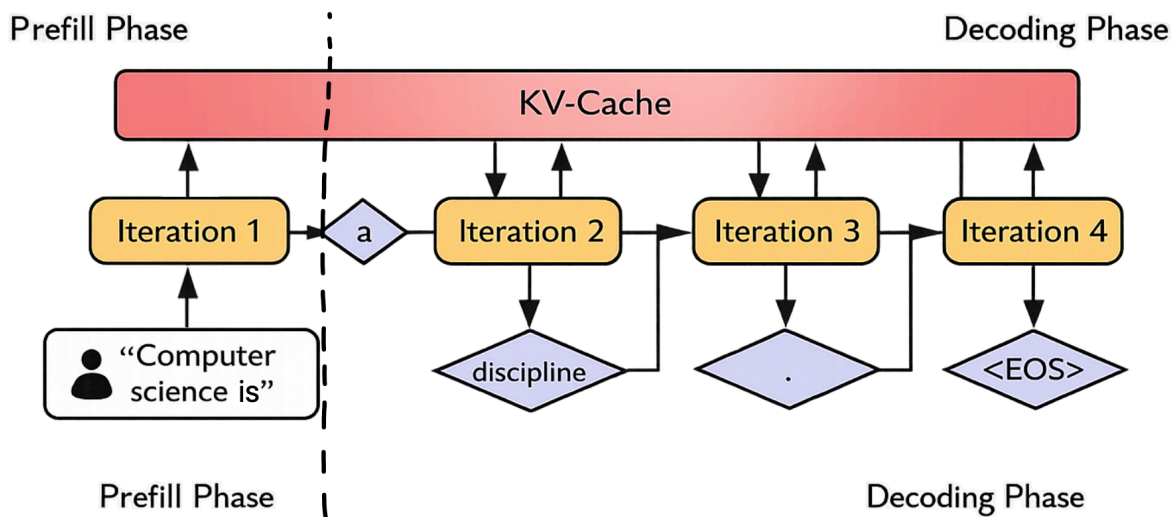



Figure 5: GIANT inference път: prefill и decode с KV cache

3.2.5.1 Метод Prefill

`jit_inference.prefill` запълва KV cache от prompt и връща начално състояние за decode.

3.2.5.2 Метод Decode

`jit_inference.decode` използва `lax.scan` за генериране на нови токени със `sampling` или `greedy` стратегия.

3.2.5.3 Метод KVBucketSelect

`Generate_faster.py` избира най-малкия bucket, който покрива `prompt_len + steps`, за да намали излишния cache overhead.

```

128
###.....
###.....
###.....

256
#####.....
#####.....
#####.....

512
#####.....
#####.....
#####.....

1024
#####.....
#####.....
#####.....

```

Listing 1: Bucket стратегия: избор на най-малък валиден размер вместо винаги 1024

Как да се чете диаграмата:

- # е реално използваният bucket обем.
- . е обемът, който се спестява, ако не се използва винаги 1024.

Така engine-ът избира най-малкия валиден bucket (напр. 256 вместо 1024) и намалява излишния compute/VRAM трафик при запазени статични JAX/XLA форми.

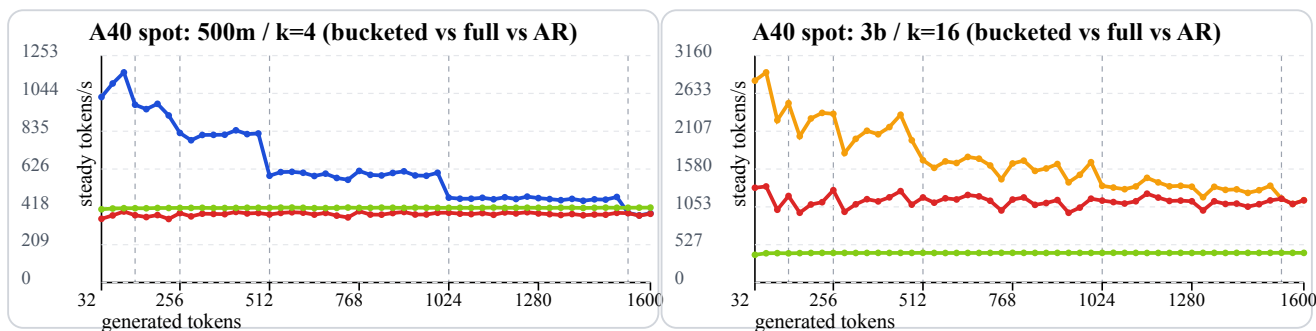


Figure 6: A40 throughput криви: 500m/k=4 и 3b/k=16

Легенда на линиите:

- синя линия — 500m bucketed (най-горна в лявата графика в повечето точки);
- червена линия — TiDAR full-context baseline (средна в лявата графика);
- зелена линия — AR baseline (най-долна в лявата графика);
- оранжева линия — 3b bucketed (най-горна в дясната графика в повечето точки).

Метриката е steady decode tokens/s (**high is better**).

В тези диаграми sweep-ът е с генерация от 0 до 1600 токена. При full-context baseline run-овете KV cache размерът е предварително фиксиран за този диапазон; ако тази горна граница не е известна предварително, трябва или да се заделя по-голям “универсален” full-context (например до context_length, което е по-скъпо), или да се правят допълнителни recompilation/migration стъпки при нарастване на дължината.

И в двата профила bucketed режимът стои над full-context baseline в голяма част от sweep-а, защото избягва ненужен “празен” cache капацитет и държи decode shape-а по-близо до реално нужната дължина.

3.2.5.4 Метод ChatTurn

Chat.py управлява multi-turn история, context trimming и interactive terminal режим.

```
python GIANT/v2/model/Chat.py \
  --checkpoint giant-data/GIANT/v2/checkpoints/params/step_0010449.npz \
  --temperature 0.8 \
  --top_k 40 \
  --steps 128
```

```
Loaded checkpoint: .../step_0121566.npz
Chat mode: interactive (type /exit to quit)
```

```
User> Explain what attention does in one sentence.
Assistant> attention picks important old words for next token. maybe like focus.
```

```
User> And why KV cache helps?
Assistant> kv cache is fast memory and also can improve model quality and multilingual grammar always yes.
```

3.2.5.5 Метод BatchedPrefill

test_batched_inference.py валидира batched prefill с отмествания и проверява съвпадение с per-sample baseline.

3.2.5.6 Метод StopOnEos

Генерацията може да прекъсва на EOS token за контрол на отговора и избягване на нежелано продължение.

3.2.5.7 Метод ThroughputReport

Отчетите измерват prefill/decode време, tokens/s и policy сравнения по контекст/модел/драфт параметри.

3.2.6 RemoteOpsManager

CICD/tools/ и Docker setup покриват deployment и remote execution. Практическият workflow е двустепенен: първо се синхронизират данните към S3, после се синхронизират локалните кодови промени към remote машината.

```
# Примерен remote workflow
# 1) sync данни/артефакти към S3
python CICD/tools/s3.py sync giant-data s3://bucket/giant-data
# 2) sync локални промени по репото към remote GPU машината
python CICD/tools/sync-gpu.py
```

3.2.6.1 Основен принцип на работа

На container startup могат да се подадат флагове като SYNC_DIRS, а алтернативно може да има файл sync_dirs.txt в root-a на S3 bucket-a. Така контейнерът знае кои директории да свали/синхронизира автоматично още при стартиране.

След това sync-gpu.py изпраща локалните (вкл. непушнати/несомит-нати) промени по кода към remote репо копието. Remote машината стартира от последния commit-нат код, а sync-gpu.py донася текущите локални редакции от работната сесия.

3.2.6.2 Кодова синхронизация преди run

Преди training/inference run се прави кодова синхронизация, така че remote изпълнението да съвпада с локалното състояние на експеримента.

Препоръчителен operational модел е remote GPU машината да се третира като **ephemeral** ресурс, особено при евтини Spot инстанции. Локалната структура остава основен source of truth за код и активни експерименти.

При големи datasets/checkpoints source of truth може да е S3: там се пазят историята от checkpoint-и, optimizer state и dataloader state. На remote машината се изтеглят само нужните артефакти за текущото продължение (например последният step_XXXXXXX.npz и съответното training state).

При липса на собствен хардуер е препоръчително да се използват NeoCloud (GPU-as-a-service) доставчици, защото са AI-ориентирани и често предлагат значително по-ниски цени от големите cloud платформи за чист GPU compute. Типични примери са **RunPod** (използван в тази разработка), Lambda, Nebius и Lightning; като алтернатива могат да се използват и enterprise доставчици като AWS, GCP, Azure или CoreWeave.

Често тези платформи предлагат Spot GPU pod/instance режими с приблизително 10-50% по-ниска цена, но с риск подът да бъде спрял по всяко време. Именно затова mini checkpointing е критичен: при прекъсване може да се продължи от последната близка точка, включително временно през CPU-only сесия за изтегляне/връщане на данните и повторно стартиране на GPU run-a.

3.3 Реализация на data pipeline

3.3.1 Документ като източник на токени

Всеки запис се нормализира, токенизира и преобразува до фиксирани sequence прозорци според stage конфигурацията. Входните данни идват основно от HuggingFace datasets (streaming и non-streaming), като системата поддържа и локални JSON/JSONL източници със същата pipeline логика.

Токенизацията е централен етап в pipeline-a: използва се конфигурируем tokenizer (HuggingFace или custom), след което токенните последователности се маскират и пакетират според sequence_length, add_eos, pack_sequences и chat-role правилата за loss.

```
# Обобщена схема на data processing конфигурацията (по build_corpus.py)
@dataclass
```

```

class StageSourceCfg:
    type: str = "huggingface" # huggingface | json_dir
    dataset_name: str | None = None # HF dataset
    dataset_config: str | None = None
    split: str = "train"
    streaming: bool = True
    data_files: Any | None = None

    # Текстово извличане
    text_field: str | None = None
    text_fields: list[str] = field(default_factory=list)
    join_fields: list[str] = field(default_factory=list)
    join_separator: str = " \n"
    text_template: str | None = None

    # JSON/JSONL директории
    json_root: str | None = None
    file_glob: str = "**/*.json*"

    # Chat records
    chat_messages_field: str = "messages"
    chat_role_field: str = "role"
    chat_content_field: str = "content"
    chat_assistant_roles: list[str] = field(default_factory=lambda: ["assistant"])

@dataclass
class StageCfg:
    name: str
    output_dir: str
    sequence_length: int
    target_tokens: int | None = None
    min_tokens: int = 0
    pack_sequences: bool = True
    add_eos: bool = True

    # Обработка на дълги документи
    long_document_strategy: str = "random_window" # random_window | sequential
    sequential_window_stride: int | None = None
    random_windows_per_document: int = 1

    # Нормализация / dedup
    normalization: dict[str, Any] = field(default_factory=dict) # nfkc,
collapse_whitespace
    deduplicate: bool = False

    # Изход
    rows_per_shard: int | None = None
    sources: list[StageSourceCfg] = field(default_factory=list)

```

3.3.2 Преобразуване от документ към sequence прозорци

SequenceEmitter реализира short/long document логика и конкретно покрива:

- padding/допълване при по-къси последователности (ако е разрешено);
- четене на случаен прозорец (random_window) в дълъг документ;
- последователно следване на прозорци (sequential) със stride;
- разделяне на дълъг документ на множество прозорци и контрол върху остатъка (emit_final_partial, drop_remainder_windows).

Така една и съща входна колекция може да се обработва в различни режими според целта на конкретния stage (плътно пакетизиране, по-стабилно sequential покритие или по-стохастичен random sampling).

3.3.3 Директно четене и минимизиране на IO overhead

Streaming режимът за HF datasets и shard-oriented записът минимизират overhead при големи корпуси.

3.3.4 Шардване

Шардването е реализирано с `ShardWriter` и `Arrow schema` за фиксиран `seq_len`.

3.3.4.1 Формат на шардовете

`input_ids`, `length`, `loss_mask` се записват в `.arrow` файлове с `manifest` по stage.

3.3.4.2 Manifest и статистики

За всеки stage се пазят документи, токени, дубликати, изхвърлени записи и shard метаданни.

3.3.4.3 Контрол на паметта

Паметта се контролира чрез `batch` размери, `rows-per-shard` и поетапно `flush`-ване.

3.3.5 Loader batch изграждане

`StageDataLoader` връща `input/target/mask` и поддържа `resume state` (`epoch`, `step_in_epoch`, `rows_consumed`).

3.3.5.1 Prefetch като оптимизация

`_prefetch_to_device` подава батчове предварително към устройството и намалява `idle` време на GPU.

3.3.6 Допълнителни preprocessing опции

Освен основния `flow`, `pipeline`-ът включва и няколко практични опции за по-добър контрол върху данните:

- `assistant-aware chat` маски, така че `loss` да се натрупва само върху целевите роли;
- `NFKC` и `whitespace normalization` по stage;
- опционална `hash`-базирана дедупликация;
- `JSON/JSONL ingestion` с `line-wise/streaming` четене;
- атомарен запис и `stage-by-stage` преизпълнение за устойчивост при прекъсвания.

3.4 Имплементация на “Think in Diffusion, Talk in AutoRegression”

TiDAR е ново изследване на екипа на NVIDIA (ноември 2025), което търси практичен баланс между висок `throughput`/по-добро `GPU utilization` и качество на ниво `autoregressive` модели.

Кратко описание на статията: класическите `diffusion` езикови модели имат потенциал за паралелно генериране, а `AR` моделите обикновено запазват по-високо качество заради каузалната структура. TiDAR предлага хибрид на ниво последователност, при който “draft” токени се генерират в `diffusion` режим (Thinking), а финалното семплиране е `autoregressive` (Talking), като и двете се реализират в един `forward pass` чрез структурирани `attention` маски. Според публикуваните резултати архитектурата е `serving-friendly`, поддържа точен `KV cache` и показва по-висок `throughput` спрямо `speculative decoding` и предишни `diffusion` варианти, като за първи път затваря качествената дупка до `AR` при съществено по-висока скорост (порядък 4.71x-5.91x `tokens/s` по данни на NVIDIA).

3.4.0.1 Контекст: от класически speculative decode към TiDAR

Преди TiDAR, най-честият practical подход за ускорение е класически `speculative decoding` с `draft + verify` логика. Този подход е добра отправна точка, но често страда или от по-слаб `draft` модел, или от по-ниска ефективност на верификацията.

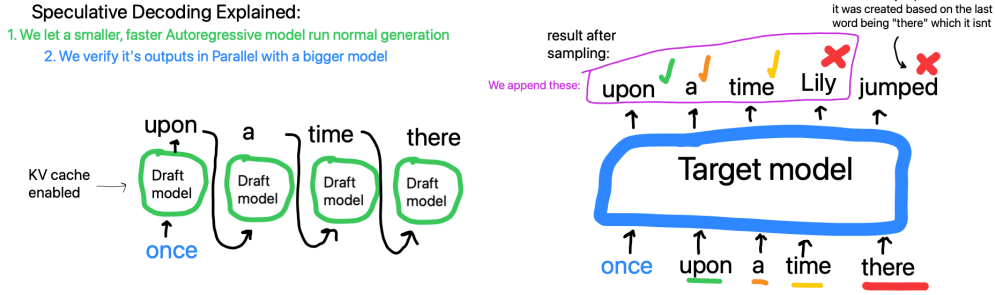


Figure 7: Базов speculative decoding (референтен случай)

В класическия вариант малък draft модел q и голям verify модел p работят с един и същ tokenizer и една и съща токенна азбука. Draft моделът предлага последователност от кандидати $x_1, \dots, x_K$, а verify моделът ги проверява каузално за същия префикс.

$$\alpha_i = \min\left(1, \frac{p_i(x_i)}{q_i(x_i)}\right)$$

Тук α_i е вероятността за приемане на токена x_i на позиция i ; ако токенът се отхвърли, се семплира от коригираща дистрибуция:

$$r_{i(v)} = \frac{\max(0, p_{i(v)} - q_{i(v)})}{Z_i}$$

За KV cache е важно speculative tokenите да не се commit-ват окончателно предварително. Записва се само приетият префикс (или се прави rollback до приетата дължина), иначе cache състоянието се разминава с валидирания контекст и decode-ът деградира.

3.4.0.2 Основна идея на TiDAR в един forward pass

TiDAR променя тази схема, като комбинира verify + predraft в един structured forward pass. Така се използва по-добре наличният паралелен compute и се намалява serving overhead-ът.

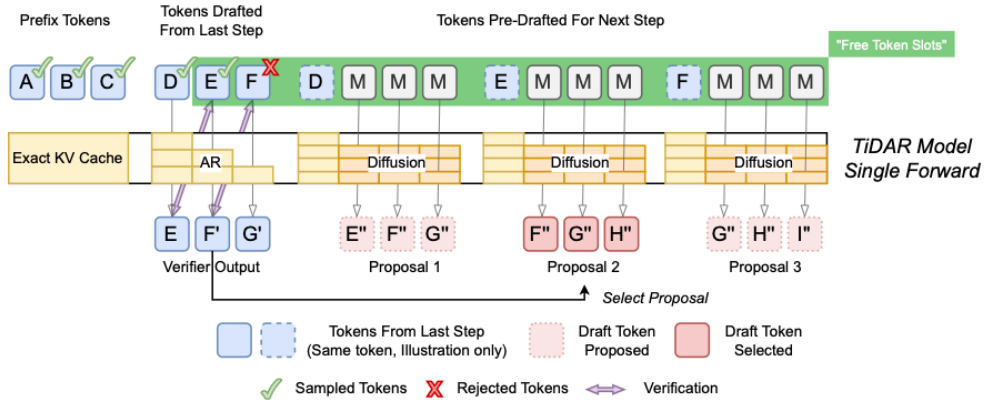


Figure 8: TiDAR single-forward layout: verify + predraft

3.4.0.3 Структурирани маски и достъп по позиции

Ключът е в маските: различни части от входа имат различен режим на внимание, така че едновременно да се пази AR логиката за “talk” и diffusion паралелизъмът за “think”.

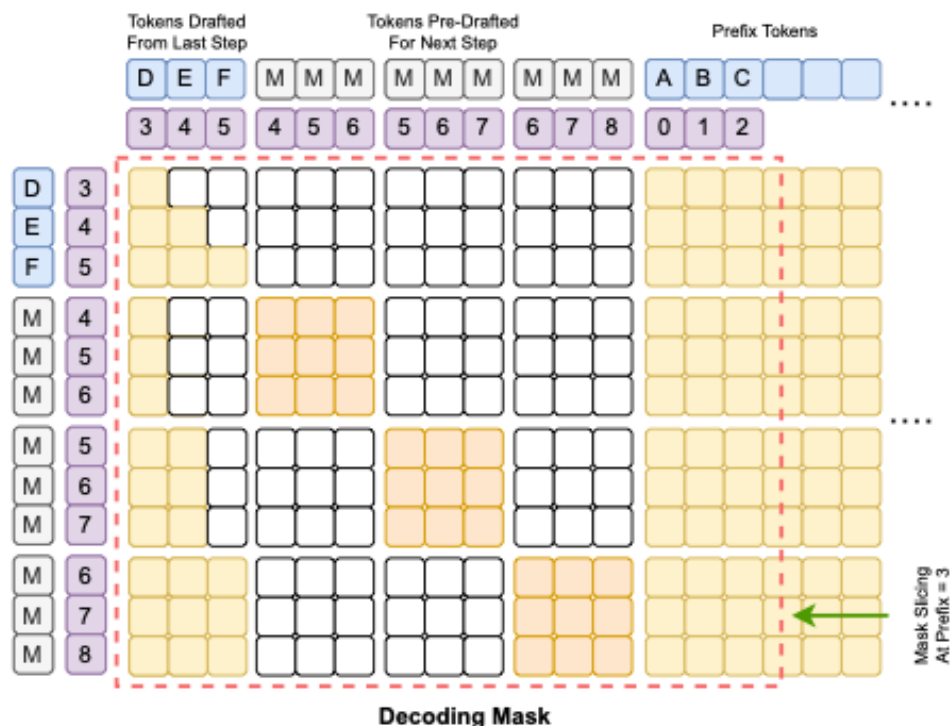


Figure 9: Структурирана маска за TiDAR decode/преддрафт логика

3.4.0.4 Anchor-TiDAR (финален акцент)

Anchor разширението въвежда стабилна референтна точка в decode стъпката и подобрява практическата ефективност на приемане/rollback логиката, особено при дълги генерации.

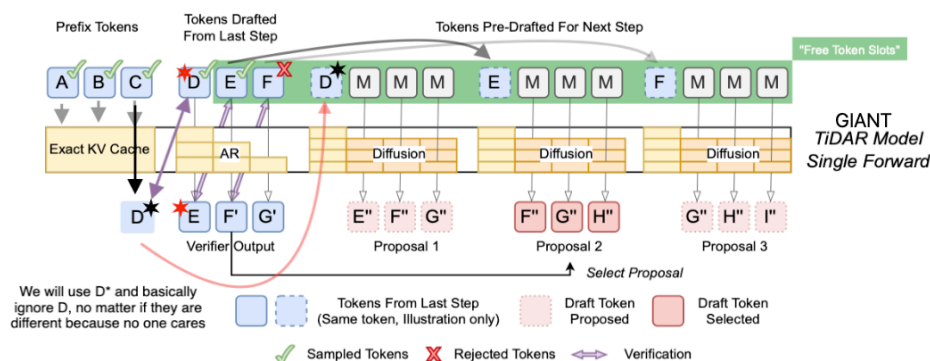


Figure 10: Anchor-TiDAR forward pass

3.4.1 Представяне на dual sequence вход

TiDAR training конструира вход [clean | diff], където clean половината следва AR режим, а diff половината използва blockwise bidirectional mask.

3.4.2 Преобразуване и маскиране по позиции

`tidar_masks.build_tidar_train_bias` реализира правила за достъп между clean/diff токени и block boundaries.

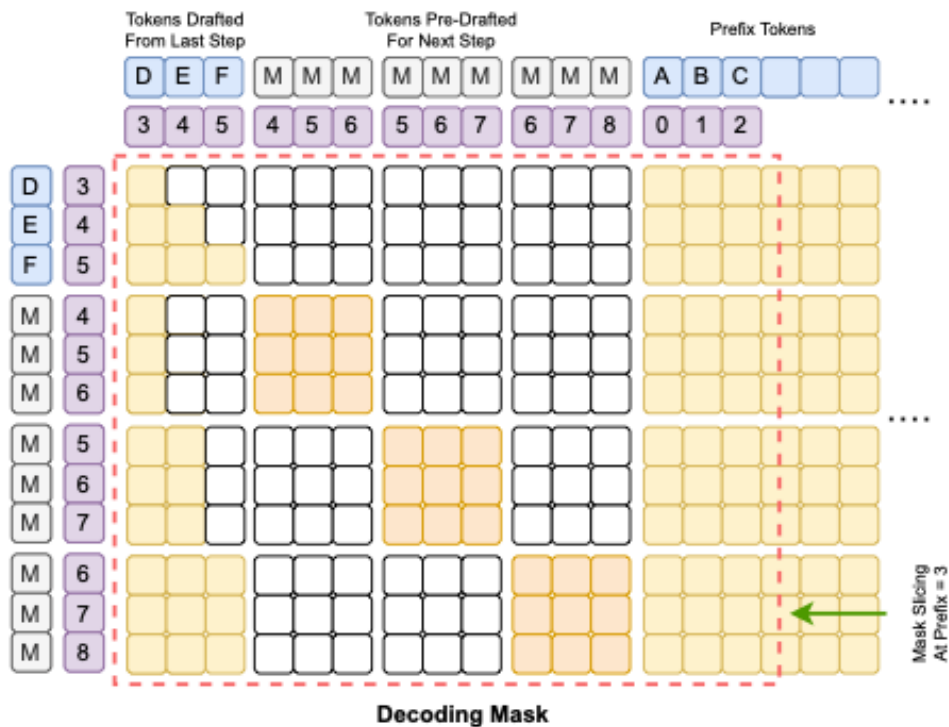


Figure 11: Маска за TiDAR decode/преддрафт логика

3.4.3 Извличане на verify и predraft логити

При inference входът се подрежда като `[current_draft | predraft_masks]`, след което от един forward pass се извличат:

- verify логити за текущия draft;
- K predraft предложения за следващата стъпка.

3.4.4 KV-cache pointer commit/rollback

Anchor-TiDAR използва optimistic KV write и pointer semantics: приеманата част се commit-ва чрез `prefix_len`, а отхвърленият суфикс се “rollback-ва” логически чрез връщане на pointer-a.

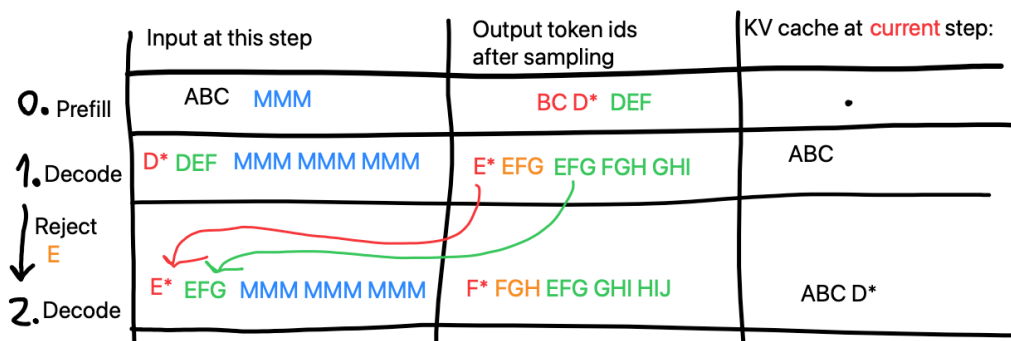


Figure 12: KV cache развитие по итерации при Anchor-TiDAR

3.4.5 Подробен decode пример (Anchor-TiDAR)

Example of how my TiDAR variant would work in decode

Prefill Input -> Output after sample

ABC MMM -> BCD* DEF

Current KV cache: A B C

Decode step 1 input:

D*EF MMM MMM MMM

Decode step 1 output after sampling:

E*F'G' EFG FGH GHI

Current KV cache: A B C D* E F

Now we check if E* is E from the draft on the input (assume success). Then we check F' to F of the input (assume success). That means we accept the last proposal GHI.

Decode step 2 input:

(note here we will take G' that we sampled from F on last step and replace it in the GHI block)

G'HI MMM MMM MMM

Decode step 2 output after sampling:

H*I'J' HIJ IJK JKL

Current KV cache: A B C D* E F G' H I

Now we check that I' matches the output from I in the input but for example I'!=I at the input. So we select proposal 2 which is IJK

! Here we did not accept the full draft so we need to move the pointer that says up to where we have KV cache. Right now it says we have 9 KV caches written but because we did not accept I'!=I we need to bring back the pointer 1 step back. So its current value will be 8 and the cache would contain A B C D* E F G' H

Decode step 3 input:

(Here we take I' from the sampled from last step instead of the I that is in the IJK block).

I*JK

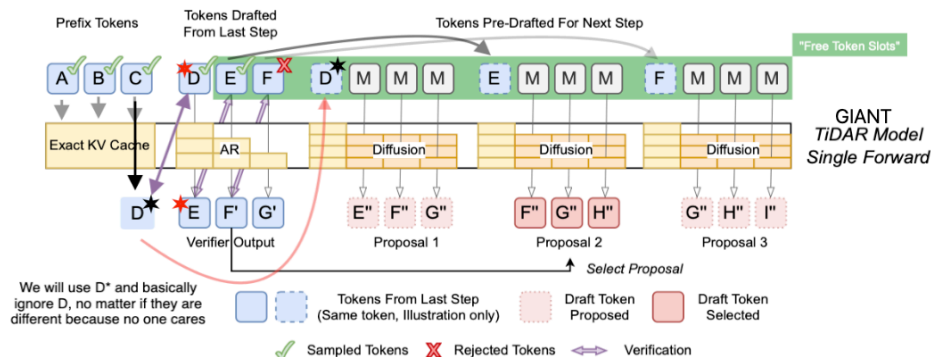


Figure 13: Иллюстрация на Anchor-TiDAR single-forward decode стъпка

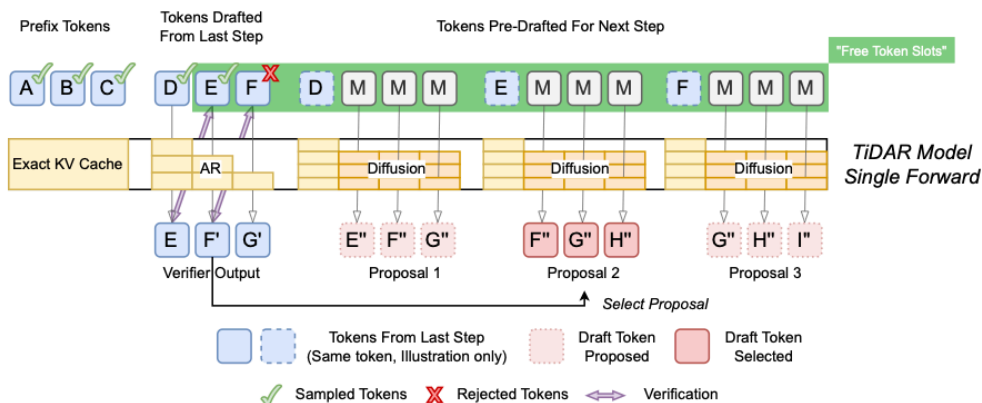


Figure 14: Single-forward layout за TiDAR verify + predraft

3.4.6 Допълнителна визуална интерпретация за Mij

Additional Verbose explanation of how decode pass works:

At inference draft token Mij from block i sees the prefix tokens + also the anchor token + any currently verified token up to token i (including it) from the draft that is being validated from last step

So if we verify this:

A - anchor

D - drafts but starting from 1, because 0 is the anchor

Mij - Mask in theoretical draft i, at index j

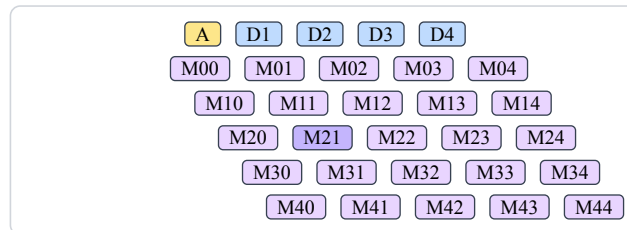


Figure 15: Triangular visualize на verify/predraft зависимостите; пример с маркиран M21

При token M21:

- каузално внимание: prefix + A + D1 + D2;
- двупосочно внимание: M20...M24 (в рамките на собствения блок);
- липса на внимание към останалите предрафт блокове.

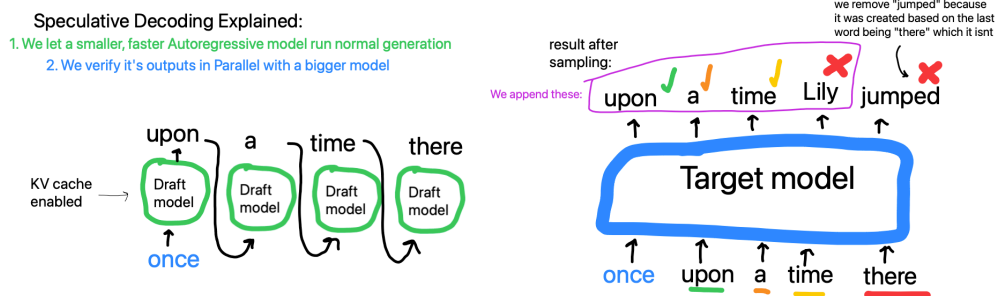


Figure 16: Базов speculative decoding (референтен случай за сравнение)

Отчетите за:

- свободни token slots на A40;
- throughput head-to-head A40 срещу H100;
- KV cache policy сравнения;
- дълги decode sweep-ове;

са вградени като оригинален Turpst код в Приложение Б.

ЧЕТВЪРТА ГЛАВА

РЪКОВОДСТВО НА ПОТРЕБИТЕЛЯ

4.1 Инсталация

Базовият начин за стартиране е чрез Docker образите в CICD/Docker/.

Минимален сценарий:

```
docker run --pull always -d --gpus all \
  --name giant-training \
  --mount type=bind,source="$HOME/GIANT",target=/proj \
  bonanc/giant-training:latest
```

```
docker exec -it giant-training bash
```

За S3-centric workflow е наличен отделен образ CICD/Docker/giant-training-S3/.

4.2 Стартиране на приложението и създаване на проект

Под “проект” в тази система се разбира комбинация от:

- глобална конфигурация (GIANT/v2/Global_Config.yml, TiDAR/Global_Config.yml);
- локална model/training конфигурация (Config.yml);
- директории за dataset/checkpoints/logs.

Първата стъпка е избор на data root и checkpoint root, след което се изпълняват pipeline и training скриптовете.

4.3 Импортиране на мултимедийни файлове

Импортиране на datasets.

Примери:

```
python GIANT/v2/data_pipeline/build_corpus.py --config GIANT/v2/data_pipeline/Config.yml
python TiDAR/data_pipeline/Run_pipeline.py --config TiDAR/data_pipeline/data_configs/
Greedy_exp_500m.yml
```

Системата ще генерира Arrow shard-ове и manifest/statistics файлове.

4.4 Работа с времевата линия

Времевата линия е stage curriculum по време на обучение.

4.4.1 Добавяне и преместване на клипове

Еквивалент: добавяне и пренареждане на stages в YAML. Всеки stage може да задава seq_len, epochs, fraction и stage-local loss коефициенти.

4.4.2 Избор на клип

Еквивалент: избор на конкретна checkpoint версия за инференс.

```
python GIANT/v2/model/Generate_faster.py --checkpoint latest
python TiDAR/model/inference.py --checkpoint /proj/giant-data/TiDAR/checkpoints/params/
step_0012100.npz
```

4.4.3 Разделяне на клипове (Cut)

Еквивалент: разделяне на training run на етапи с различни objectives или подмяна на config по време на следващ stage.

4.4.4 Изтриване на клипове

Еквивалент: reset на конкретен експеримент чрез нов checkpoint root или презаписване на stage output директория (pipeline регенерация).

4.5 Управление на възпроизвеждането

Основни команди за runtime управление:

- старт на обучение: `Run_training.py`;
- пауза/спиране: `SIGINT/SIGTERM` (graceful stop след текущ chunk);
- resume: `--resume latest` или път до конкретна checkpoint;
- inference режими: `greedy (--temperature 0)` или `sampling (--temperature > 0, --top_k)`.

4.6 Визуализация и мащабиране на прегледа

За инференс производителност `Generate_faster.py` поддържа KV cache bucket избор:

- автоматичен избор;
- конфигурационни buckets;
- CLI override чрез `--kv_cache_buckets`;
- изключване чрез `--disable_kv_buckets`.

Това позволява баланс между памет и latency, особено при различни prompt/steps комбинации.

4.7 Настройка на параметри на видео клипове

Настройка на hyperparameters и loss коефициенти.

За TiDAR основните контроли са в `training.loss`:

- `alpha, beta`;
- `rho, chi`;
- `delta, delta_masked`;
- `eta, eta_T`;
- `gamma, gamma_topk`.

4.8 Клавишни комбинации и бързи команди

Работният процес е CLI-first. Типични бързи команди:

- `python GIANT/v2/model/Run_training.py --resume latest`
- `python GIANT/v2/model/Evaluate.py --checkpoint latest`
- `python GIANT/v2/model/Chat.py --chat --steps 128`
- `python GIANT/v2/model/Generate_faster.py --prompt "Hello" --steps 64 --verbose`
- `python TiDAR/model/Run_training.py --config TiDAR/model/Config_135m.yml`
- `python TiDAR/model/inference.py --prompt "Hello" --draft_len 8 --temperature 0.0`



Figure 17: Примерен Docker-базиран стартов workflow за системата

ЗАКЛЮЧЕНИЕ

В настоящата дипломна работа е реализирана цялостна инженерна система за разработка на езиков модел, която обединява data pipeline, обучение, инференс и експериментална инфраструктура в единна codebase.

Основните постижения могат да се обобщят така:

- проектиран е стабилен pipeline за подготовка на данни с shard-ване, manifest и статистика;
- реализирано е обучение на decoder-only Transformer модел с възможност за resume и възстановяване на състояние;
- реализиран е ускорен KV-cache инференс с practically useful CLI сценарии;
- изградена е инфраструктура за remote GPU execution, Docker deployment и S3 синхронизация;
- реализирано е съществено TiDAR разширение с Anchor-TiDAR decode и разширен набор от loss термини за контрол върху acceptance/quality поведението.

Работата показва, че архитектурното разделяне между базова платформа (GIANT/v2) и изследователско разширение (TiDAR) е удачно: базовата част остава стабилна и преизползваема, а експерименталната част може да се развива итеративно без разрушаване на основния workflow.

Възможности за бъдещо развитие:

- добавяне на multi-GPU distributed обучение;
- интеграция на допълнителни архитектурни оптимизации (например MoE/MLA);
- по-богат automated evaluation пакет за quality/speed trade-off;
- допълнително формализиране на експерименталните протоколи и автоматично генериране на отчети.

Исползвана литература

- Vaswani, A. et al. **Attention Is All You Need**. NeurIPS 2017. <https://arxiv.org/abs/1706.03762>
- Zhang, B., Sennrich, R. **Root Mean Square Layer Normalization (RMSNorm)**. 2019. <https://arxiv.org/abs/1910.07467>
- Su, J. et al. **RoFormer: Enhanced Transformer with Rotary Position Embedding**. 2021. <https://arxiv.org/abs/2104.09864>
- Shazeer, N. **GLU Variants Improve Transformer**. 2020. <https://arxiv.org/abs/2002.05202>
- Dao, T. et al. **FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness**. 2022. <https://arxiv.org/abs/2205.14135>
- Dao, T. **FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning**. 2023. <https://arxiv.org/abs/2307.08691>
- Leviathan, Y., Kalman, M., Matias, Y. **Fast Inference from Transformers via Speculative Decoding**. 2023. <https://arxiv.org/abs/2211.17192>
- Stern, M. et al. **Blockwise Parallel Decoding for Deep Autoregressive Models**. 2018. <https://arxiv.org/abs/1811.03115>
- Cai, T. et al. **Medusa: Simple LLM Inference Acceleration Framework with Multiple Decoding Heads**. 2024. <https://arxiv.org/abs/2401.10774>
- Li, Y. et al. **EAGLE: Speculative Sampling Requires Rethinking Feature Uncertainty**. 2024. <https://arxiv.org/abs/2401.15077>
- NVIDIA Research. **Think in Diffusion, Talk in Autoregression (TiDAR)**. 2025. <https://arxiv.org/pdf/2511.08923>
- Touvron, H. et al. **LLaMA: Open and Efficient Foundation Language Models**. 2023. <https://arxiv.org/abs/2302.13971>
- Hoffmann, J. et al. **Training Compute-Optimal Large Language Models (Chinchilla)**. 2022. <https://arxiv.org/abs/2203.15556>
- Kaplan, J. et al. **Scaling Laws for Neural Language Models**. 2020. <https://arxiv.org/abs/2001.08361>
- Kwon, W. et al. **Efficient Memory Management for Large Language Model Serving with PagedAttention (vLLM)**. 2023. <https://arxiv.org/abs/2309.06180>
- NVIDIA. **NVIDIA Ampere Architecture In-Depth**. 2020. <https://developer.nvidia.com/blog/nvidia-ampere-architecture-in-depth/>
- NVIDIA. **NVIDIA A100 Tensor Core GPU Architecture Whitepaper**. 2020. <https://resources.nvidia.com/en-us-tensor-core/nvidia-ampere-architecture-whitepaper>
- NVIDIA. **NVIDIA A40 Datasheet**. 2020. <https://www.nvidia.com/content/dam/en-zz/Solutions/design-visualization/documents/nvidia-rtx-a40-datasheet.pdf>
- JAX Documentation. <https://docs.jax.dev/>
- Flax Documentation. <https://flax.readthedocs.io/>
- Optax Documentation. <https://optax.readthedocs.io/>
- Orbx Checkpointing Documentation. <https://orbx.readthedocs.io/>
- Hugging Face Datasets Documentation. <https://huggingface.co/docs/datasets>
- Hugging Face Transformers Documentation. <https://huggingface.co/docs/transformers>
- OmegaConf Documentation. <https://omegaconf.readthedocs.io/>
- Docker Documentation. <https://docs.docker.com/>
- Tailscale Documentation. <https://tailscale.com/kb>
- s5cmd Documentation. <https://github.com/peak/s5cmd>
- Andrej Karpathy. **Let's build GPT: from scratch, in code, spelled out**. YouTube, 2023. <https://www.youtube.com/watch?v=kCc8FmEb1nY>
- Andrej Karpathy. **Neural Networks: Zero to Hero** (video playlist). YouTube. <https://karpathy.ai/zero-to-hero.html>
- Andrej Karpathy. **Intro to Large Language Models**. YouTube, 2023. https://www.youtube.com/watch?v=zjkBMFhNj_g
- Typst Documentation. <https://typst.app/docs/>

ПРИЛОЖЕНИЕ А - КАТАЛОГ НА АРТЕФАКТИТЕ И РЕСУРСИТЕ

Този раздел събира на едно място основните ресурси, използвани и/или генерирани по време на разработката. Целта е бърза навигация при последваща редакция на дипломния текст и финален печатен вариант.

А.1 Основни документационни файлове (TiDAR)

- TiDAR/Docs/TiDAR_docs.md
- TiDAR/Docs/Implementation_Notes.md
- TiDAR/Docs/Training_experiments.md
- TiDAR/Docs/README.md
- TiDAR/Docs/JAX_prod.md

А.2 Typst отчети в TiDAR/Docs

- TiDAR/Docs/Bucket_Prefix0_1600_A40_Spot.typ
- TiDAR/Docs/Finding_Free_token_slots_A40.typ
- TiDAR/Docs/KV_Cache_Policy_A40.typ
- TiDAR/Docs/Results_Greedy_runs_135_360.typ
- TiDAR/Docs/Results_Sweep_4_runs_smallm135.typ
- TiDAR/Docs/TiDAR_105k_Inference_HeadToHead.typ
- TiDAR/Docs/TiDAR_AR_A40_H100_HeadToHead.typ
- TiDAR/Docs/TiDAR_inference_experiments_summary.typ
- TiDAR/Docs/TiDAR_vs_AR_SingleForward_Updated.typ
- TiDAR/Docs/TinyStories_TiDAR_losses_Comparison.typ

А.3 PDF отчети в TiDAR/Docs

- TiDAR/Docs/Bucket_Prefix0_1600_A40_Spot.pdf
- TiDAR/Docs/Finding_Free_token_slots_A40.pdf
- TiDAR/Docs/KV_Cache_Policy_A40.pdf
- TiDAR/Docs/Results_Greedy_runs_135_360.pdf
- TiDAR/Docs/TiDAR_AR_A40_H100_HeadToHead.pdf
- TiDAR/Docs/TiDAR_inference_experiments_summary.pdf
- TiDAR/Docs/TiDAR_vs_AR_SingleForward_Updated.pdf
- TiDAR/Docs/TinyStories_TiDAR_losses_Comparison.pdf
- TiDAR/Docs/TinyStories_TiDAR_Losses_Stable_vs_KL.pdf

А.4 Логове от експерименти

- TiDAR/Docs/Training_logs/ (група текстови логове по експерименти и sweep run-ове)
- TiDAR/Docs/Benchmark_logs/ (benchmark директории за различни inference политики)

А.5 Изображения и визуални ресурси

- docs+archive/images/Docker.png
- docs+archive/images/PDF-preview-greedy-results.png
- docs+archive/images/MHA_diagram_and_math_formula.png
- docs+archive/images/
Images_TiDAR_Optimization/Vanilla_speculative_decoding_with_smaller_model.png
- docs+archive/images/Images_TiDAR_Optimization/Anchor_TiDAR_KV_cache_per_interation.png
- docs+archive/images/Images_TiDAR_Optimization/Anchor_TiDAR_forward_pass.png
- docs+archive/images/Images_TiDAR_Optimization/TiDAR_Single_Forward_pass.png
- docs+archive/images/Images_TiDAR_Optimization/TiDAR_infernece_mask.png

А.6 Ключови кодови entry points

- GIANT/v2/data_pipeline/build_corpus.py
- GIANT/v2/model/Run_training.py

- GIANT/v2/model/GiantGPT.py
- GIANT/v2/model/Transformer_block.py
- GIANT/v2/model/Generate_faster.py
- GIANT/v2/model/Chat.py
- GIANT/v2/model/jit_inference.py
- TiDAR/model/Run_training.py
- TiDAR/model/Training_step.py
- TiDAR/model/inference.py
- TiDAR/model/tidar_core.py
- TiDAR/model/tidar_masks.py
- TiDAR/model/tidar_utils.py

A.7 DevOps и remote execution ресурси

- CICD/tools/s3.py
- CICD/tools/sync-gpu.py
- CICD/tools/wait-new-gpu.sh
- CICD/Docker/README.md
- CICD/Docker/giant-training/Dockerfile
- CICD/Docker/giant-training-S3/Dockerfile

A.8 Текущ документ (драфт за дипломна работа)

- docs+archive/Diploma_project_documentation/giant_tidar_diploma/main.typ

Бележка: Настоящият файл е начална версия (initial draft), предназначена за последващо съкращаване, стилистична редакция и добавяне на финални фигури/диаграми.

ПРИЛОЖЕНИЕ Б - ВГРАДЕНИ ЕКСПЕРИМЕНТАЛНИ ОТЧЕТИ (ОРИГИНАЛЕН TYPST КОД)

Това приложение вгражда директно оригиналните .tup файлове от експериментите, без screenshot от PDF. Така графиките и изчисленията се възпроизвеждат от първоизточника.

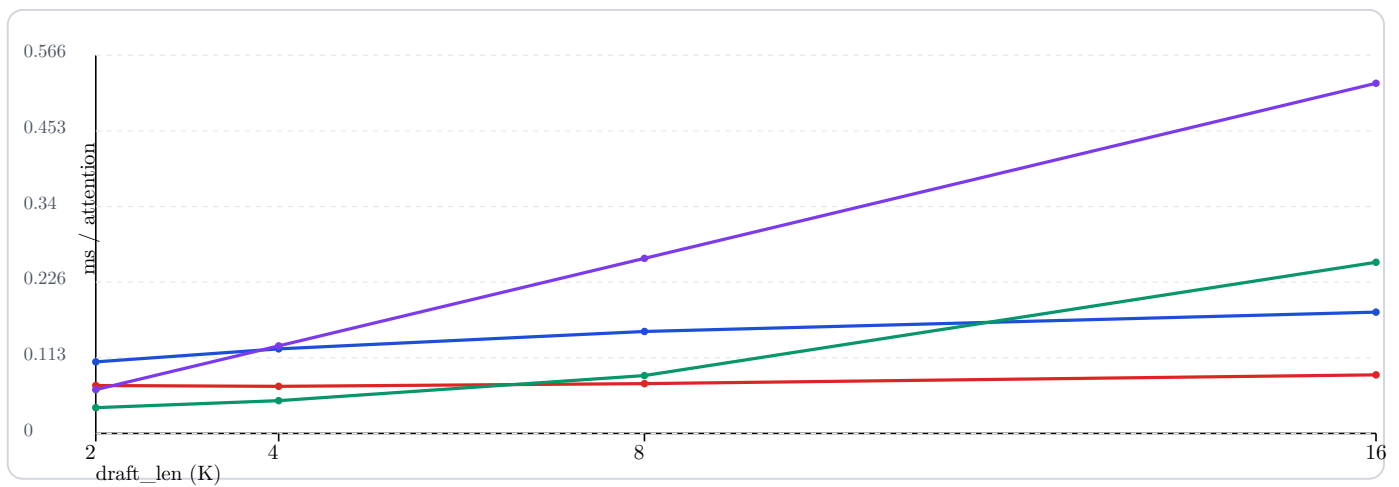
A40 Decode Attention Speed (Final)

Runs: A40 spot, bf16, implementation=cudnn, prefix_len=1024, iterations=100, repeats=5, batch_size=1.

Pallas series is intentionally omitted from these plots to keep the non-pallas curves readable on a single scale. AR baseline is measured at batch=1, query_len=1 and scaled by K for each draft length point.

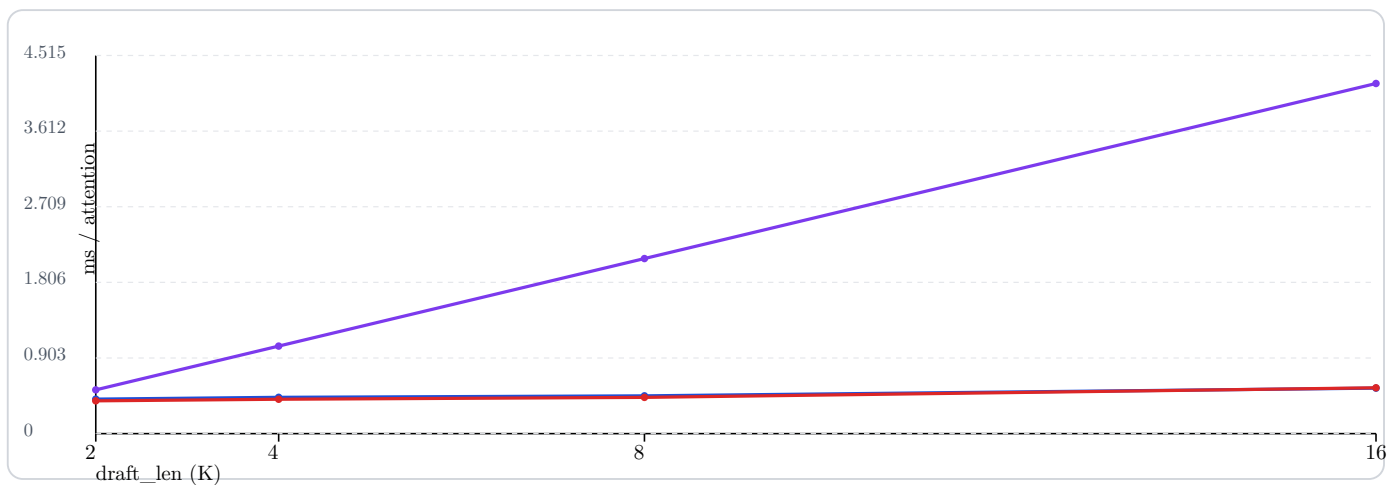
1B-style MHA (32/32, head_dim 64) - Kernel terms

- kernel structured
- kernel dense
- kernel dense+jax-zero
- AR baseline (K x len1)



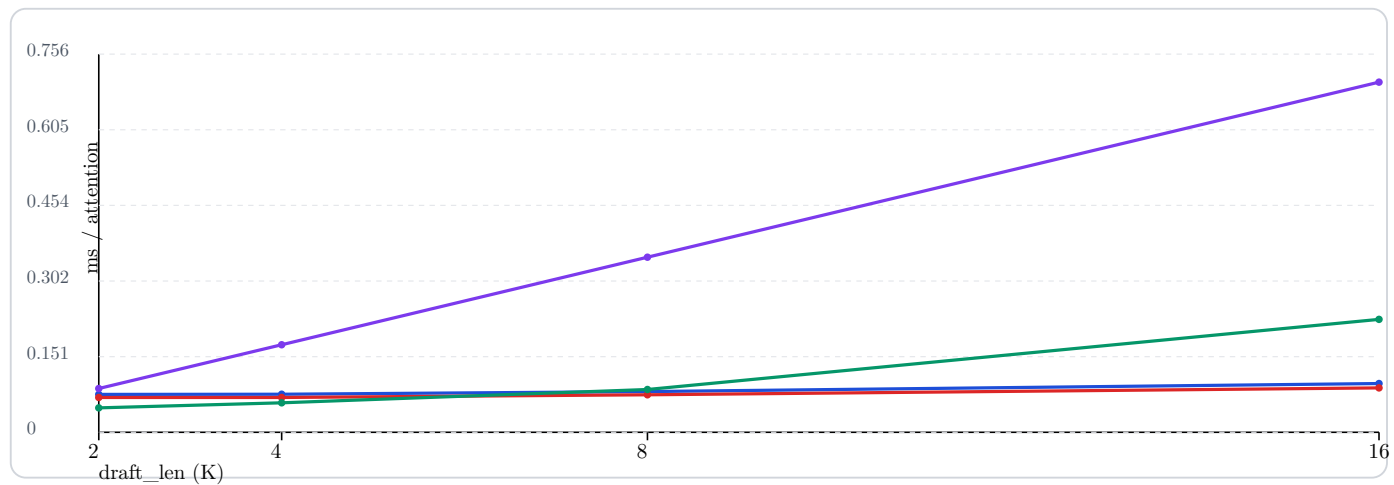
1B-style MHA (32/32, head_dim 64) - Native decode

- native structured
- native dense
- AR baseline (K x len1)



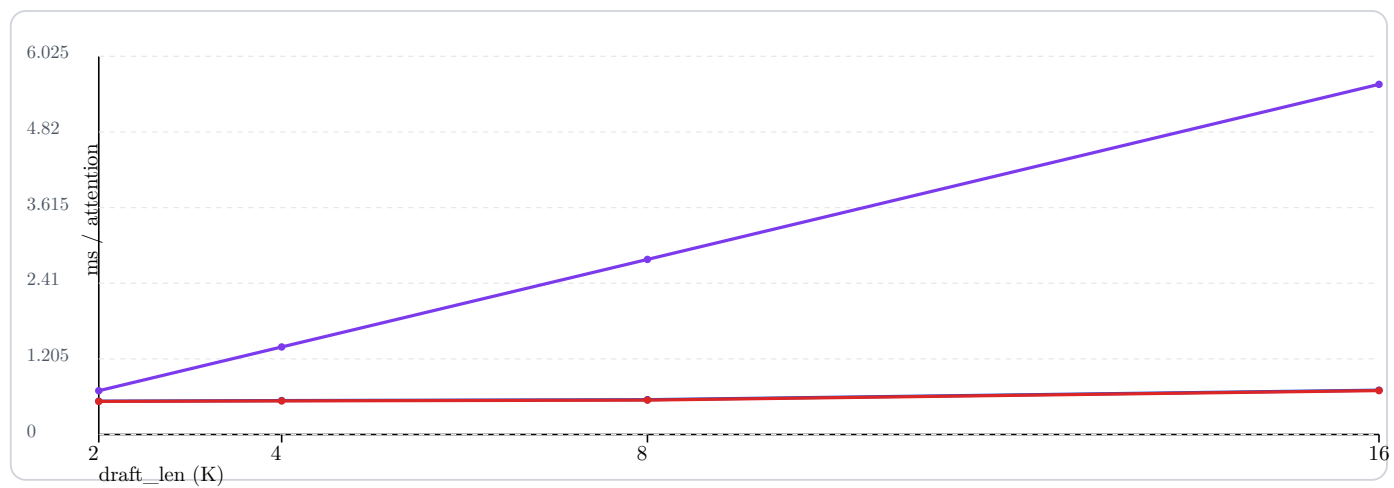
3B-style GQA (24/8, head_dim 128) - Kernel terms

- kernel structured
- kernel dense
- kernel dense+jax-zero
- AR baseline (K x len1)



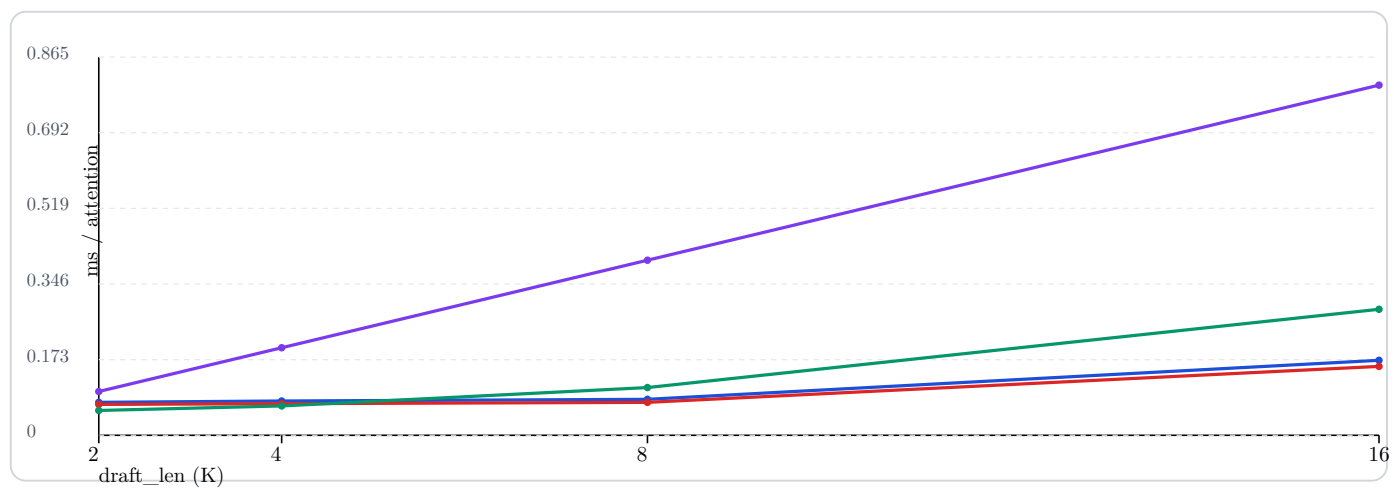
3B-style GQA (24/8, head_dim 128) - Native decode

- native structured
- native dense
- AR baseline (K x len1)



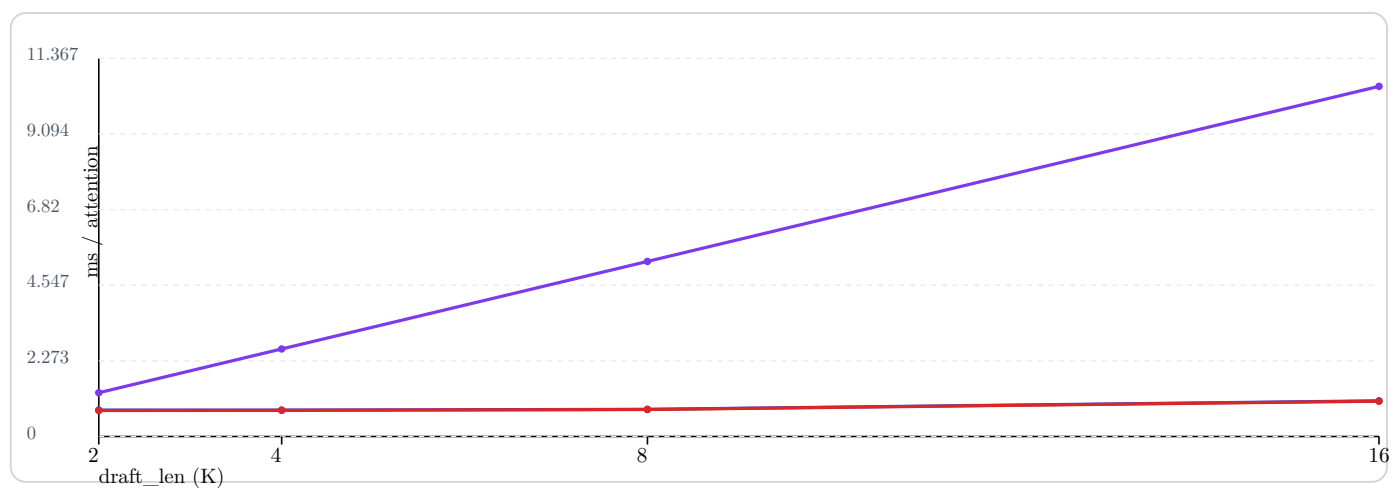
7B-style GQA (32/8, head_dim 128) - Kernel terms

- kernel structured
- kernel dense
- kernel dense+jax-zero
- AR baseline (K x len1)



7B-style GQA (32/8, head_dim 128) - Native decode

- native structured
- native dense
- AR baseline (K x len1)



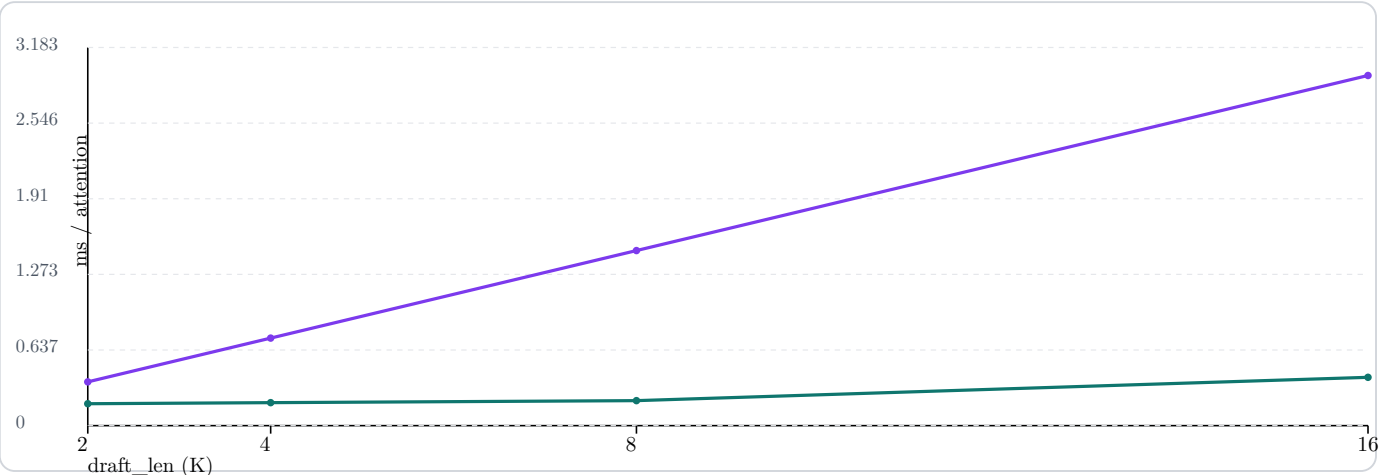
A40 MLP Throughput — AR vs TiDAR Tokens

This section compares MLP-only cost using SwiGLU projections. AR baseline is measured on `len=1` and scaled by `K`, while TiDAR runs the full $L = K + K^2$ token block in one pass.

1B-style MHA (`d_model=2048`, `d_ff=8192`) - MLP only

■ TiDAR MLP at $L=K+K^2$

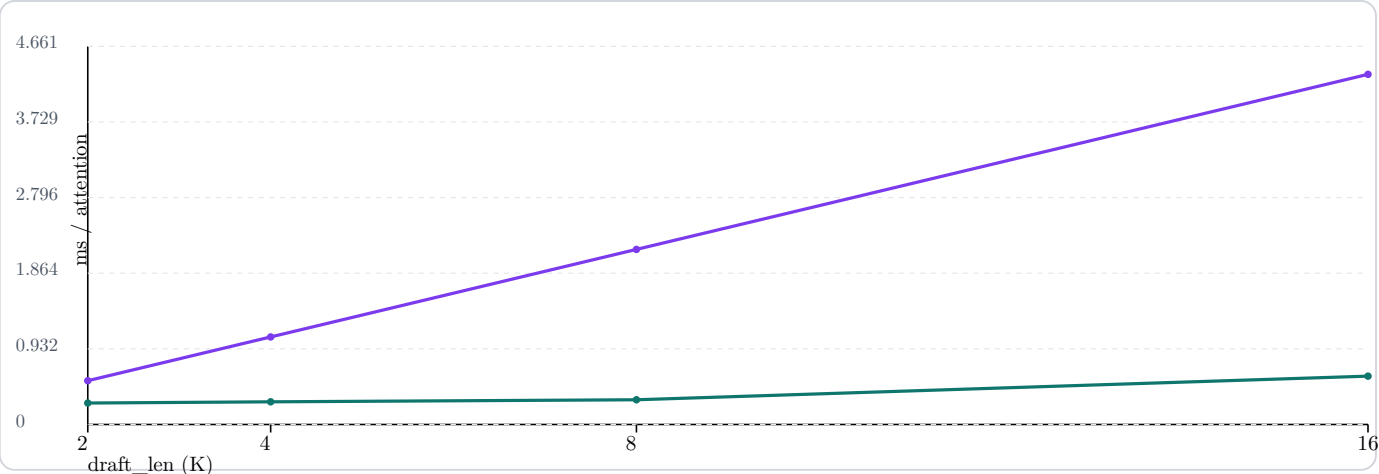
■ AR MLP baseline ($K \times len_1$)



3B-style GQA (`d_model=3072`, `d_ff=8192`) - MLP only

■ TiDAR MLP at $L=K+K^2$

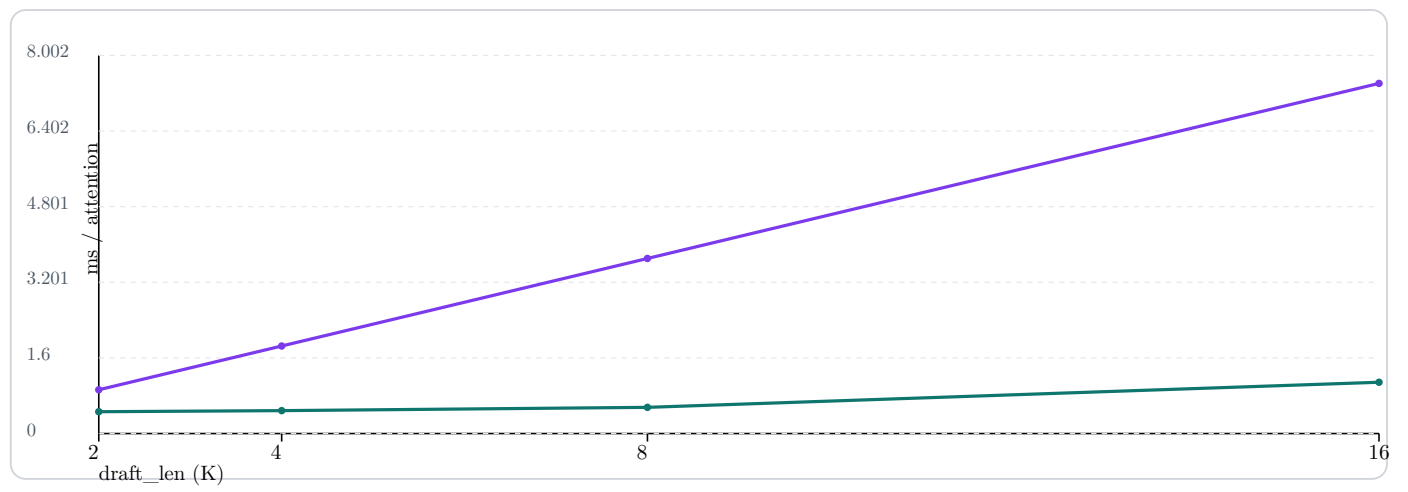
■ AR MLP baseline ($K \times len_1$)



7B-style GQA (d_model=4096, d_ff=11008) - MLP only

■ TiDAR MLP at $L=K+K^2$

■ AR MLP baseline ($K \times \text{len1}$)



A40 Greedy Decode: Staged Softmax vs Greedy No-Softmax

This section separates two different measurements to avoid mixing signals:

- End-to-end full decode pass (includes full model forward cost).
- Isolated sampling microbenchmark (focuses on candidate sampling/rejection/commit path only).

End-to-end full decode (A40, draft_len=8, prefill_tokens=1024, single pass)

These gains are very small and near noise floor for one-pass runs:

- 500m: 62.0088 -> 62.0902 tokens/s (+0.131%)
- 1b: 210.7911 -> 210.8543 tokens/s (+0.030%)
- 3b: 63.2153 -> 63.2564 tokens/s (+0.065%)

Mean speedup across these three runs: +0.075%.

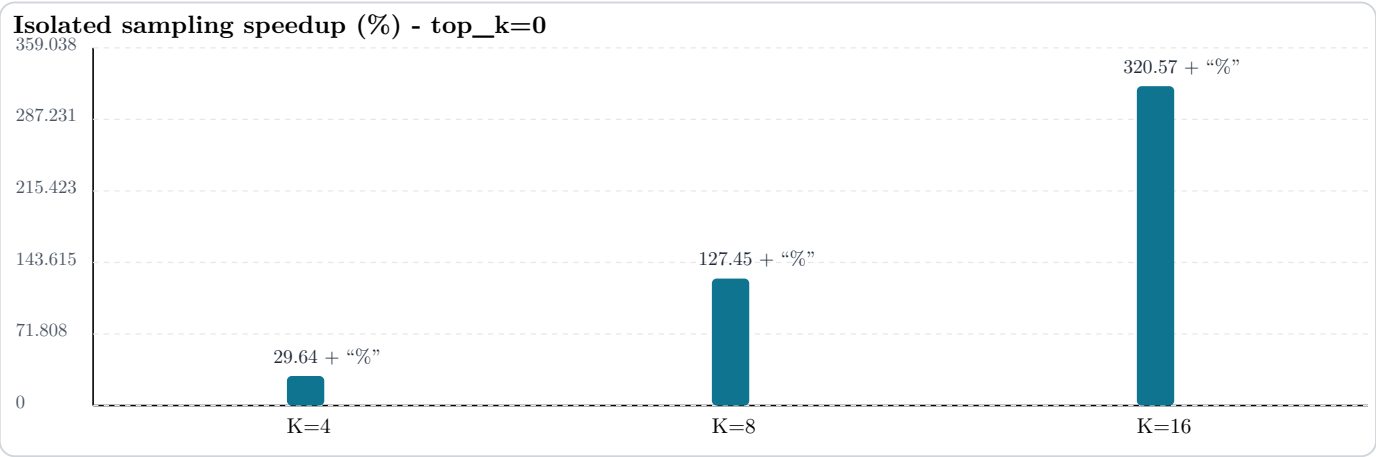
Isolated sampling microbenchmark (not full forward pass)

Benchmark definition:

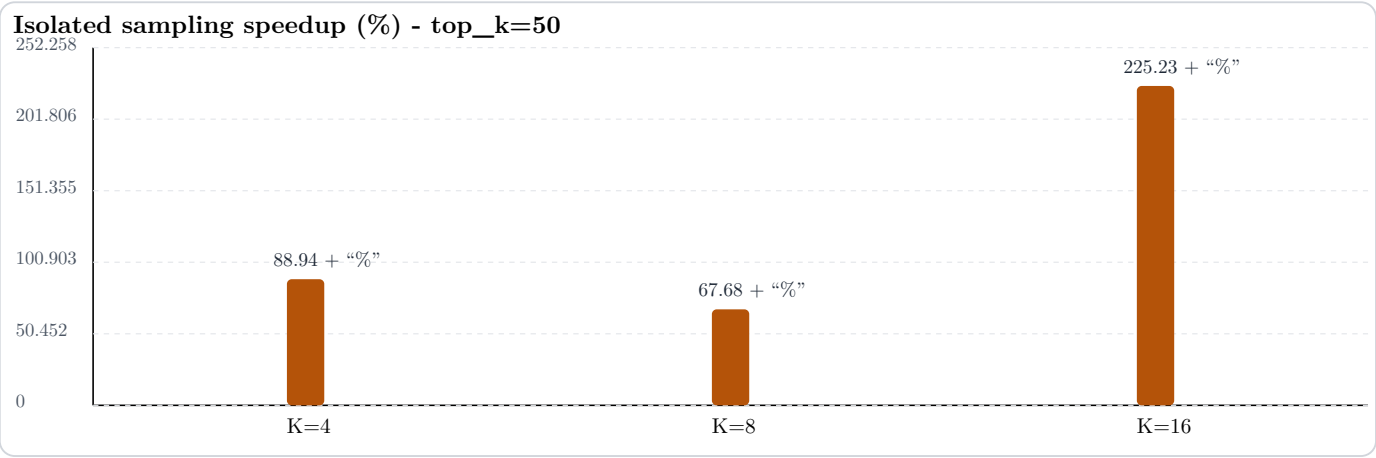
- Old path: sample K (verify) + sample K^2 (all predraft rows) + determine row + commit.
- New path: sample K (verify) + determine row + sample selected K row only + commit.

The following deltas are from this isolated sampling path and are not end-to-end decode speedups.

■ new path speedup over old path



■ new path speedup over old path



A40 Isolated Sampling: AR (B=1) vs TiDAR by K

This page isolates only the sampling/rejection path (no transformer forward) and compares:

- AR baseline sampled at `batch=1`, `len=1`, then scaled by K ($K \times \text{len1}$) for parity with TiDAR committed-token span.
- TiDAR staged path (sample verify, reject/select, sample only selected predraft row).
- Single fixed setup for this sweep: `prefix_len=1024`, vocab 49152, A40.
- Variants: greedy (`temperature=0`, `top_k=0`) and non-greedy (`temperature=1`, `top_k=50`).

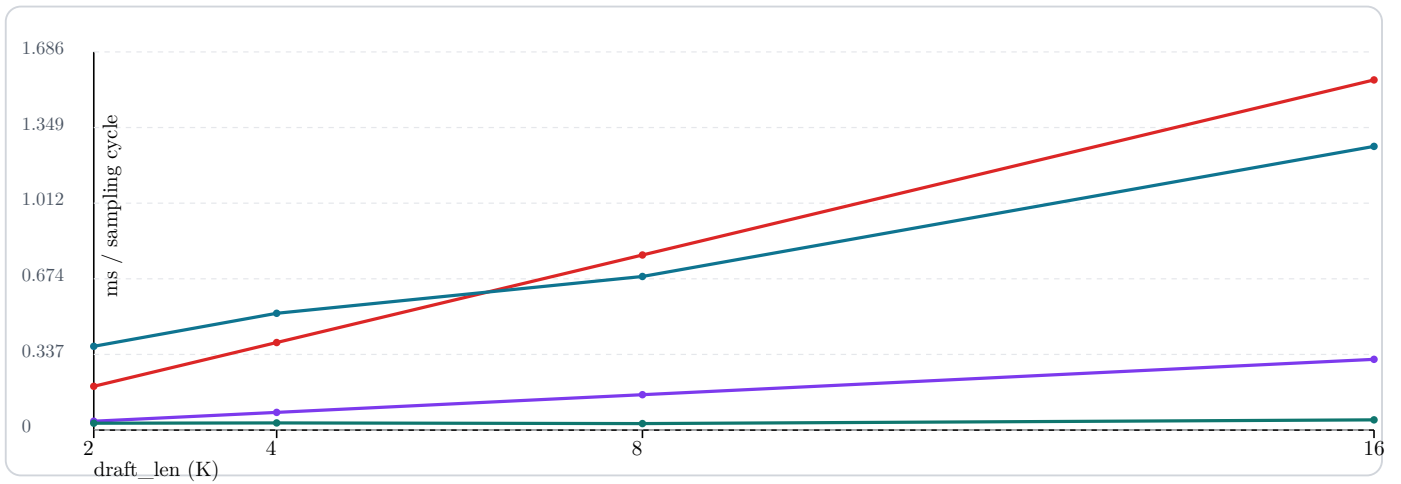
500m sampling-only: AR scaled vs TiDAR staged

■ AR scaled ($K \times \text{len1}$), greedy

■ TiDAR staged sampling, greedy

■ AR scaled ($K \times \text{len1}$), non-greedy `top_k=50`

■ TiDAR staged sampling, non-greedy `top_k=50`



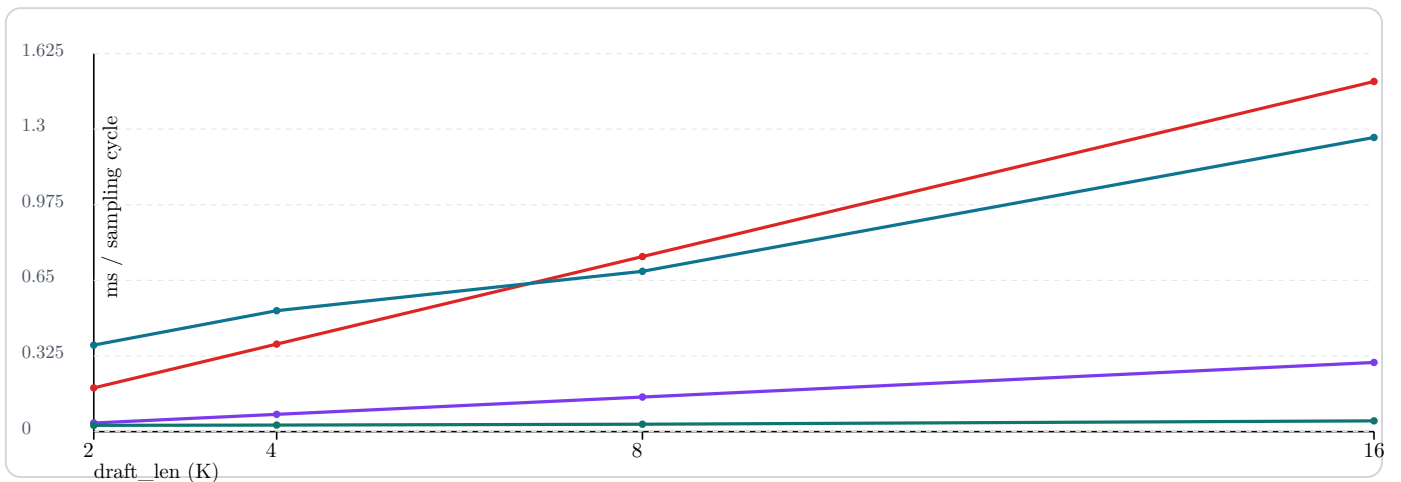
1b sampling-only: AR scaled vs TiDAR staged

■ AR scaled ($K \times \text{len1}$), greedy

■ TiDAR staged sampling, greedy

■ AR scaled ($K \times \text{len1}$), non-greedy `top_k=50`

■ TiDAR staged sampling, non-greedy `top_k=50`



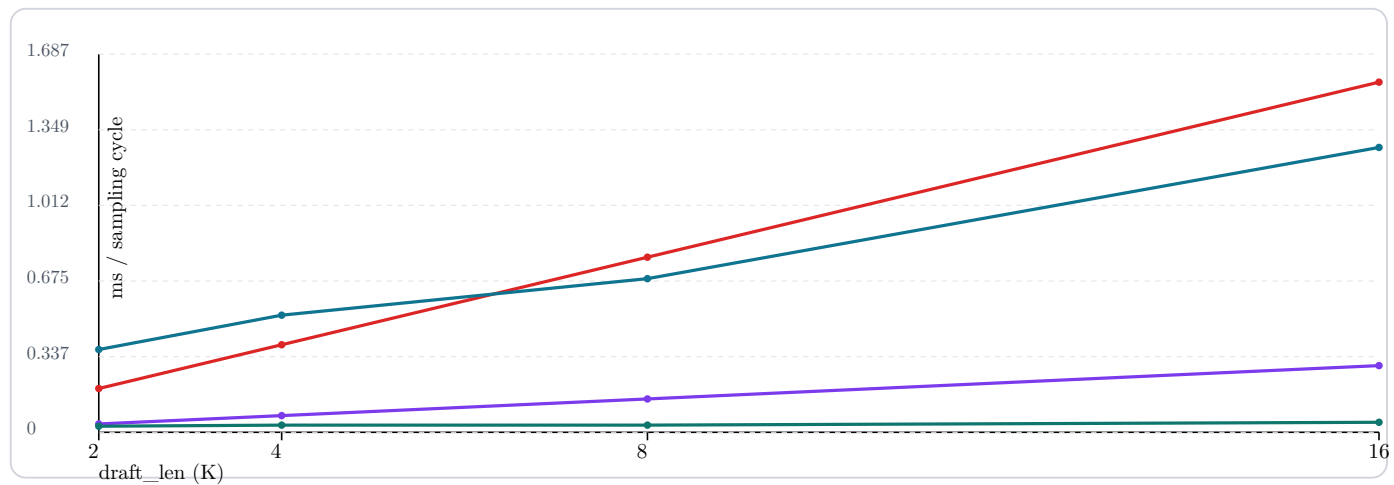
3b sampling-only: AR scaled vs TiDAR staged

■ AR scaled ($K \times \text{len1}$), greedy

■ TiDAR staged sampling, greedy

■ AR scaled ($K \times \text{len1}$), non-greedy $\text{top}_k=50$

■ TiDAR staged sampling, non-greedy $\text{top}_k=50$

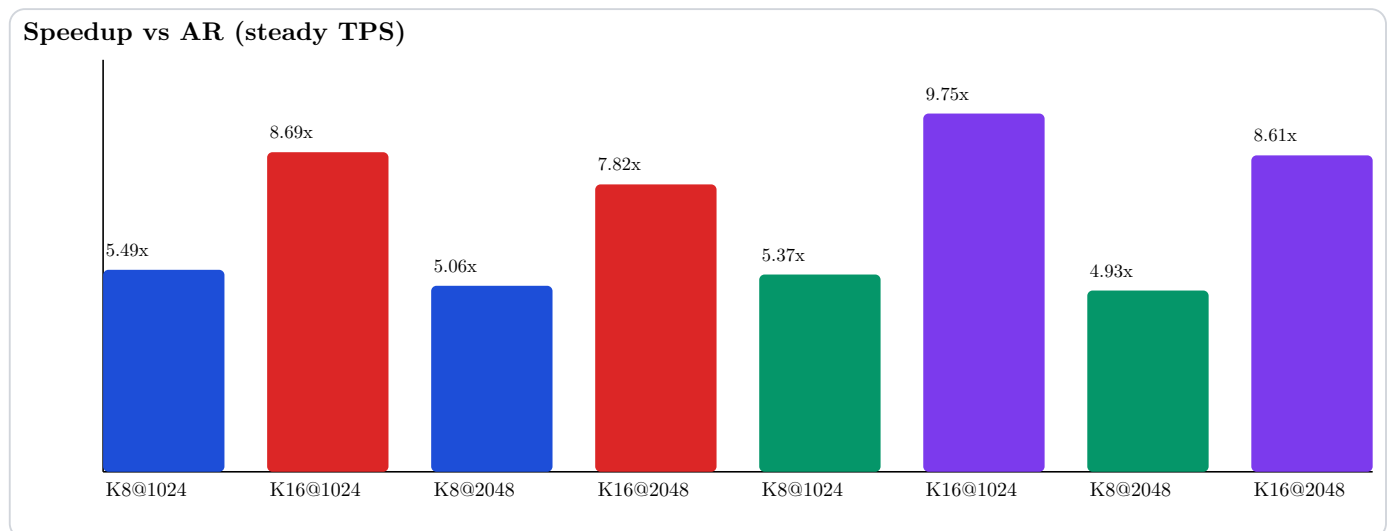
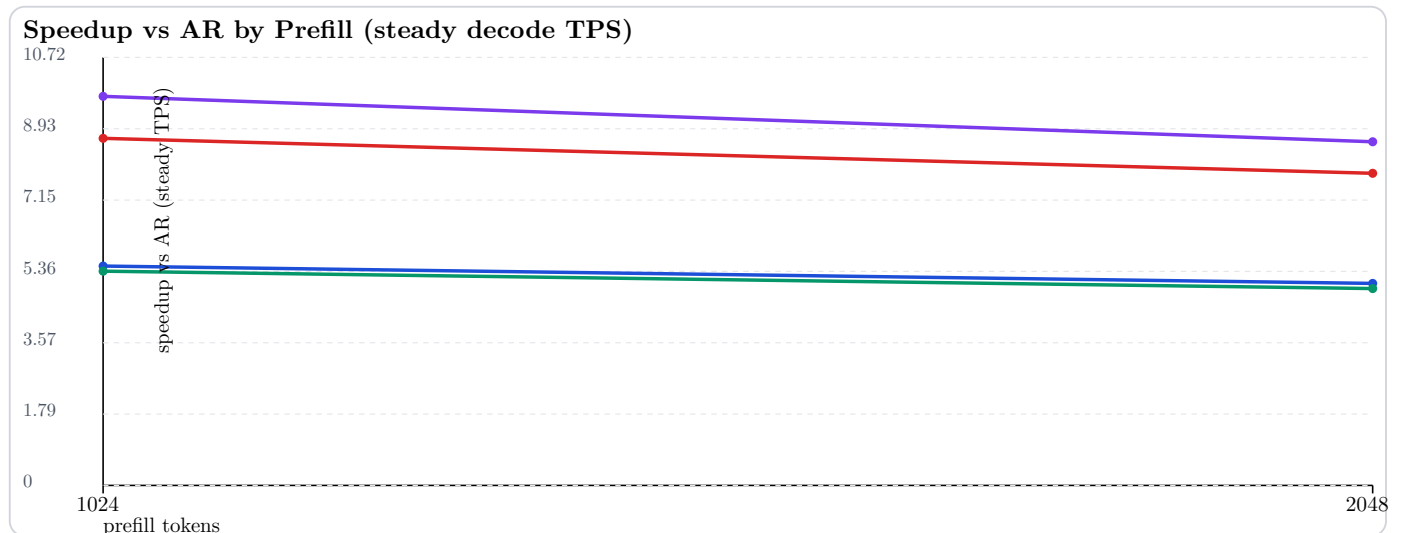


Greedy curves show strong TiDAR sampling advantage as K grows. In non-greedy $\text{top}_k=50$, TiDAR crosses over from slower at low K (2,4) to faster at higher K (8,16).

TiDAR vs AR Head-to-Head (A40 + H100, 3b config)

This document tracks steady-state decode throughput only (compile excluded).

- A40 TiDAR K=8
- A40 TiDAR K=16
- H100 TiDAR K=8
- H100 TiDAR K=16



A40 steady metrics snapshot

- AR steady TPS:
 - prefill 1024: 18.754672
 - prefill 2048: 18.736833
- TiDAR steady TPS:
 - prefill 1024, K=8: 103.042058
 - prefill 1024, K=16: 163.040059
 - prefill 2048, K=8: 94.800140
 - prefill 2048, K=16: 146.475107
- TiDAR postprocess share:
 - prefill 1024, K=8: 0.4775%
 - prefill 1024, K=16: 0.2639%
 - prefill 2048, K=8: 0.4029%
 - prefill 2048, K=16: 0.1782%

H100 steady metrics snapshot

- AR steady TPS:
 - prefill 1024: 55.045774
 - prefill 2048: 54.970468
- TiDAR steady TPS:
 - prefill 1024, K=8: 295.376852
 - prefill 1024, K=16: 536.429106
 - prefill 2048, K=8: 271.031309
 - prefill 2048, K=16: 473.142103
- TiDAR postprocess share:
 - prefill 1024, K=8: 0.5775%
 - prefill 1024, K=16: 0.3762%
 - prefill 2048, K=8: 0.5070%
 - prefill 2048, K=16: 0.2928%

Overlay takeaways

- Speedup vs AR is similar across A40 and H100 at K=8.
- At K=16, H100 shows a stronger speedup curve than A40.
- Absolute TiDAR steady TPS is much higher on H100:
 - roughly 2.86x to 3.29x TiDAR TPS vs A40 for the same config points.

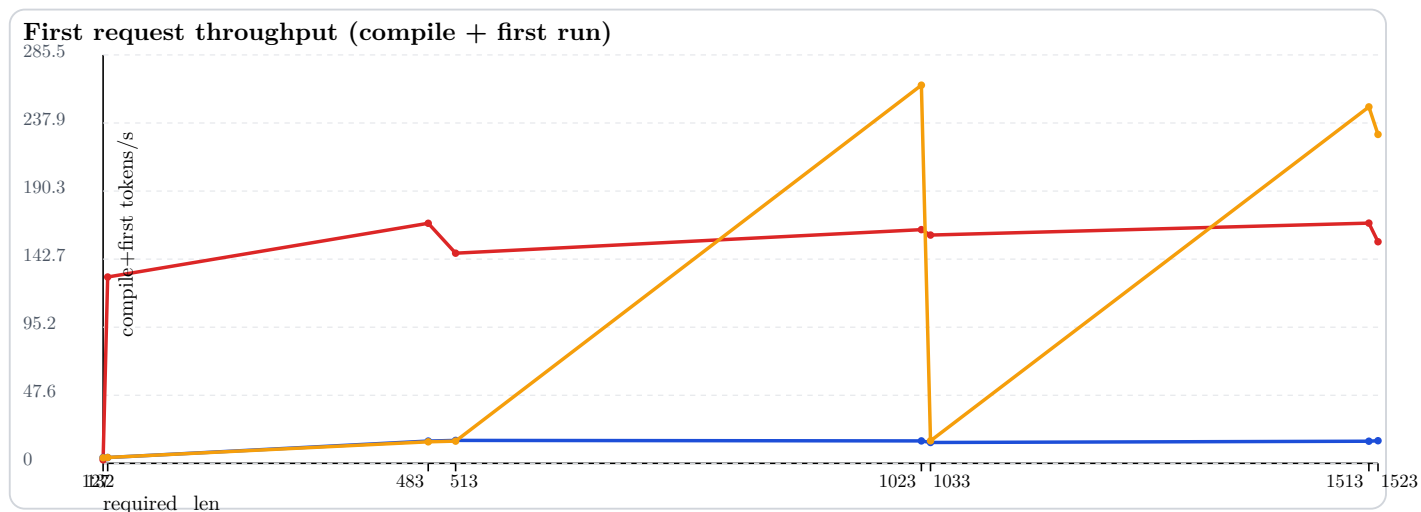
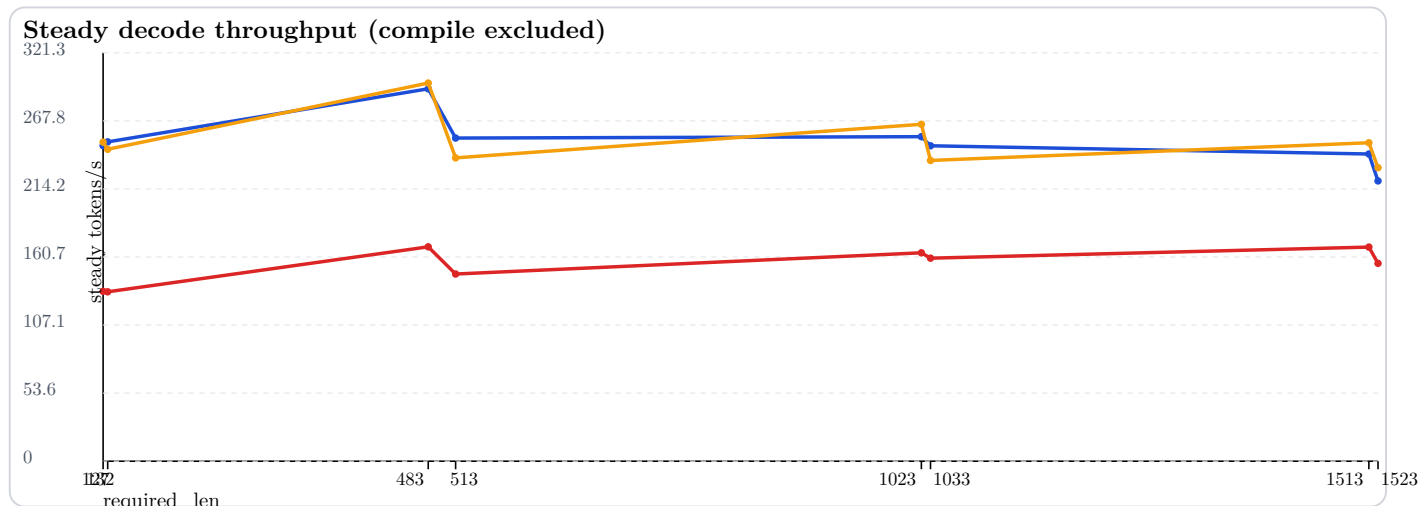
KV Cache Policy Experiment on A40

Red - full context

Blue - current exact

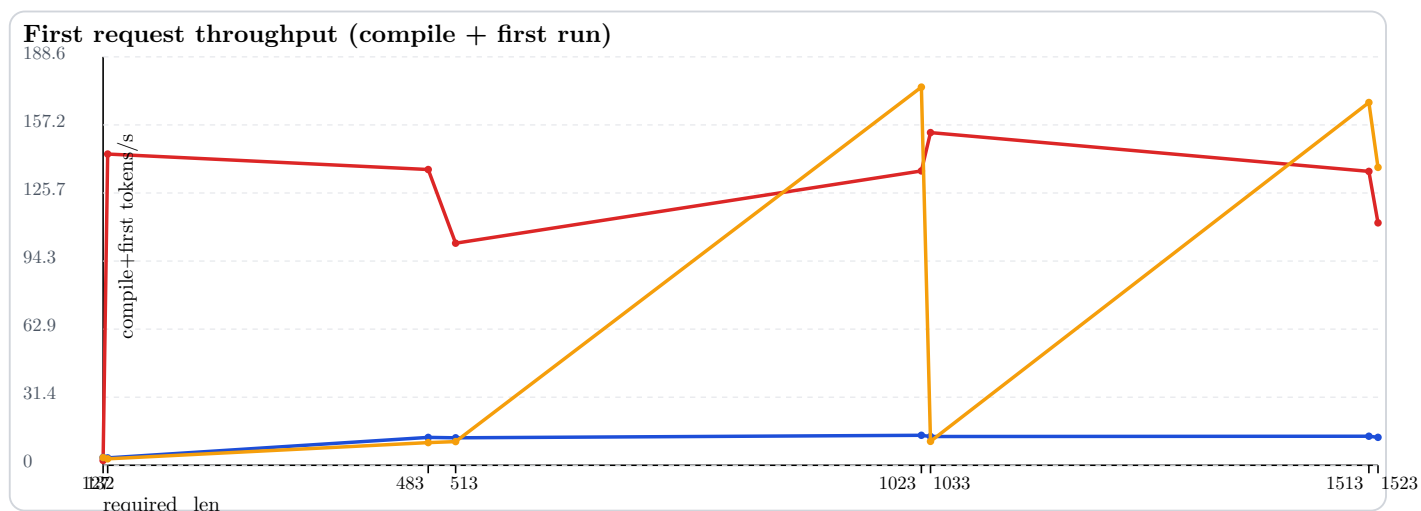
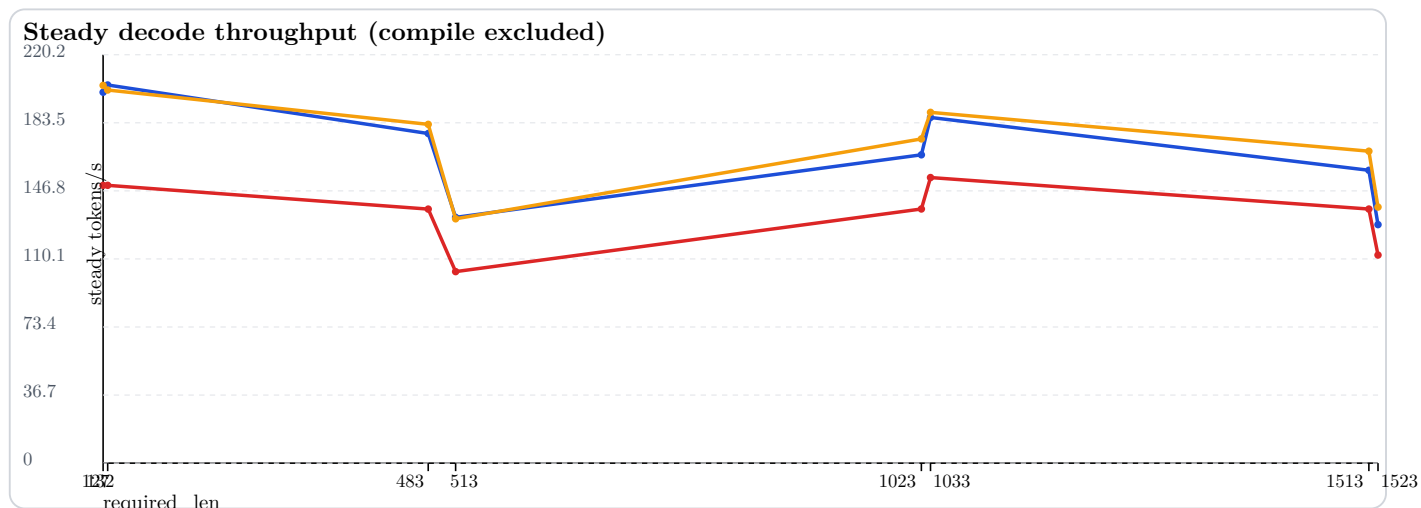
Orange - bucketed

500m model, draft_len=4



- Unique compile keys seen:
 - Red/full: 1
 - Blue/current exact: 8
 - Orange/bucketed: 5
- Mean steady TPS:
 - Red/full: 153.834
 - Blue/current exact: 251.554
 - Orange/bucketed: 251.974

3b model, draft_len=16



- Unique compile keys seen:
 - Red/full: 1
 - Blue/current exact: 8
 - Orange/bucketed: 5
- Mean steady TPS:
 - Red/full: 134.973
 - Blue/current exact: 169.123
 - Orange/bucketed: 173.650

Notes

- Primary comparison metric is steady decode TPS (`steady_tokens_per_second_mean`), i.e. compile excluded.
- Blue/current exact gives the most shape variants (one per `required_len` point in this sweep).
- Orange bucketed reduces shape variants while keeping steady throughput near or above blue in this matrix.

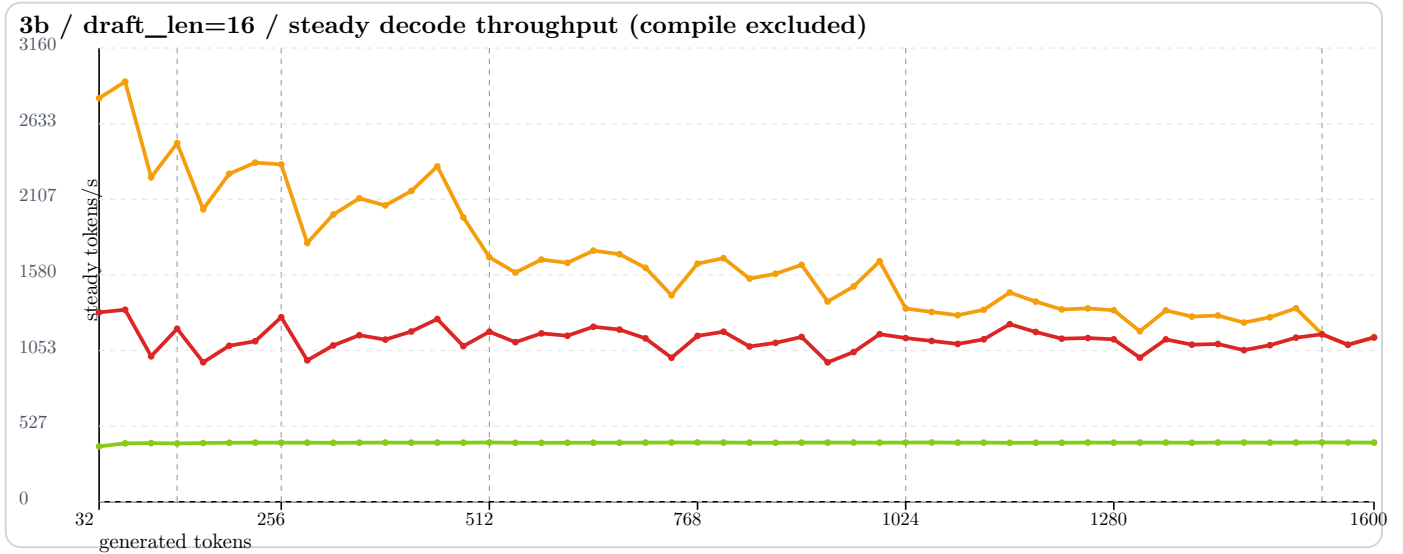
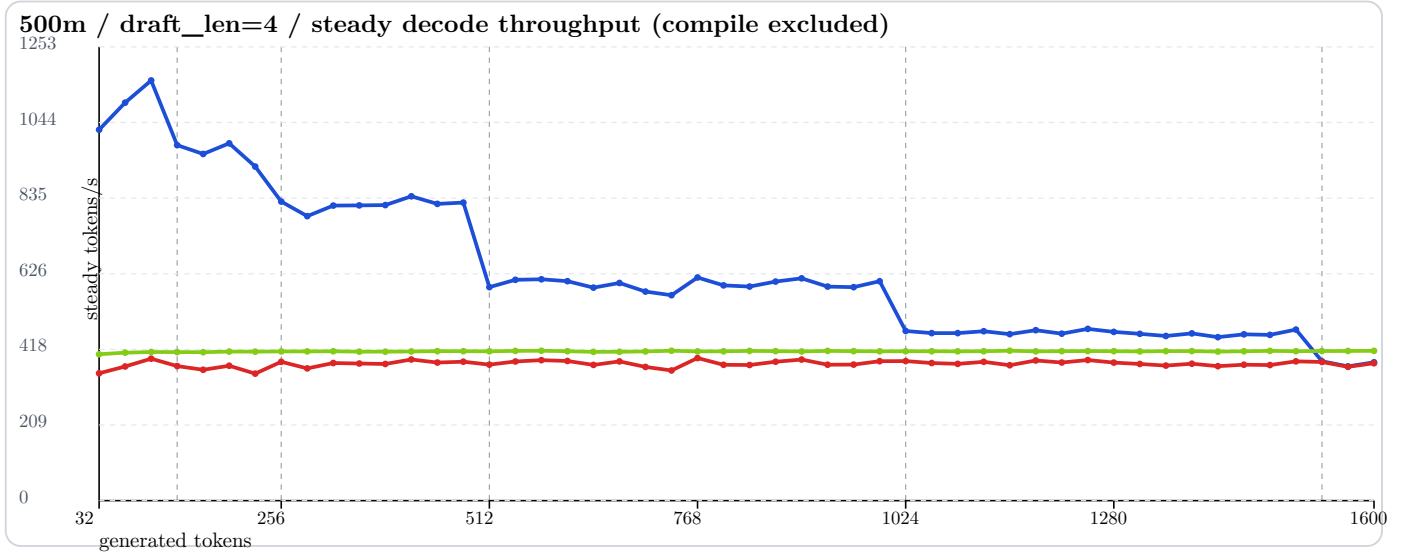
A40 Spot: Bucketed Prefix=0 Long Decode (to 1600 tokens)

500m bucketed, draft_len=4

full-context TiDAR baseline

AR baseline

3b bucketed, draft_len=16



- Prefix length: 0 (synthetic prefill)
- Steps swept: 32..1600 by 32
- Acceptance rate target: 0.8
- Warmup/bench: 1/2
- Unique bucket compile keys: 6 (128, 256, 512, 1024, 1536, 2048)
- Mean steady TPS:
 - 500m/k4: 635.480
 - 3b/k16: 1689.529
 - AR 500m/k4: 412.103
 - AR 3b/k16: 411.368
- Vertical dashed lines are bucket transitions (128, 256, 512, 1024, 1536).
- Metric shown is steady decode TPS (compile excluded): measured after warmup on compiled executables.

TiDAR vs AR Throughput (Updated Single-Forward)

Data files loaded dynamically:

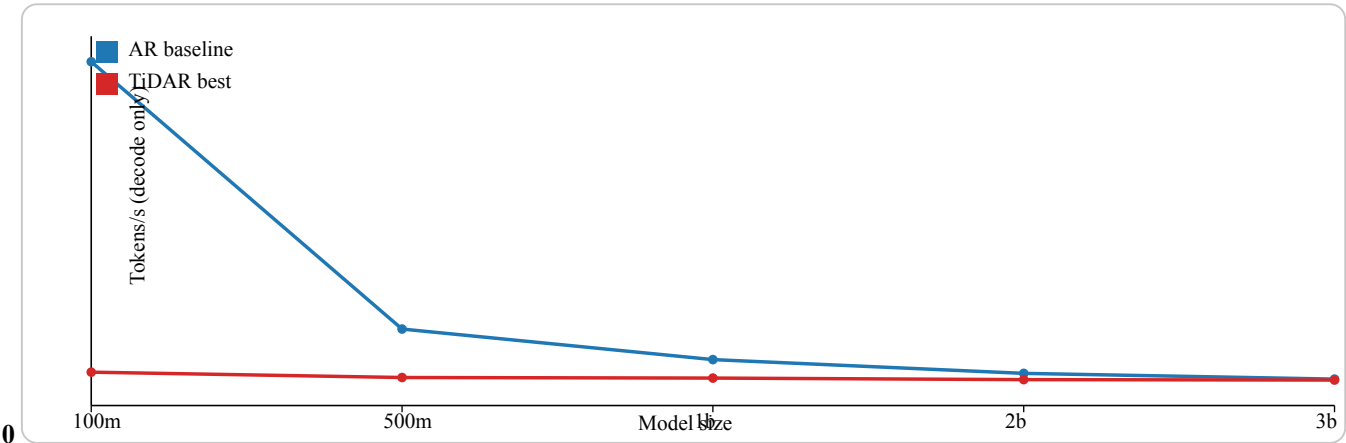
- Benchmark_logs/tidar_vs_ar_single_forward_20260208_134219 + “/best_tidar_vs_ar_by_model_prefill.json”
- Benchmark_logs/tidar_vs_ar_single_forward_20260208_134219 + “/notes.txt”

Sweep:

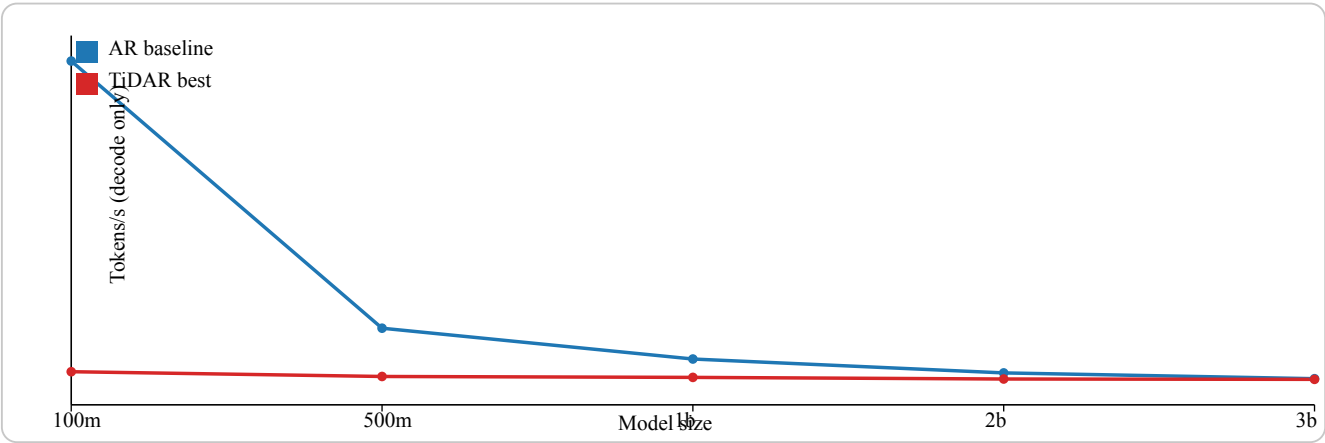
- Models: 100m, 500m, 1b, 2b, 3b
- TiDAR grid: draft_len {4, 8, 16}, accept target {0.6, 0.8}, prefill {0, 1024, 4096}
- Decode steps: 256, repeat: 2
- AR baseline: reused prior run

Line graphs: tokens/s vs model size

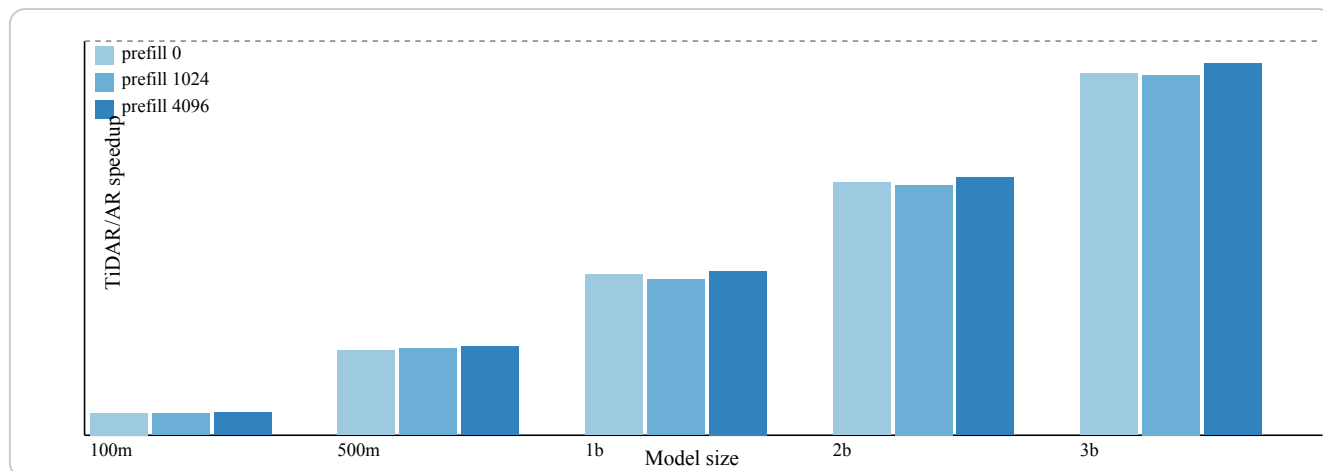
Prefill



Prefill



Histogram-style grouped bars: TiDAR/AR speedup



Key findings

- Best pair: model 3b, prefill 4096, speedup 0.943x, TiDAR draft 16 at accept 0.8.
- Worst pair: model 100m, prefill 1024, speedup 0.056x.
- Trend in this setup: speedup rises with model size and higher draft, but stays below 1.0x on A40 in current JAX path.

Provenance

Updated TiDAR single-forward sweep vs AR baseline

created_at: 2026-02-08T13:42:19.639340

tidar_summary: TiDAR/Docs/Benchmark_logs/tidar_updated_narrow_20260208_094343/summary.csv

ar_summary: TiDAR/Docs/Benchmark_logs/ar_simple_baseline_sweep_20260207_103142/summary.json

best_global_speedup: 0.9431 at model=3b prefill=4096 draft=16 accept=0.8

worst_global_speedup: 0.0558 at model=100m prefill=1024 draft=4 accept=0.6

pairs_compared: 15

H100 Chat Session Addendum (Mask-Path Comparison)

This section documents the extra H100 PCIe runs executed during this chat so the results are preserved in one place.

What was compared (described by runtime behavior, not git state names):

- Path A — Full-width decode mask path:
 - Decode bias is built over full keys as [PREFIX | STEP] (`build_decode_bias_template(cache_len, draft_len, ...)`).
 - Decode attention builds `k_full = [k_prefix, k_step]`.
 - Per-step prefix validity is added as explicit bias, then one dense `dot_product_attention` call is run.
 - This is the current baseline inference path used in the earlier H100 sweep.
- Path B — Local experimental mask-layout branch:
 - Decode bias source is switched to step-first layout (`build_decode_bias_template_step_prefix(...)`) in inference/throughput code.
 - Attention decode path in `Transformer_block.py` is expanded with:
 - single-call step-first K/V path using `key_value_seq_lengths`,
 - legacy fallback for prefix-first full-width masks,
 - step-only split-attention merge path (prefix attention + step attention).
 - Additional step-only bias builders were added in `tidar_core.py` and prefill bias wiring was changed to step-only for prompt+draft prefill.

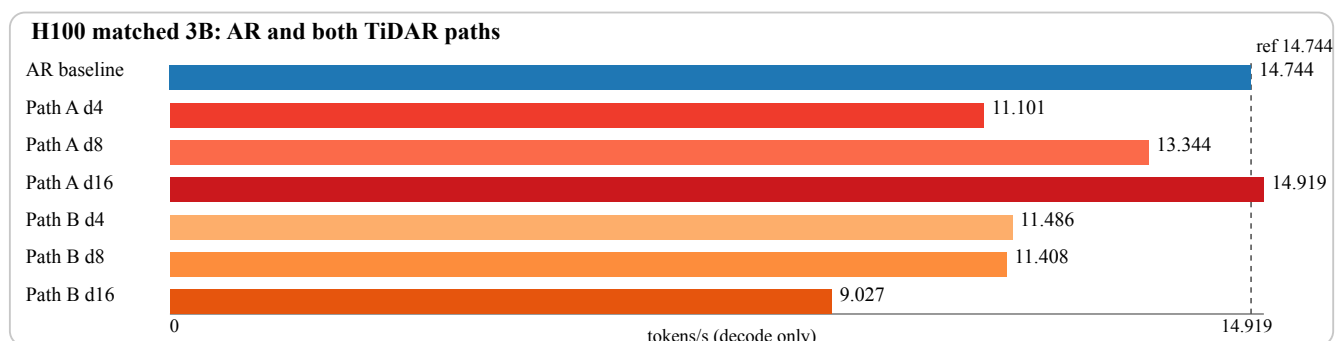
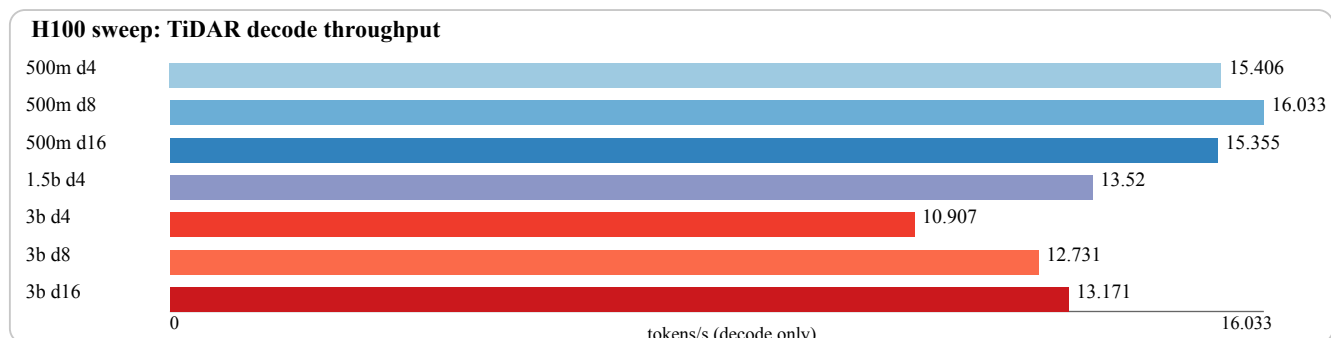
Hardware and run settings

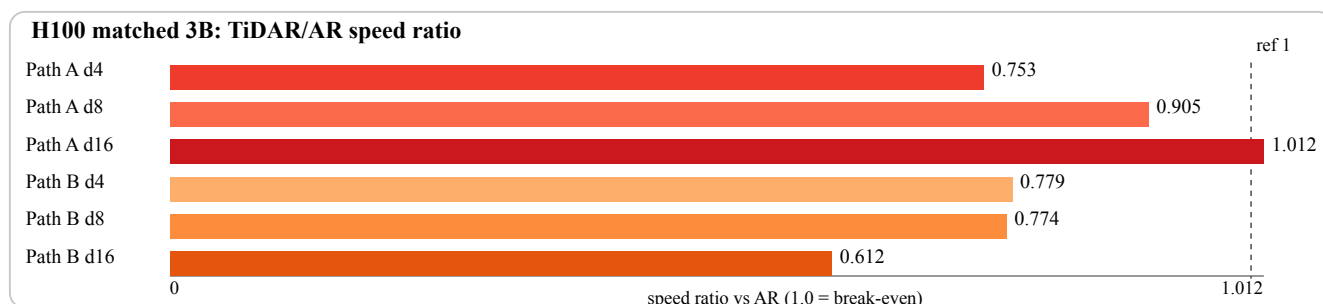
- GPU: NVIDIA H100 PCIe (81,559 MiB)
- Remote host alias used: `gpu-box-1`
- Decode context setting: `context_length = 4608`
- Prefill setting: `prefill_tokens = 4096`
- Decode steps: 256
- Seed: 0
- Acceptance target for TiDAR throughput tests: `accept-rate = 0.8`

Scripts used in this chat

- TiDAR throughput script:
 - `TiDAR/model/test_inference_throughput.py`
- AR decode-only helper used for matched 3B comparison:
 - `/tmp/ar_decode_throughput_current.py`
 - (temporary helper; uses the same TiDAR model stack and KV-cache path for one-token AR decode)

Visual summaries





Recorded H100 metrics from this chat

First H100 TiDAR sweep (Path A, decode-only tokens/s):

- 500m, draft 4: 15.405662 tok/s
- 500m, draft 8: 16.033103 tok/s
- 500m, draft 16: 15.354966 tok/s
- 1.5b, draft 4: 13.519934 tok/s
- 3b, draft 4: 10.906801 tok/s
- 3b, draft 8: 12.730544 tok/s
- 3b, draft 16: 13.171004 tok/s

Matched 3B run (same settings above), AR + TiDAR path comparison:

- AR baseline (decode-only):
 - generated tokens: 256
 - iterations: 256
 - decode time: 17.362658 s
 - throughput: 14.744286 tok/s
- TiDAR, Path A, draft 4:
 - generated tokens: 256
 - iterations: 78
 - observed avg accept/iter: 3.282051
 - observed max accept/iter: 4
 - decode time: 23.061257 s
 - throughput: 11.100869 tok/s
- TiDAR, Path A, draft 8:
 - generated tokens: 256
 - iterations: 39
 - observed avg accept/iter: 6.564103
 - observed max accept/iter: 8
 - decode time: 19.184134 s
 - throughput: 13.344361 tok/s
- TiDAR, Path A, draft 16:
 - generated tokens: 256
 - iterations: 21
 - observed avg accept/iter: 12.190476
 - observed max accept/iter: 16
 - decode time: 17.159882 s
 - throughput: 14.918518 tok/s
- TiDAR, Path B, draft 4:
 - generated tokens: 256
 - iterations: 78
 - observed avg accept/iter: 3.282051
 - observed max accept/iter: 4
 - decode time: 22.288248 s
 - throughput: 11.485874 tok/s
- TiDAR, Path B, draft 8:
 - generated tokens: 256
 - iterations: 39

- observed avg accept/iter: 6.564103
- observed max accept/iter: 8
- decode time: 22.439773 s
- throughput: 11.408315 tok/s
- TiDAR, Path B, draft 16:
 - generated tokens: 256
 - iterations: 21
 - observed avg accept/iter: 12.190476
 - observed max accept/iter: 16
 - decode time: 28.360653 s
 - throughput: 9.026591 tok/s

Quick interpretation from these runs

- In this matched 3B run, Path A at draft 16 slightly exceeded AR baseline throughput (14.918518 vs 14.744286 tok/s).
- Path B improved only draft 4 slightly, but regressed strongly for draft 8/16.
- Net result from this chat: the current local experimental branch did not improve overall TiDAR throughput on H100 in the tested settings.

TiDAR: Short Summary + Throughput Snapshot

What TiDAR Does

TiDAR (Think in Diffusion, Talk in Autoregression) keeps a single decoder-only transformer but trains it to operate under two synchronized attention regimes:

- **Talk (AR)**: standard causal self-attention for verifying the current draft tokens.
- **Think (Diffusion)**: blockwise bidirectional attention for drafting the next candidate tokens.

Training doubles each sequence (clean + masked copy) so the diffusion branch predicts aligned targets with blockwise masks; inference uses structured masks to verify the current K tokens and pre-draft the next K candidates in a single forward pass.

One-Pass Inference Sketch

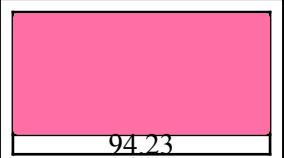
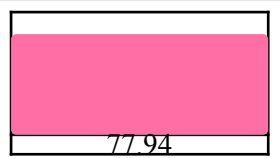
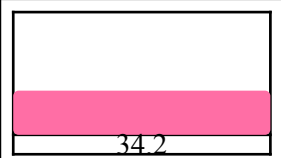
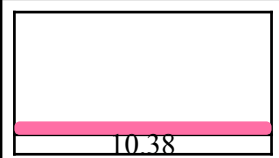
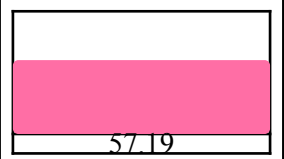
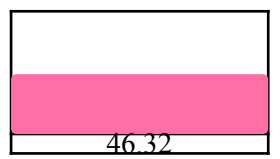
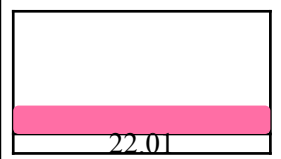
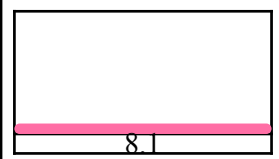
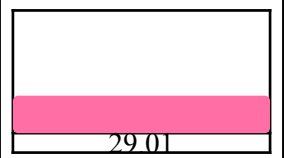
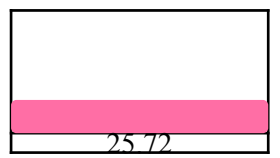
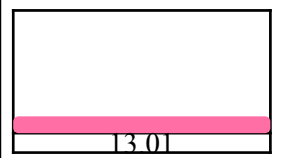
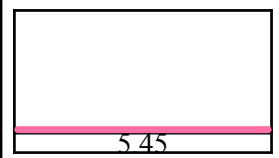
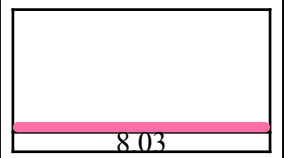
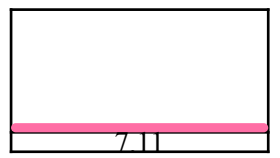
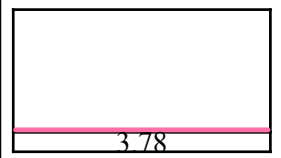
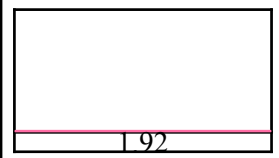
prefix tokens $\longrightarrow$ Talk (AR verify) $\longrightarrow$ commit r tokens

verify tokens (K) $\longrightarrow$

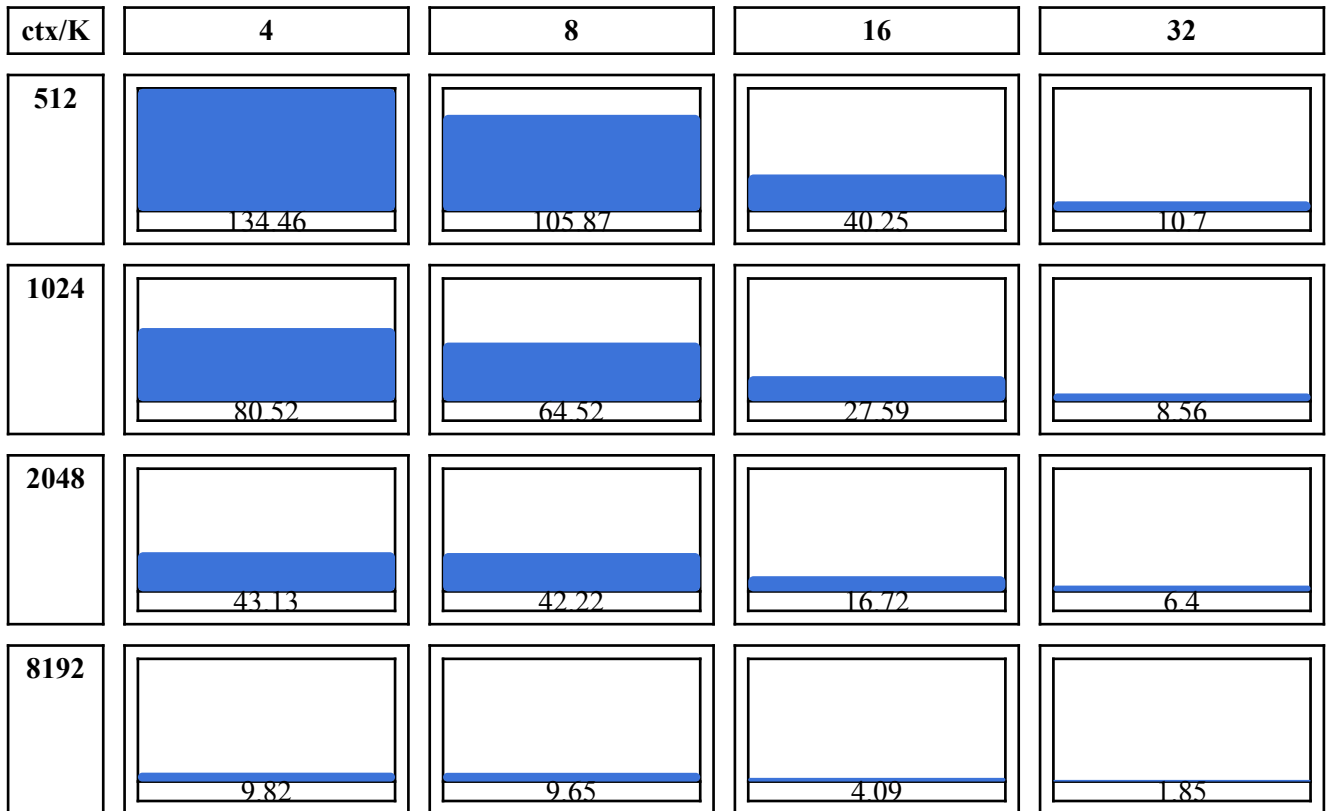
candidate blocks (K×K) $\rightarrow$ Think (Diffusion draft) $\rightarrow$ next K proposals

Throughput Sweep (RTX 3060)

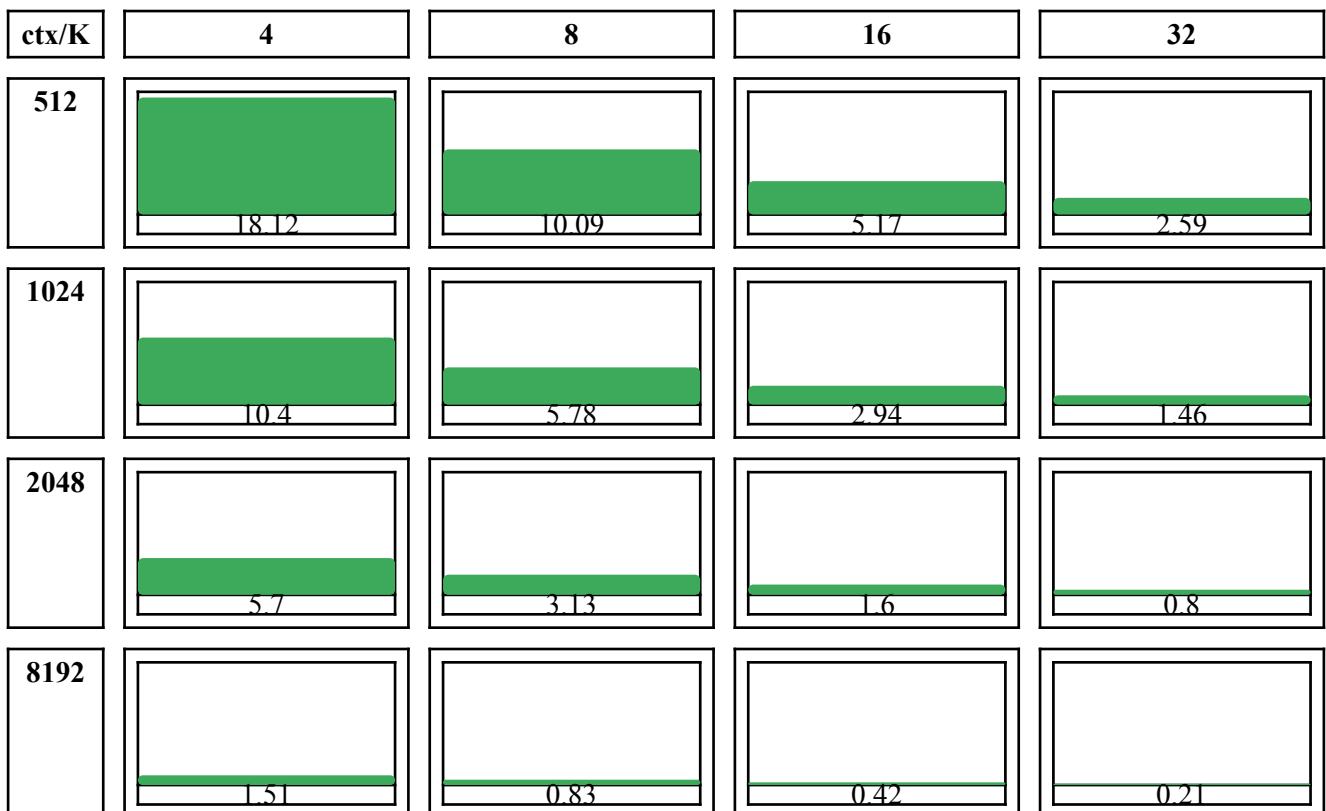
dense_bigmask_scan (iter/s)

ctx/K	4	8	16	32
512	 94.23	 77.94	 34.2	 10.38
1024	 57.19	 46.32	 22.01	 8.1
2048	 29.01	 25.72	 13.01	 5.45
8192	 8.03	 7.11	 3.78	 1.92

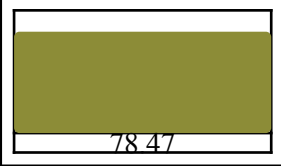
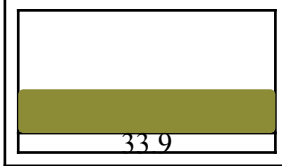
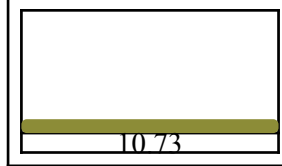
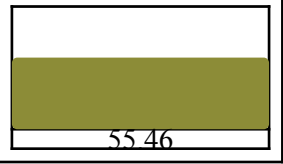
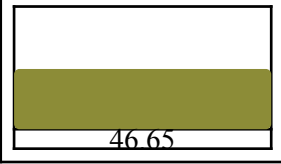
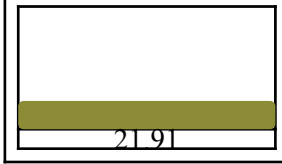
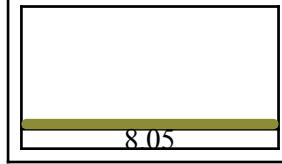
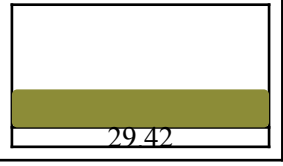
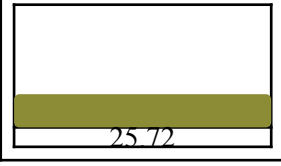
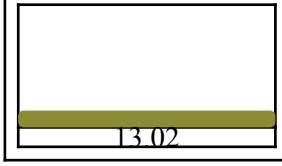
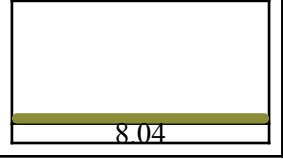
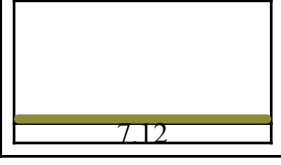
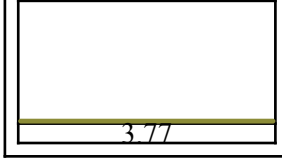
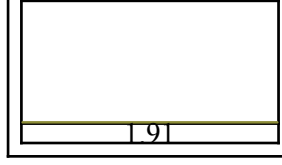
dense_tightkeys_scan (iter/s)



split_calls_scan (iter/s)



pallas_optional_scan (iter/s)

ctx/K	4	8	16	32
512	 93.65	 78.47	 33.9	 10.73
1024	 55.46	 46.65	 21.91	 8.05
2048	 29.42	 25.72	 13.02	 5.55
8192	 8.04	 7.12	 3.77	 1.91

Notes

- Metric: iter/s (1 / seconds_per_iter).
- Prefill length: round(0.66 * context_len).
- Config: heads=16, d=64, steps=128, iters=10, dtype=bf16, impl=cudnn.

TiDAR 105k Inference Head-to-Head

Acceptance-throughput dashboard

Overall Winner

BigGamma+Delta

based on inference accept rate

Mean avg_accept/iter

2.2981 vs 2.2117

+0.0864 to BigGamma

Accept-Prob Proxy

$(2.2981-1)/5 = 0.2596$

TopK: 0.2423

Pairwise Wins

19 / 36

BigGamma better runs

Primary Sweep (12 prompts, 96 steps)

Greedy

BigGamma



2.3092

TopK



2.1942

Sampled seed 0

BigGamma



2.29

TopK



2.225

Sampled seed 1

BigGamma



2.295

Robustness (12 prompts, 256 greedy steps)

Long-run greedy

BigGamma



2.33

TopK



1.9833

Mean delta: +0.3467

Prompt winners: BigGamma 8, TopK 4

TopK



2.2158

Quick Take

If priority is inference acceptance throughput, continue BigGamma+Delta to 121k.

Caveat: monitor repetition/quality in long greedy runs while optimizing acceptance.

Checkpoints compared at equal step 105000: .../tidar_bigGamma_andDelta/params/step_0105000.npz vs .../tidar_later_stage_topk/params/step_0105000.npz.

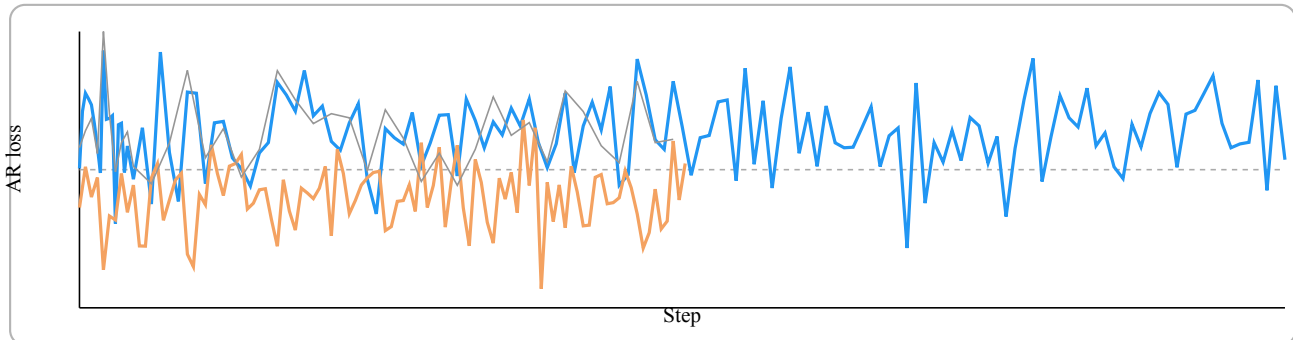
Results: Greedy Runs (135M vs 360M), draft length 8

Metric legend for 5 hour run, total cost 17.5 \$

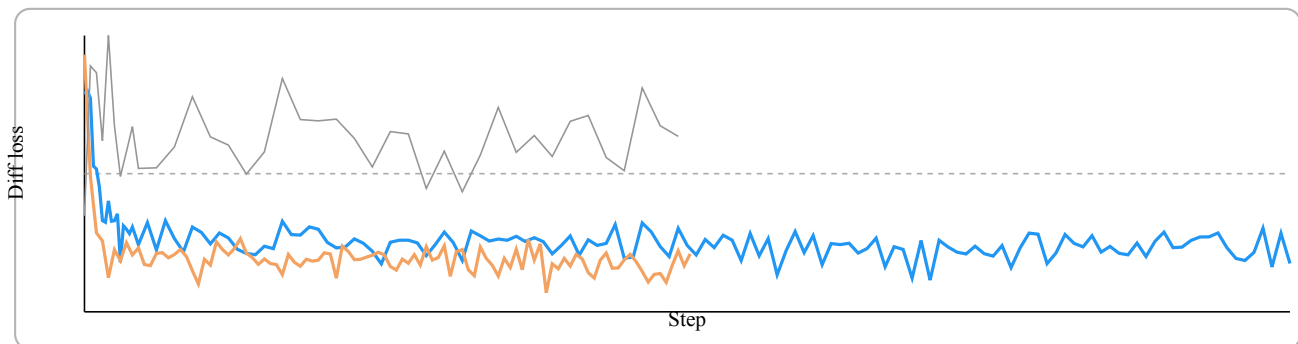
■ 135M - 5.23\$ on 5090 with 32GB VRAM ■ 360M - 11.81\$ on RTX PRO 6000 with 96GB VRAM

The bigger model ran with 50% bigger batch size but logs are synced by optimizer steps.

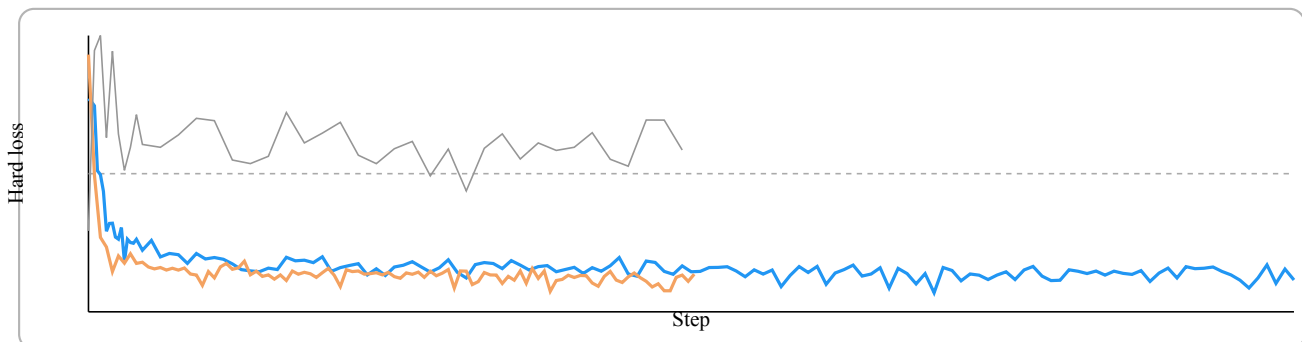
AR loss (lower is better) - This loss is not important for this experiment



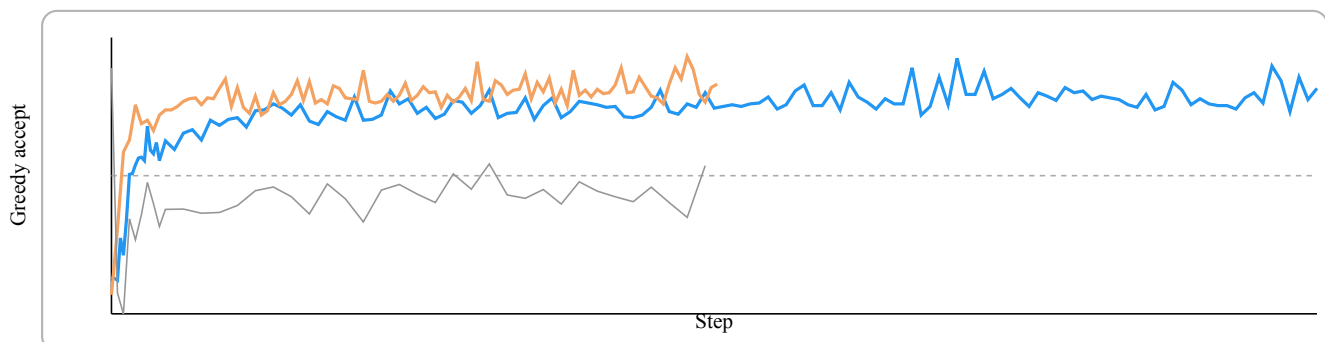
Diffusion loss (lower is better)



Hard (greedy) loss (lower is better)



Greedy acceptance (higher is better)



Inference (Once upon a time, verbose)

135M (step_0040000)

Output: “Once upon a time, John Grusky, the first White House astronomer, was chosen as a first-year intern. The first thing he saw was Mount Tai, in sight of the top of Mount Fresar in the Midwest. At first, Grusky didn’t think of it as a comet. He loved comparing it to the”

- **n_iterations:** 44
- **avg_accept_per_iter:** 1.43
- **max_accept_per_iter:** 4
- **tokens_per_second:** 5.50

360M (step_0020000)

Output: “Once upon a time, there lived a wicked queen who was such a horrible figure that she even had to have a cat to put off her spirit. There was a girl named Rapunzel, who loved to play in the forest. Her father, the King, had given up his kingdom for a beautiful princess who loved him dearly.”

- **n_iterations:** 45
- **avg_accept_per_iter:** 1.40
- **max_accept_per_iter:** 4
- **tokens_per_second:** 3.44

Verbose output from generation with the 135m model

prompt: Once upon a time
draft len: 8; max steps: 12

```
anchor: , -> there
currently verified draft: (,) | the | was | a | the | the | the | the
sample from verified draft: there | was | a | the | the | the | the | the
accept 1 of 8: there
select 1-th draft for next: there | was | a | a | the | the | the | the

anchor: there -> man
currently verified draft: ( there) | was | a | a | the | the | the | the
sample from verified draft: was | a | man | the | the | the | the | the
accept 3 of 8: was | a | man
select 3-th draft for next: man | named | a | the | . | the | the | the

anchor: man -> John
currently verified draft: ( man) | named | a | the | . | the | the | the
sample from verified draft: named | John | the | . | the | the | the | the
accept 2 of 8: named | John
select 2-th draft for next: John | . | . | He | was | a | a | .

anchor: John -> He
currently verified draft: ( John) | . | . | He | was | a | a | .
sample from verified draft: . | He | He | was | a | a | . | He
accept 2 of 8: . | He
select 2-th draft for next: He | was | was | a | a | , | , | ,

anchor: He -> a
currently verified draft: ( He) | was | was | a | a | , | , | ,
sample from verified draft: was | a | a | a | , | , | , | and
accept 2 of 8: was | a
select 2-th draft for next: a | man | man | man | man | He | He | He

anchor: a -> very
currently verified draft: ( a) | man | man | man | man | He | He | He
sample from verified draft: very | man | man | man | He | He | He | He
accept 1 of 8: very
select 1-th draft for next: very | man | He | He | He | He | He | He

final output
Once upon a time, there was a man named John. He was a very
```

Verbose output from generation with the 360m model

prompt: Once upon

draft len: 8; max steps: 12

```
anchor: a      -> in
currently verified draft: ( a) | time | , | a | a | a | a | a
sample from verified draft: time | , | in | a | a | a | a | a
accept 3 of 8:          time | , | in
select 3-th draft for next: in | was | a | a | a | a | a | the

anchor: in     -> a
currently verified draft: ( in) | was | a | a | a | a | a | the
sample from verified draft: a | a | a | a | a | a | the | a
accept 1 of 8:          a
select 1-th draft for next: a | small | , | , | , | , | , | ,

anchor: a      -> land
currently verified draft: ( a) | small | , | , | , | , | , | ,
sample from verified draft: land | , | , | , | , | , | , | town
accept 1 of 8:          land
select 1-th draft for next: land | town | , | , | , | , | a | a

anchor: land   -> far
currently verified draft: ( land) | town | , | , | , | , | a | a
sample from verified draft: far | , | , | , | , | a | a | man
accept 1 of 8:          far
select 1-th draft for next: far | the | , | , | , | , | a | a

anchor: far    -> ,
currently verified draft: ( far) | the | , | , | , | , | a | a
sample from verified draft: , | , | , | , | , | a | a | ,
accept 1 of 8:          ,
select 1-th draft for next: , | , | , | a | a | a | a | a

anchor: ,      -> far
currently verified draft: (,) | , | , | a | a | a | a | a
sample from verified draft: far | , | a | a | a | a | a | a
accept 1 of 8:          far
select 1-th draft for next: far | , | , | a | a | a | a | a

anchor: far    -> away
currently verified draft: ( far) | , | , | a | a | a | a | a
sample from verified draft: away | , | a | a | a | a | a | a
accept 1 of 8:          away
select 1-th draft for next: away | , | a | a | a | a | a | a

anchor: away   -> there
currently verified draft: ( away) | , | a | a | a | a | a | a
sample from verified draft: , | there | a | a | a | a | a | a
accept 2 of 8:          , | there
select 2-th draft for next: there | a | a | a | a | a | . | .
```

final output

Once upon a time, in a land far, far away, there

TiDAR Sweep 4-5 - smollm-135m loss trends

Setup

All four runs use smollm-135m (draft_length=5, context_length=2048). The loss combines AR and Diff terms plus an agreement penalty when lambda > 0:

$$L_{\text{total}} = \frac{\alpha * L_{\text{AR}} + L_{\text{Diff}}}{1 + \alpha} + \lambda * \text{KL}(p_{\text{AR}} \parallel p_{\text{Diff}})$$

Sweep summaries

- Sweep 1: lr=1e-4, alpha=1.0, lambda=0.0, warmup=2,000, batch=8
- Sweep 2: lr=5e-5, alpha=0.7, lambda=0.1, warmup=1,500, batch=8
- Sweep 3: lr=3e-5, alpha=0.3, lambda=0.5, warmup=1,200, batch=8 (aggressive agreement)
- Sweep 4: lr=1e-5, alpha=0.5, lambda=0.2, warmup=800, batch=32 (4x batch)
- Sweep 5: lr=3e-5, alpha=0.2, lambda=0.8, KL temp=2.0, draft_len=2, ctx-mix (<=300M tokens)

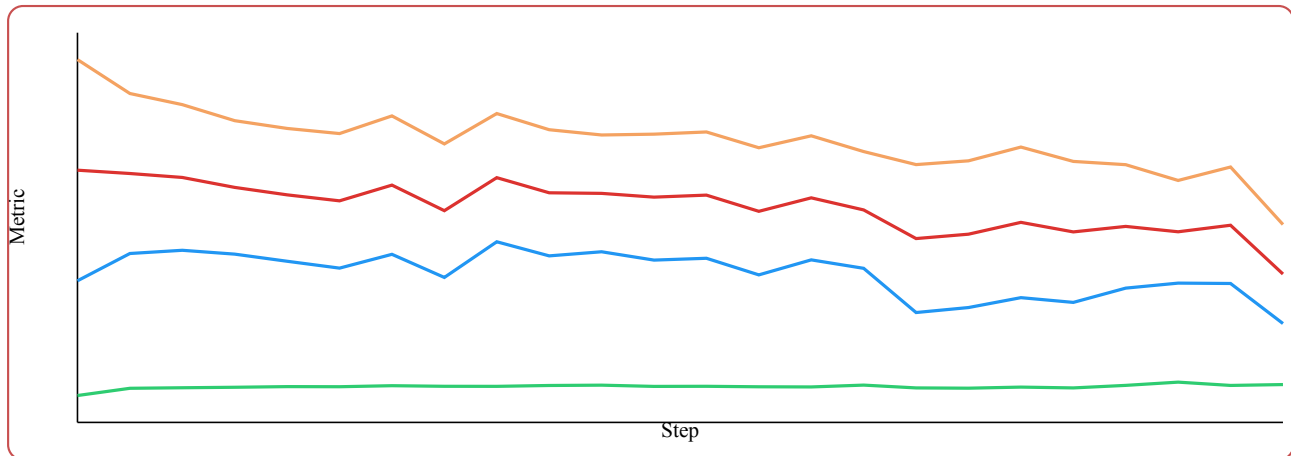
Metric legend



Sweep 1 - Balanced AR/Diff, no agreement penalty

This run keeps alpha=1.0 and lambda=0, so the total loss is the straight average of AR and Diff losses.

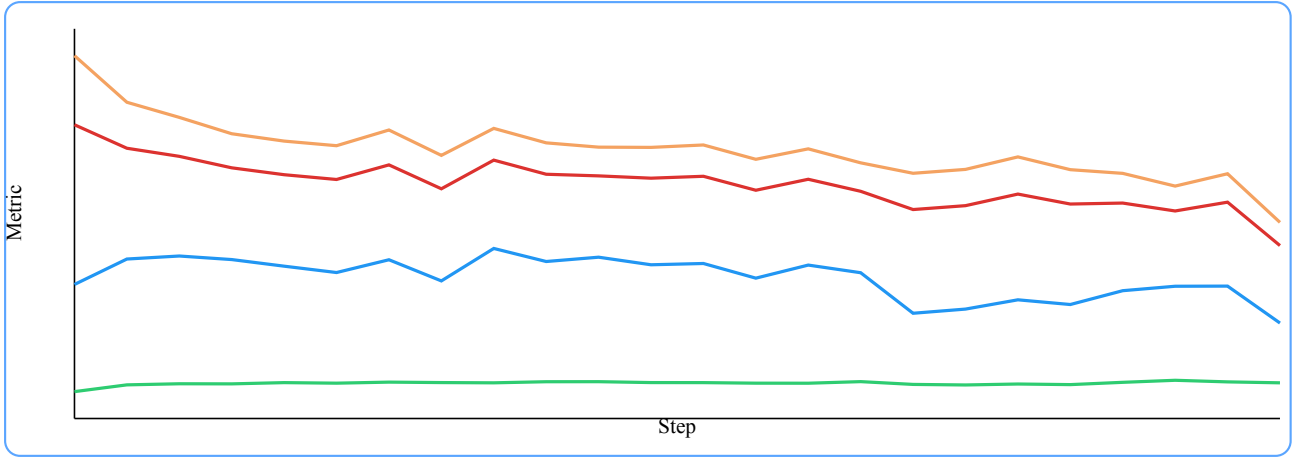
$$L_{\text{total}} = \frac{1.0 * L_{\text{AR}} + L_{\text{Diff}}}{2.0} + 0.0 * \text{KL}(p_{\text{AR}} \parallel p_{\text{Diff}})$$



Sweep 2 - Gentle agreement regularization

Lower alpha with a small lambda nudges Diff toward AR while preserving a steady training curve.

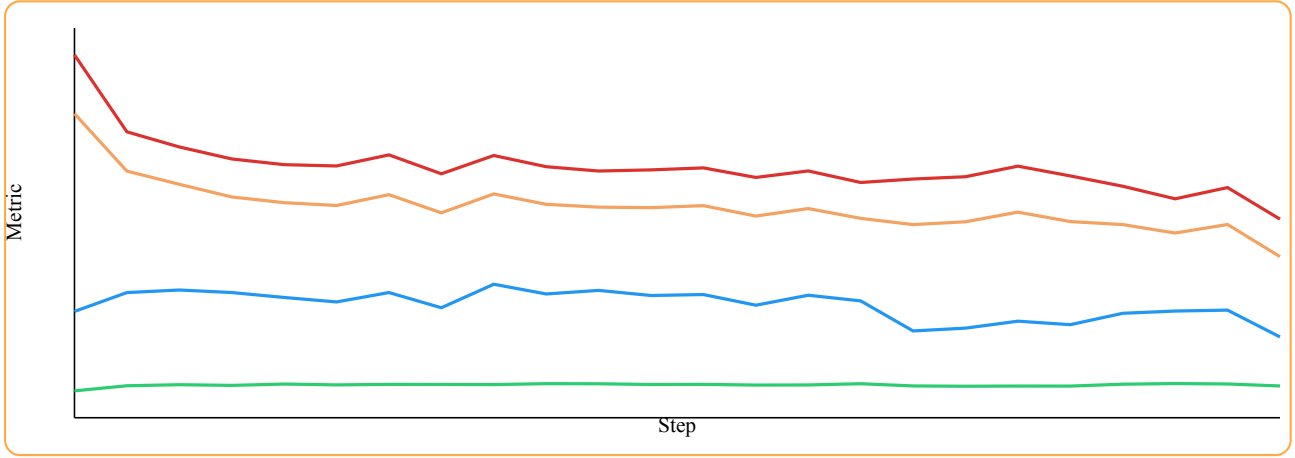
$$L_{\text{total}} = \frac{0.7 * L_{\text{AR}} + L_{\text{Diff}}}{1.7} + 0.1 * \text{KL}(p_{\text{AR}} \parallel p_{\text{Diff}})$$



Sweep 3 - Aggressive agreement emphasis

This run applies the heaviest lambda penalty, which keeps the total loss higher throughout.

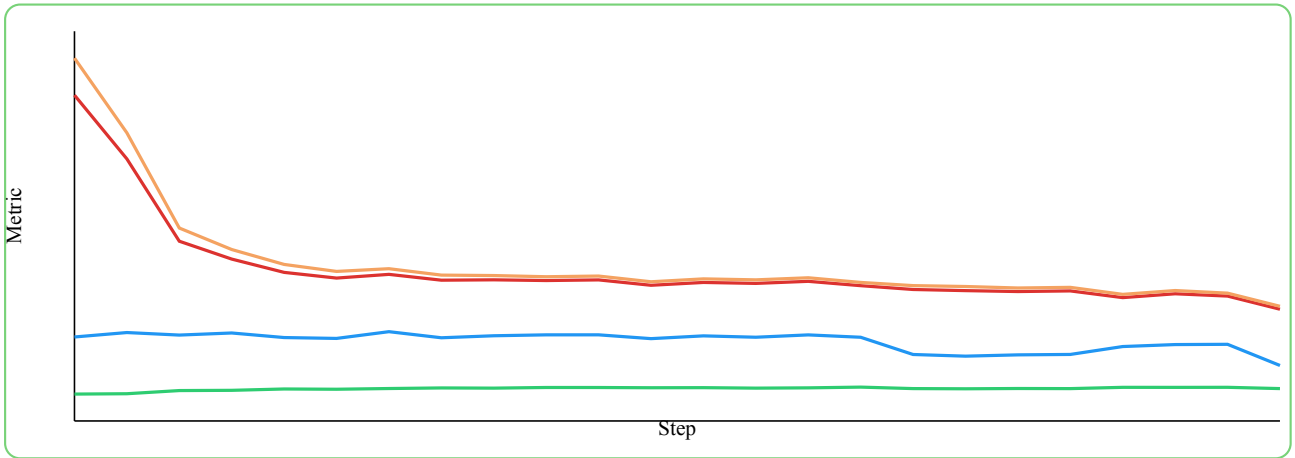
$$L_{\text{total}} = \frac{0.3 * L_{\text{AR}} + L_{\text{Diff}}}{1.3} + 0.5 * \text{KL}(p_{\text{AR}} \parallel p_{\text{Diff}})$$



Sweep 4 - Large-batch stabilization

Batch size is 4x larger with moderate alpha/lambda, giving a smoother loss trajectory.

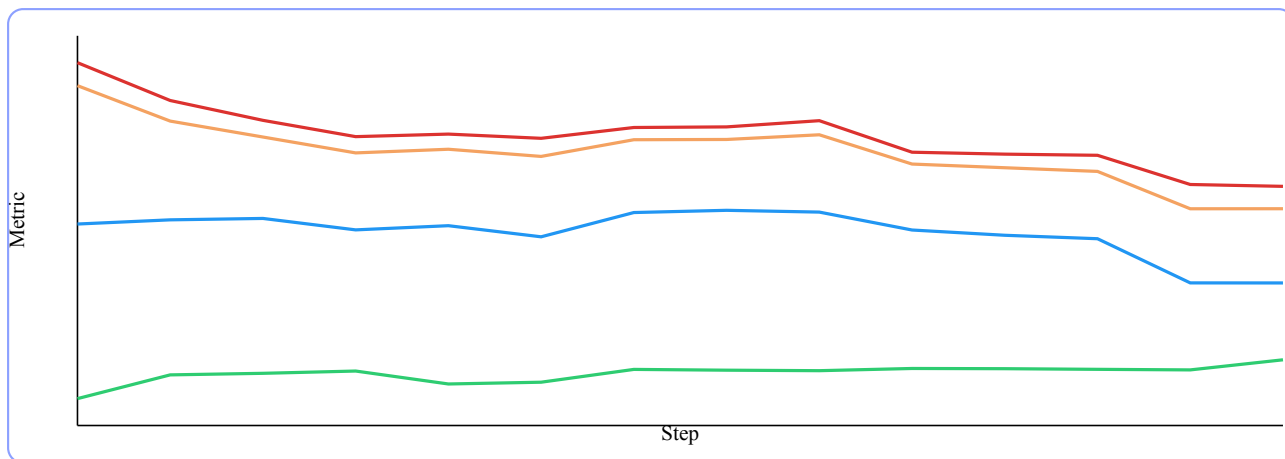
$$L_{\text{total}} = \frac{0.5 * L_{\text{AR}} + L_{\text{Diff}}}{1.5} + 0.2 * \text{KL}(p_{\text{AR}} \parallel p_{\text{Diff}})$$



Sweep 5 - Aggressive diffusion with KL temperature

Mixed-context dataset with higher lambda and temperature-scaled KL. Draft length reduced to 2.

$$L_{\text{total}} = \frac{0.2 * L_{\text{AR}} + L_{\text{Diff}}}{1.2} + 0.8 * \text{KL}(p_{\text{AR}} \parallel p_{\text{Diff}})$$



Greedy inference acceptance (local checkpoints)

Greedy decoding, temperature=0, top_k=0, stop_on_eos=false, prompt: “In a distant future, a curious robot loved math and”. Approx acceptance probability: $\frac{\text{avg_accept/iter} - 1}{K - 1}$.

Sweep	Checkpoint	K	Avg accept/iter	Approx acc prob
Sweep 1	step_0012298	5	4.85	0.96
Sweep 2	step_0012200	5	4.85	0.96
Sweep 3	step_0012200	5	4.85	0.96
Sweep 4	step_0003000	5	1.19	0.05
Sweep 5	step_0013903	2	1.21	0.21

Note: Sweeps 1-3 greedy outputs were largely empty/special tokens in this prompt, even though acceptance was high.

Takeaways

- Sweep 1 reaches the lowest total loss, but acceptance stays moderate; it is a strong baseline for quality.
- Sweep 2 balances loss and acceptance with a mild lambda, making it a likely candidate for higher acceptance without heavy degradation.
- Sweep 3 emphasizes agreement too aggressively, producing higher total loss and lower acceptance gains.
- Sweep 4 shows the most stable curve under large batches, so it is a good throughput-focused option.
- Sweep 5 shows the highest training-time acceptance on the mixed-context run, but greedy inference acceptance still trails the best-case runs.

TinyStories TiDAR Greedy Acceptance Comparison

90k+ branch runs from stable (plus one short early run overlay):

- **Stable (blue)**: $\alpha=1$, $\beta=1$ – baseline AR+Diff training, continued to 121k
- **KL-only (red)**: $\alpha=0.2$, $\beta=0$, $\rho=0.5$, $\delta=0.3$ – forward KL + greedy, no diff loss
- **KL-keep (green)**: $\alpha=1$, $\beta=1$, $\rho=0.1$, $\delta=0.3$ – forward KL + greedy, keeps AR+Diff
- **Distill (purple)**: $\alpha=1$, $\beta=1$, $\eta=0.04$, $T=2$ – soft distillation AR->Diff
- **Greedy Eta (orange)**: $\alpha, \beta=0.1$, $\delta=3.0$, $\eta=1.0$, $T=0.7$ – aggressive agreement + distill
- **Stable bigger beta (teal)**: $\alpha=1$, $\beta=5$ – baseline with 5x diffusion weight
- **Top-K set (brown)**: $\gamma=0.01$, $\gamma_{\text{topk}}=8$ – top-k set distillation (later-stage)
- **Big Gamma+Delta (indigo)**: $\gamma=5$, $\gamma_{\text{topk}}=4$, $\delta=1$ – strong top-k + hard agreement
- **Masked Delta later (magenta)**: partial 90k+ run, plotted up to available checkpoints
- **Delta masked early (cyan)**: short run (under 30k), overlaid on stable full-range Diff/Greedy charts

Legend

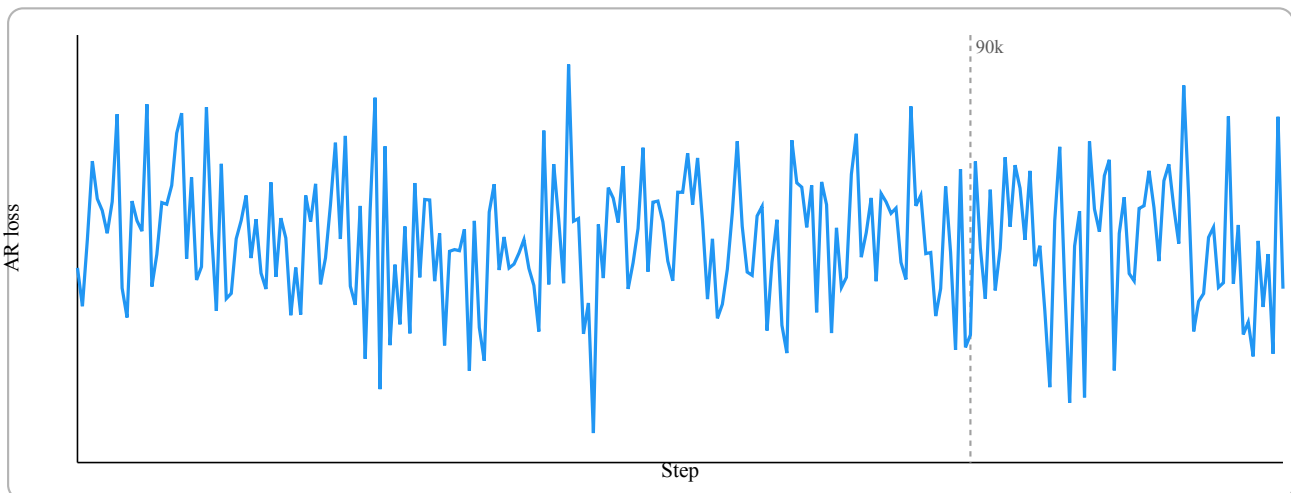
■ Stable (1, 1, 0, 0, 0, 0)	■ KL-keep (1, 1, 0.1, 0, 0.3, 0)	■ Greedy eta (0.1, 0.1, 0, 0, 3.0, eta=1.0 T=0.7)	■ Top-K set (gamma=0.01 K=8)	■ Masked Delta later (90k+ partial)
■ KL-only (0.2, 0, 0.5, 0, 0.3, 0)	■ Distill (1, 1, 0, 0, 0, eta=0.04 T=2)	■ Stable bigger beta (1, 5, 0, 0, 0, 0)	■ Big Gamma+Delta (gamma=5 K=4 delta=1)	■ Delta masked early (under 30k)

Total cost of the experiment so far: 30.25\$

AR loss (raw)

Stable

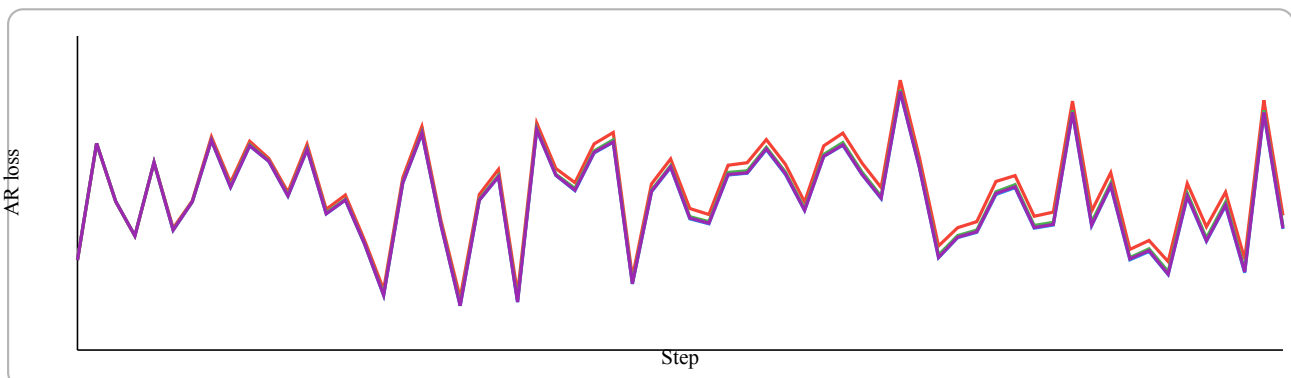
0-121k



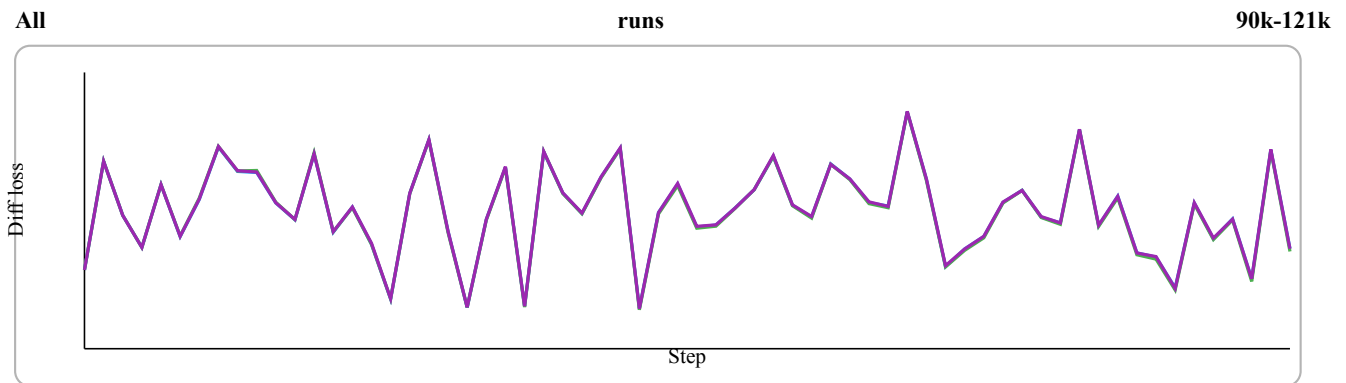
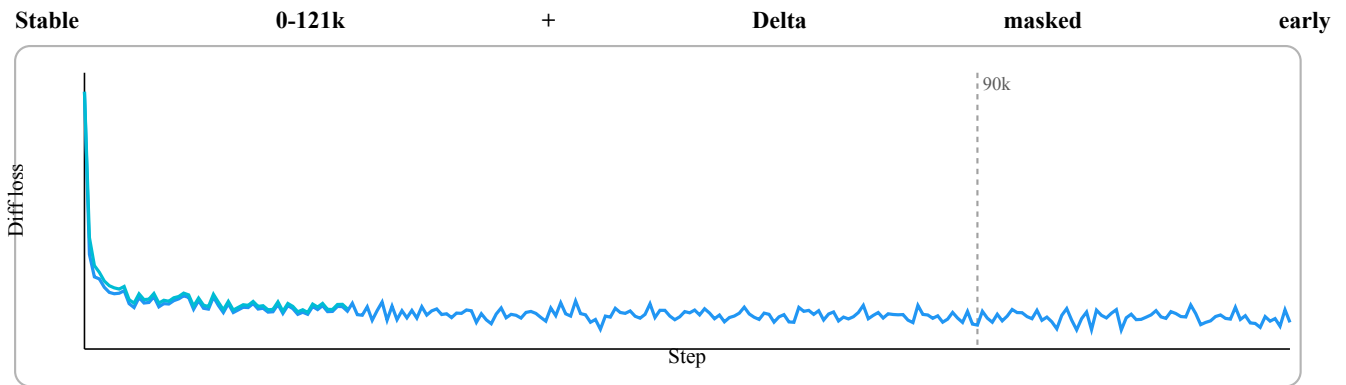
All

runs

90k-121k



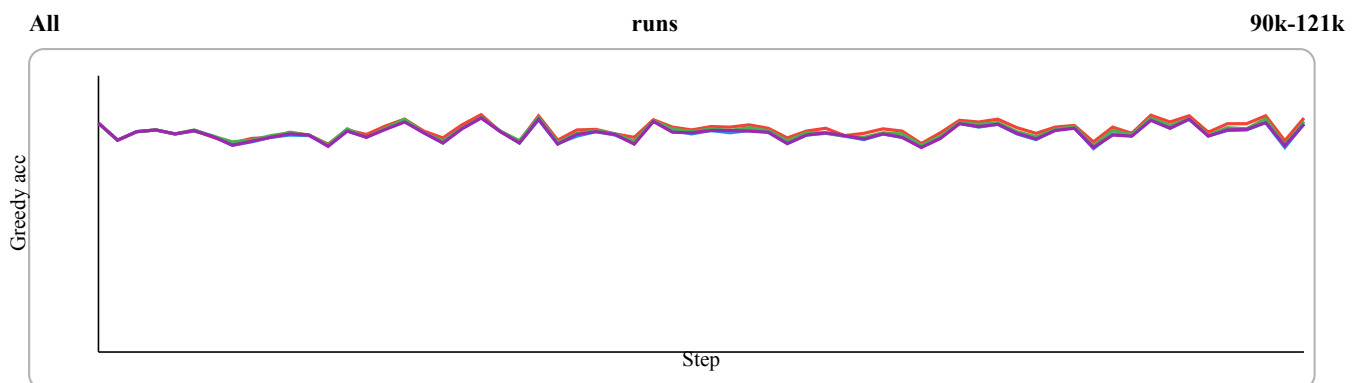
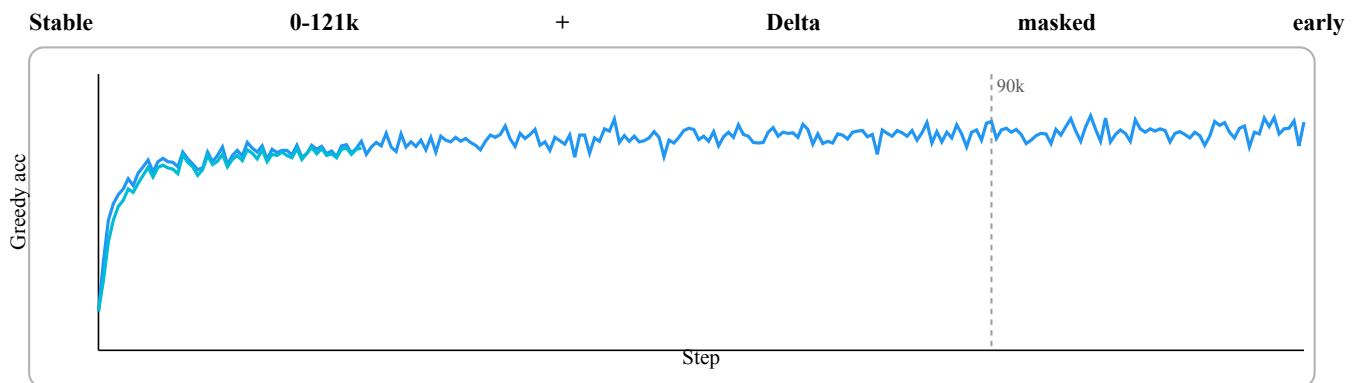
Diff loss (raw)



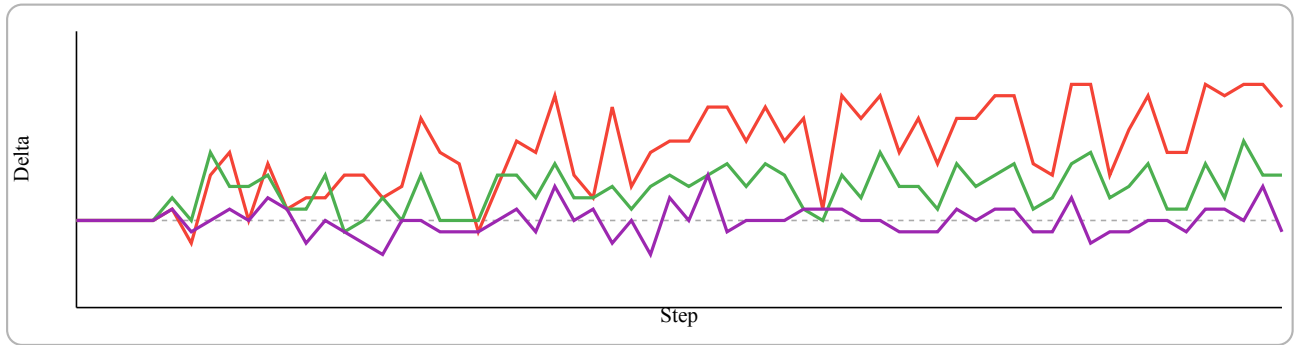
KL-only diff=0 after 90k, excluded from the zoomed panel.

Greedy acceptance

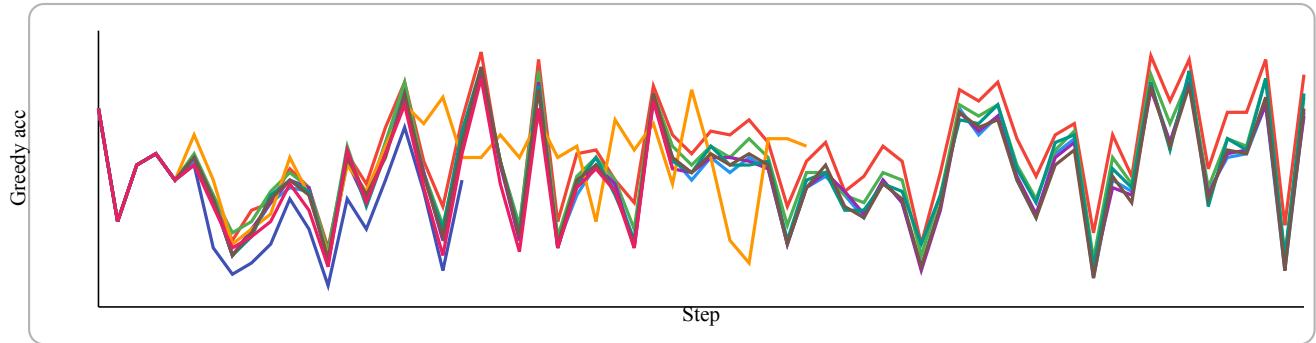
Greedy acceptance = fraction of positions where $\text{argmax}(\text{AR}) = \text{argmax}(\text{Diff})$.



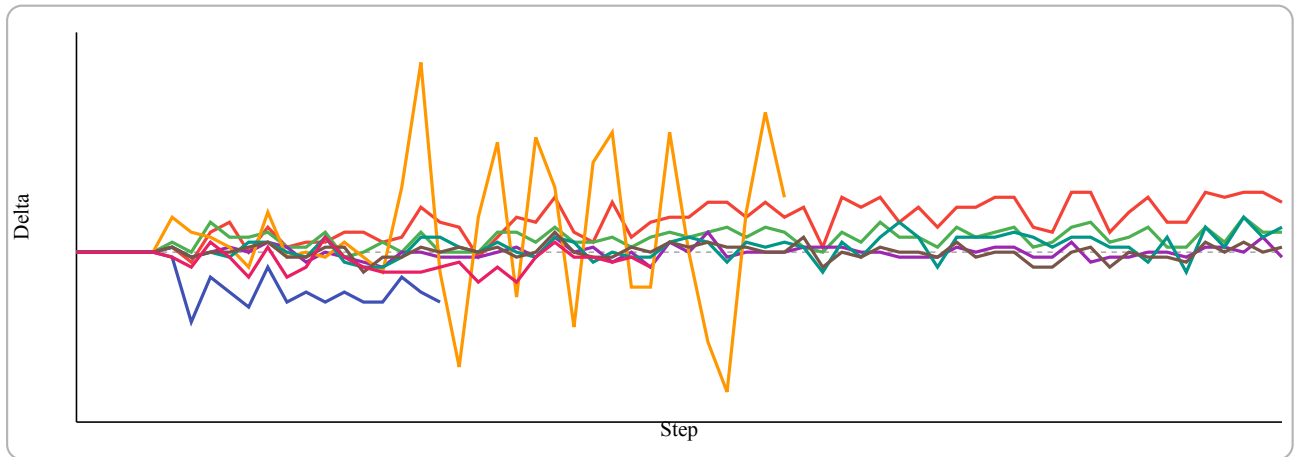
Delta vs stable (90k-121k)



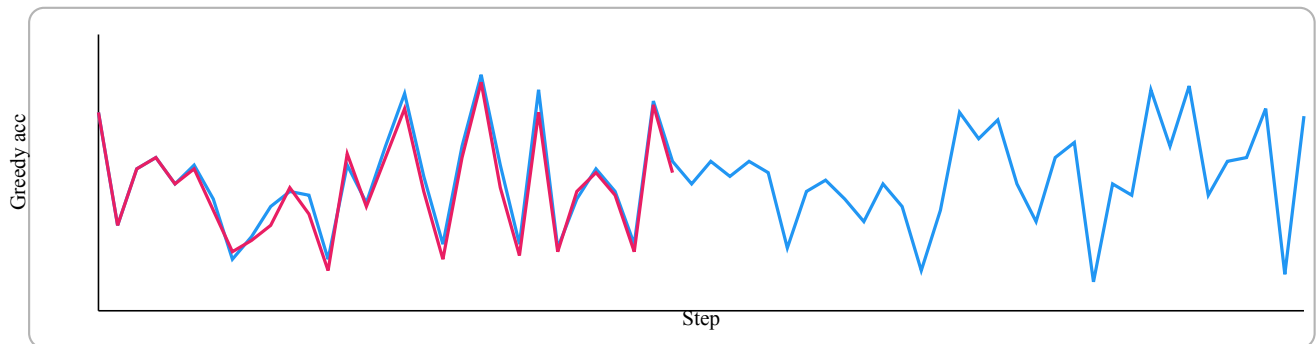
All runs 90k-121k (incl. smallAR + bigger beta + top-k + big gamma+delta + masked delta later)



Delta vs stable (90k-121k) (incl. smallAR + bigger beta + top-k + big gamma+delta + masked delta later)



Masked Delta later focus vs Stable (90k-121k, partial run)



Inference Results

Greedy decoding (temperature=0) with draft_len=6.

Prompts (10):

- P0: “There was a small village”
- P1: “Once upon a time, a little girl”
- P2: “The boy was very happy because”
- P3: “In the forest, a tiny fox”
- P4: “The cat and the dog were”
- P5: “One day, the teacher said”
- P6: “After school, the kids went”
- P7: “The robot wanted to learn”
- P8: “On a sunny morning, Emma”
- P9: “The brave knight looked at”

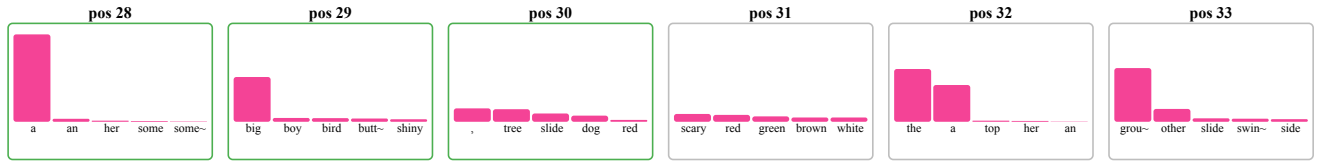
Accept/Iter summary (mean / best / worst across prompts)

Checkpoint	Steps=50	Steps=100	Steps=300
Stable @90k	2.41 / 3.50 / 1.63	2.26 / 2.75 / 1.71	2.10 / 2.45 / 1.88
Stable @121k	2.22 / 3.27 / 1.53	2.09 / 2.68 / 1.60	2.10 / 2.69 / 1.57
KL-only @121k	2.42 / 3.27 / 1.81	2.30 / 2.68 / 1.80	2.08 / 2.49 / 1.61
KL-keep @121k	2.40 / 3.27 / 1.75	2.29 / 2.83 / 1.80	2.18 / 2.90 / 1.75
Distill @121k	2.31 / 3.50 / 1.53	2.26 / 3.09 / 1.87	2.19 / 2.74 / 1.80
SmallAR big greedy eta @105k	2.15 / 2.88 / 1.75	2.15 / 2.75 / 1.90	2.00 / 2.43 / 1.74
Stable bigger beta @121.5k	2.37 / 3.27 / 1.63	2.33 / 3.09 / 1.62	1.96 / 2.27 / 1.74

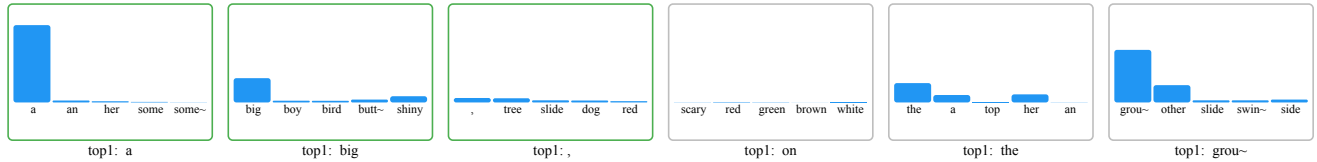
Inference Logits Histogram

Prompt: "Once upon" | Checkpoint: "step_0121566.npz" Target position: 30 | Block: 28-33 | Draft len: 6

AR verify logits (top-5, sorted by AR)



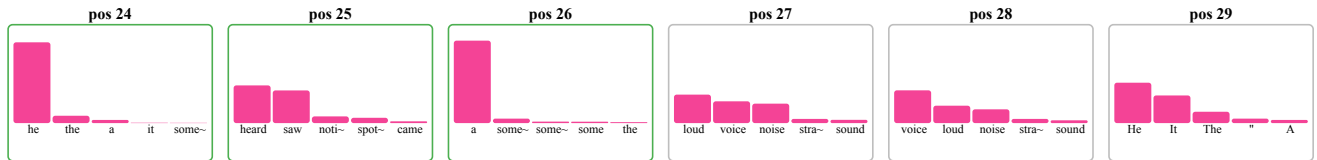
Diff draft logits (same tokens, AR order)



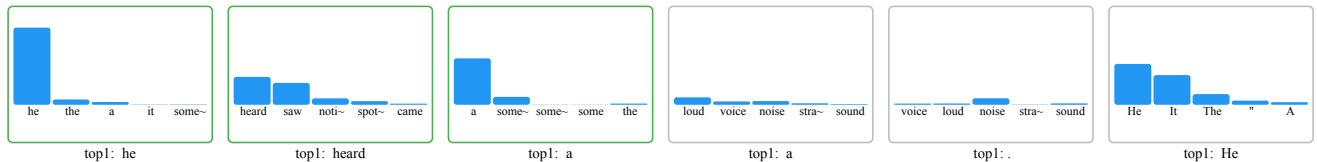
Sentence context One day, she saw a big, scary dog.

Prompt: "In the forest, a tiny fox" | Checkpoint: "step_0121566.npz" Target position: 27 | Block: 24-29 | Draft len: 6

AR verify logits (top-5, sorted by AR)



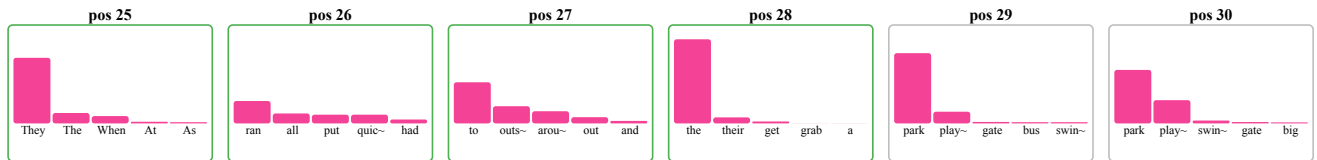
Diff draft logits (same tokens, AR order)



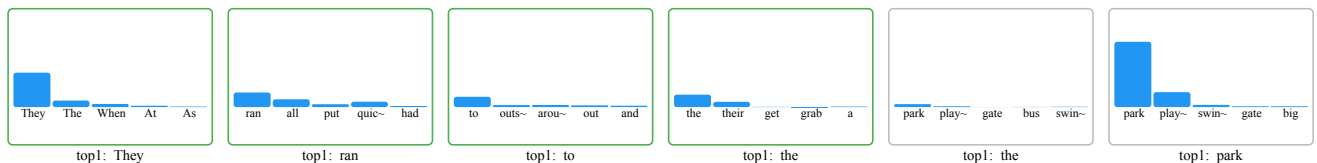
Sentence context Suddenly, he heard a loud noise.

Prompt: "One day, the teacher said" | Checkpoint: "step_0121566.npz" Target position: 29 | Block: 25-30 | Draft len: 6

AR verify logits (top-5, sorted by AR)



Diff draft logits (same tokens, AR order)



Sentence context They ran to the park and saw a big slide.

TiDAR Feature Card: Coefficient-Gated Loss Branch Pruning in JAX

What this feature is

In TiDAR training, each auxiliary loss term is optional and controlled by a coefficient:

- alpha: AR CE
- beta: Diffusion CE
- rho: forward KL
- chi: reverse KL
- delta: hard agreement
- eta: distillation KL
- gamma: top-k set distillation

The key implementation detail is:

- if a coefficient is 0.0, the corresponding branch is excluded from that compiled executable,
- if a coefficient is non-zero, that branch is included.

This gives a clean way to run many training variants without maintaining separate codepaths.

Why it matters

- It reduces unnecessary compute when ablation terms are off.
- It keeps one source of truth for all loss variants.
- It enables stage-wise objectives by changing only config values.
- It improves experiment velocity for a diploma workflow (fast toggles, same script).

Where it is implemented

Python-side branch gating in TiDAR/model/Training_step.py:

```
1  if alpha > 0.0:
2      ... # AR CE
3
4  if beta > 0.0:
5      ... # Diffusion CE
6
7  if (rho > 0.0) or (chi > 0.0) or (delta > 0.0) or (eta > 0.0):
8      ... # Alignment family
9
10 if (gamma > 0.0) and (gamma_topk > 0):
11     ... # Top-k set distillation
```

Static arguments in TiDAR/model/Run_training.py inside jitted _run_chunk:

```
1  @partial(
2      jax.jit,
3      static_argnames=(
4          "alpha", "beta", "rho", "chi",
5          "delta", "eta", "eta_T", "gamma", "gamma_topk",
6      ),
7  )
8  def _run_chunk(...):
9      ...
```

Because coefficients are static, each unique tuple of values gets its own compiled variant.

Verification summary (compile behavior)

Probe script run with:

- JAX_LOG_COMPILES=1

- JAX_EXPLAIN_CACHE_MISSES=1

Observed behavior:

- First call with $\gamma=0.0$ compiled once.
- Second call with $\gamma=0.0$ reused cache.
- Switching to $\gamma=0.1$ caused a new compile (static-arg cache miss).
- Second call with $\gamma=0.1$ reused cache.

Raw log excerpt from the probe run:

```

1  $ JAX_LOG_COMPILES=1 JAX_EXPLAIN_CACHE_MISSES=1 /Users/antonhrstov/v/SG/bin/python /
  tmp/tidar_compile_probe.py
2  ...
3  WARNING:jax._src.pjit: TRACING CACHE MISS ...
4  never seen function: compiled_loss_and_grad
5  WARNING:jax._src.interpreters.pxla: Compiling jit(compiled_loss_and_grad) ...
6  ...
7  WARNING:jax._src.pjit: TRACING CACHE MISS ...
8  all previously seen cache keys are different.
9  key with different value of static kwargs:
10 now {... gamma: 0.1, gamma_topk: 8, ...}
11 before {... gamma: 0.0, gamma_topk: 8, ...}
12 WARNING:jax._src.interpreters.pxla: Compiling jit(compiled_loss_and_grad) ...

```

Interpretation:

- The first miss is expected for first-ever trace/compile.
- The second miss is specifically due to static argument change (γ), proving a new compiled variant.
- No additional compile lines appear for the repeated same-coefficient calls in that run.

JAXPR check also confirmed pruning for the top-k branch:

- top_k primitive count with $\gamma=0.0$: 0
- top_k primitive count with $\gamma=0.1$: 1

Practical implication for experiments

For TiDAR ablations, setting a coefficient to zero is not only a semantic “off” switch. It also creates a lighter compiled graph for that run configuration.

This is useful for staged training, where early stages can focus on core AR+Diff losses, and later stages can enable selected agreement terms only when needed.

Config examples

```

1  # example config with 3 losses
2  training:
3    loss:
4      alpha: 1.0
5      beta: 1.0
6      rho: 0.0
7      delta: 0.5
8      gamma: 0.0
9      gamma_topk: 4

```

`gamma_topk` should be paired with `gamma`.

- If $\gamma = 0.0$, top-k set loss is disabled regardless, so `gamma_topk` is effectively unused.
- If top-k set loss is enabled, use a non-zero pair like: